

Training Manual for Introduction to PSOC™ Edge E84 Getting Started with the Development Ecosystem

About this document

Scope and purpose

This document is a training manual for the PSOC™ Edge E84 MCU. This Manual helps in getting started with PSOC™ Edge E84 MCU applications using Modus Toolbox™ software

Table of contents

Contents

About this document	1
Table of contents	1
1 Introduction	2
2 Required development tools	2
3 Prerequisites	2
4 Code Example 1: Hello world + Blinky	3
4.1 Objective	3
4.2 Description.....	3
4.3 Flowchart.....	3
4.4 Hardware Block Diagram and Setup	5
4.5 Project Creation.....	6
4.6 Conclusion	32
5 Code Example 2: IPC Semaphore using PDL	33
5.1 Objective	33
5.2 Description.....	33
5.3 IPC Semaphore Code Example Flowchart.....	33
5.4 Hardware Setup.....	35
5.5 Project Creation.....	35
5.6 Conclusion.....	56

1 Introduction

This manual provides specific instructions to create, configure , build and run several application code examples on the **PSOC™ Edge E84**.

2 Required development tools

1. ModusToolbox™ software v3.6 or later (tested with v3.6)
 - ModusToolbox™ Programming Tools v1.6.0
2. Board support package (BSP) minimum required version for:
 - KIT_PSE84_EVAL_EPC2: v1.2.0
3. PSOC™ Edge E84 Evaluation Kit (KIT_PSE84_EVAL Rev G)
 - Ensure BOOT SW (SW6) should be in 'High'/ON position.
 - i. **Note:** For KIT_PSE84_EVAL rev. G, BOOT SW pin is BOOT.1 (P17.6)
 - Ensure J20 and J21 should be in Tristate/Not-Connected (NC) position.
4. Serial Terminal such as Tera Term, PuTTY, or the Serial Terminal in the Eclipse IDE

3 Prerequisites

1. [Introduction to PSOC™ Edge E84 \(E1\)](#) video training session. Note you must be logged into Infineon.com for this link to the video traing session to work.
2. Make sure the required development tools are installed.

4 Code Example 1: Hello world + Blinky

4.1 Objective

This code example will teach you to create a simple UART communication by printing the “Hello World” message on a terminal and blinking an LED using a timer resource in PSOC™ Edge E84. The code example uses the PDL (Peripheral Device Layer) API reference library.

4.2 Description

This code example demonstrates the following algorithm flow:

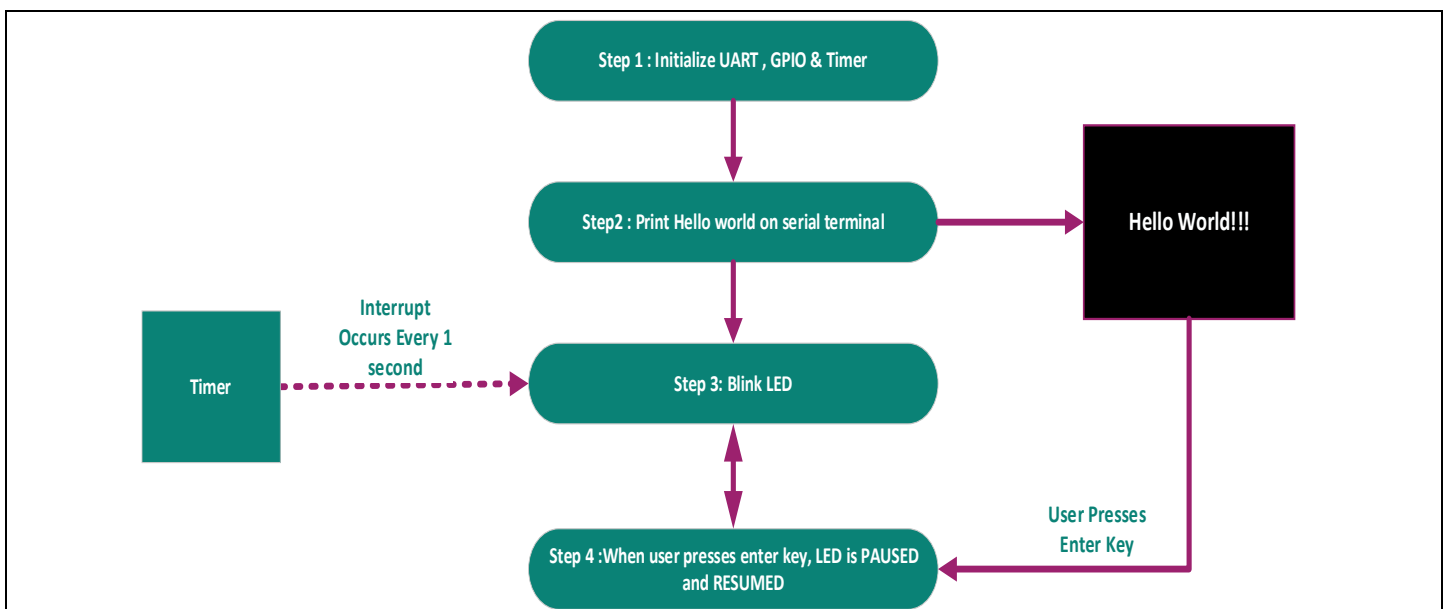


Figure 1 Hello World + Blinky Algorithm Flow

4.3 Flowchart

Below flow chart explains the flow of CM33 non-secure project

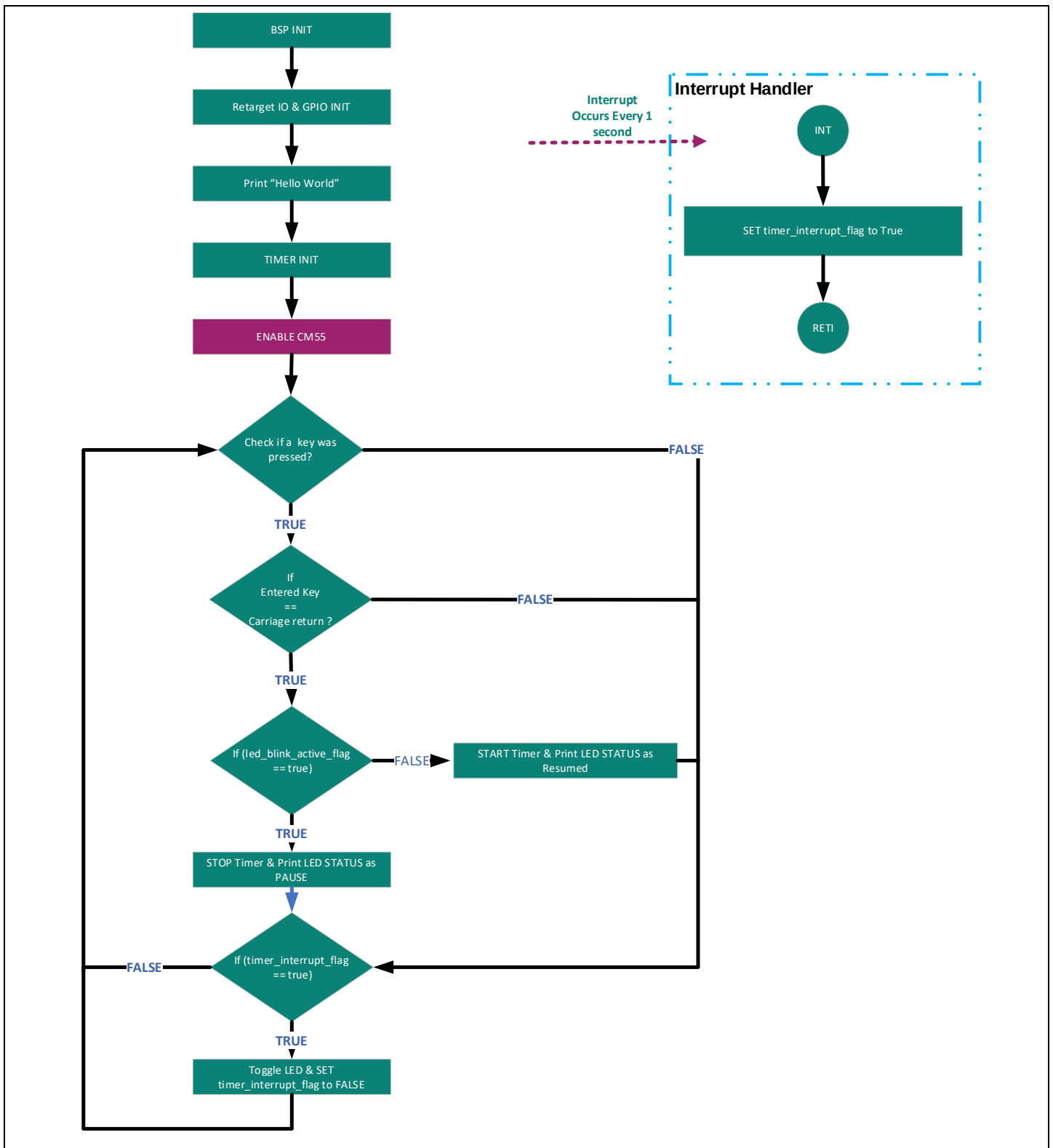


Figure 2 CM33_ns Flow Diagram

The below flow chart shows the flow of CM55 project, which is just to initialize the device and then enter a deepsleep.

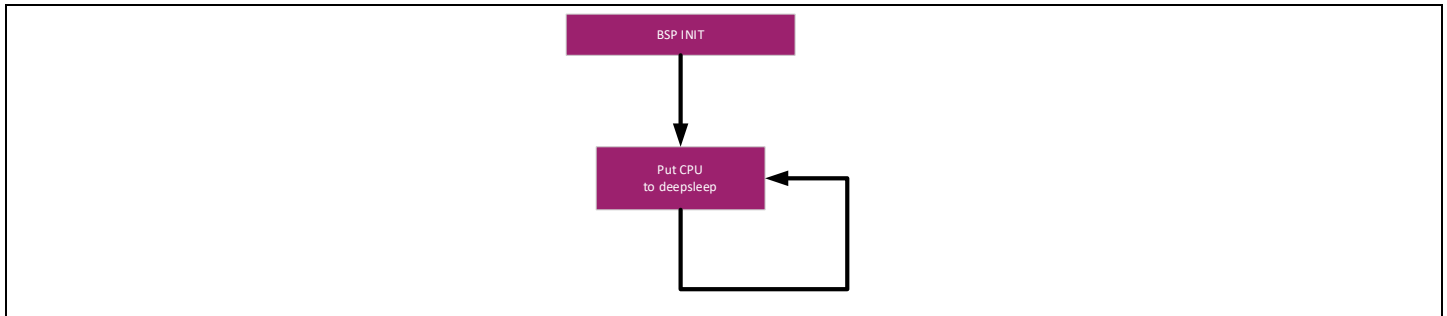


Figure 3 CM55 Flow Diagram

4.4 Hardware Block Diagram and Setup

Below is the Hardware block diagram for Hello world + Blinky example as per the schematic

UART pins:

- P6_5 (UART_RX)
- P6_7 (UART_TX)

GPIO LED Pin

- P16_7 (USER_LED)

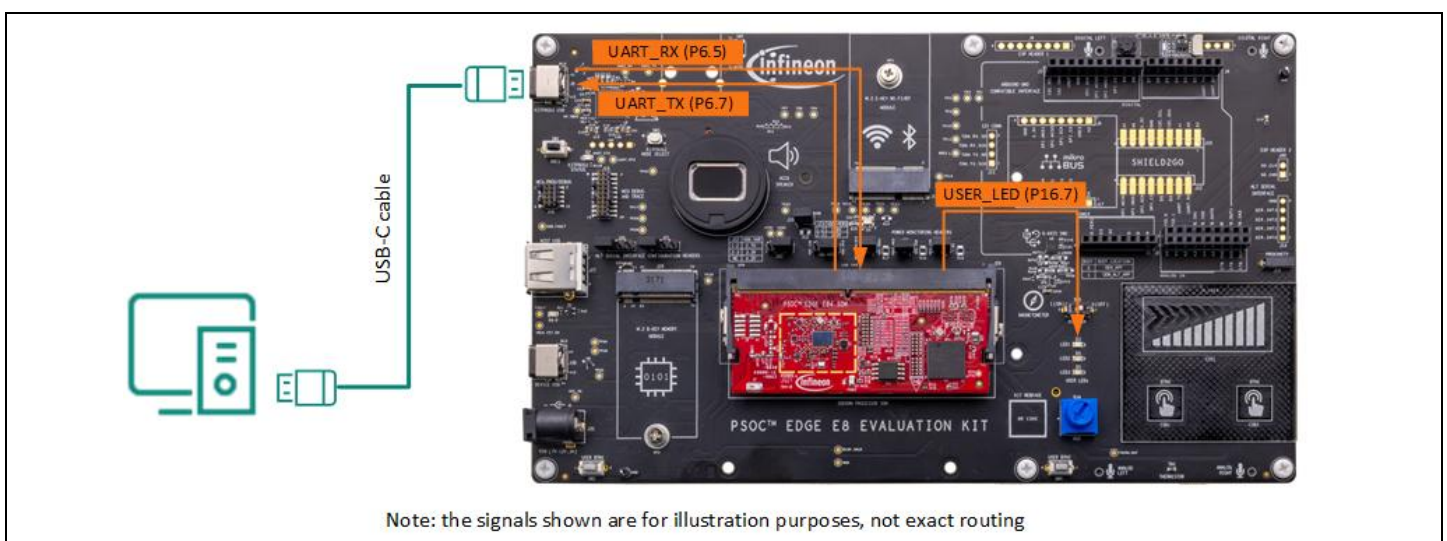


Figure 4 Hardware Diagram for UART and LED Routing on PSOC™ Edge E84 EVK

4.5 Project Creation

1. Before creating the first project we need to create our workspace in Modus™ Toolbox. Open **Modus™ Toolbox** from the **Windows Start** menu.

Note: if you have not installed **Modus tool box**, Refer [Required development tools](#) section

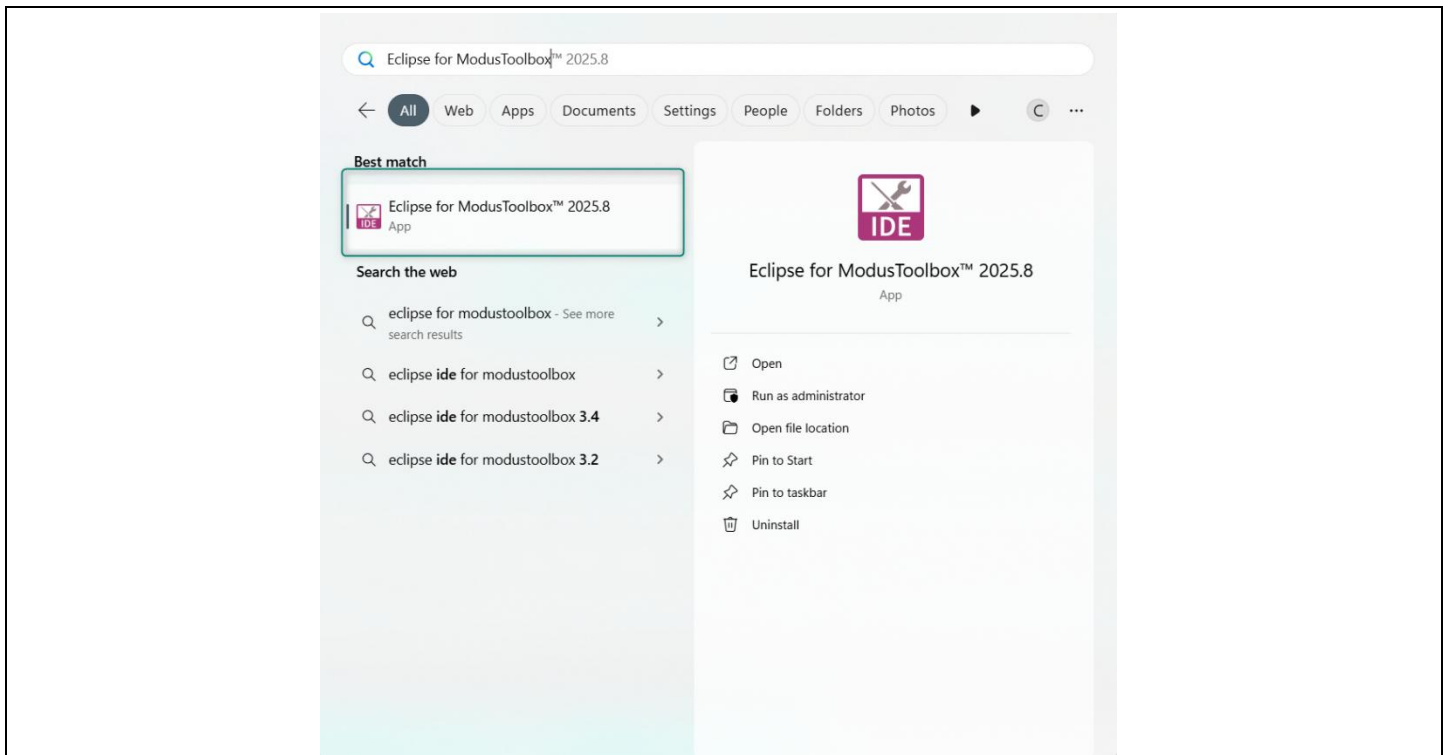


Figure 4 Launch Eclipse for ModusToolbox™

2. **Next**, you must choose a **workspace directory** for your project. After selecting a suitable directory, click the Launch button.

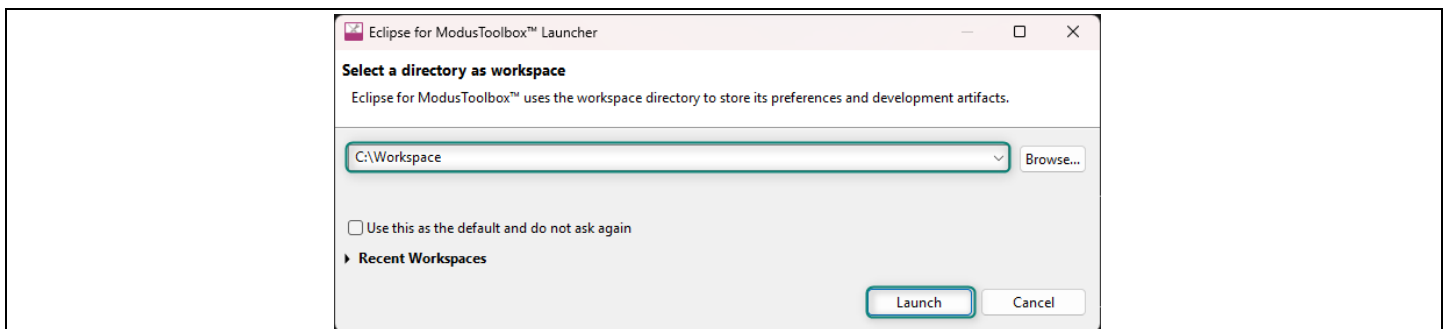


Figure 5 Select workspace directory

Introduction to PSOC™ Edge E84

Getting Started with the Development Ecosystem

Code Example 1: Hello world + Blinky

3. Hitting Launch will open a new ModusToolbox™ workspace as shown below.

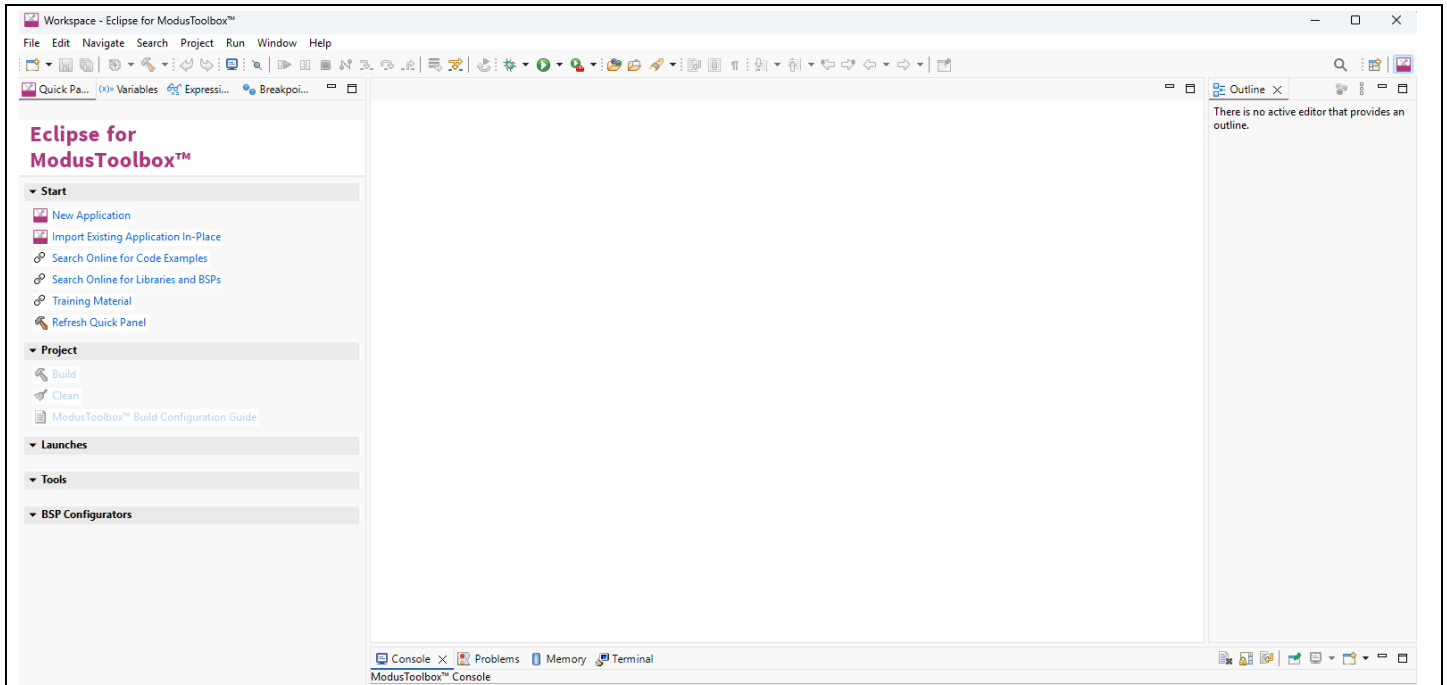


Figure 6 Initial View of Eclipse workspace

4. Now select **New Application** from the Quick Panel. Alternatively, go to **File> New> Modus Toolbox™ Application**

Introduction to PSOC™ Edge E84

Getting Started with the Development Ecosystem

Code Example 1: Hello world + Blinky

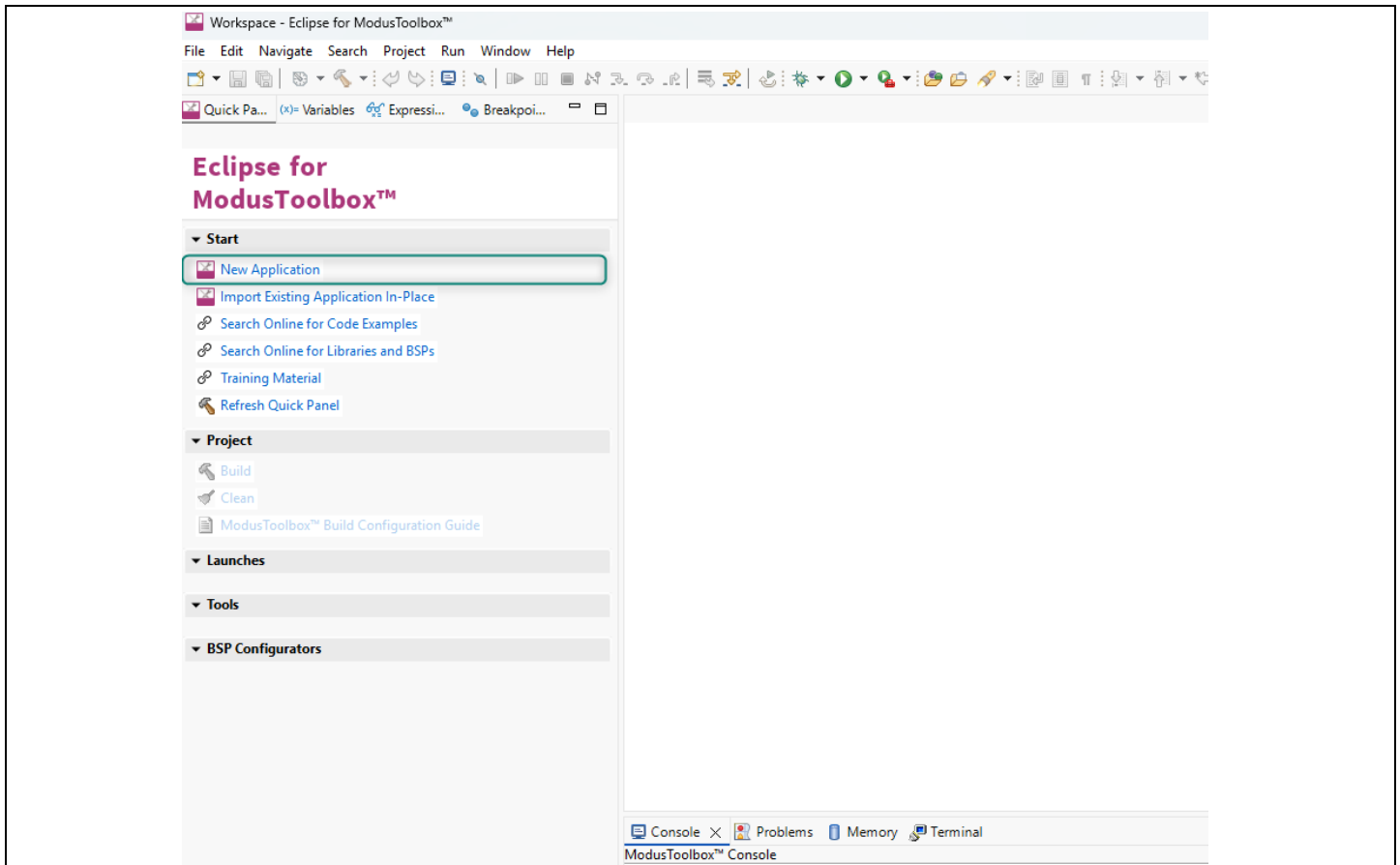


Figure 7 Create a New Application from the Quick Panel Start section

5. Clicking New Application will open the **Project Creator Tool** and prompt you to choose a **BSP** (Board Support Package).

BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.

6. Once **Projector creator** launches, Select the **KIT_PSE84_EVAL_EPC2** BSP under the PSOC™ Edge BSPs (or Detected Devices if your EVK is already connected). Then click **Next**.

Introduction to PSOC™ Edge E84

Getting Started with the Development Ecosystem

Code Example 1: Hello world + Blinky

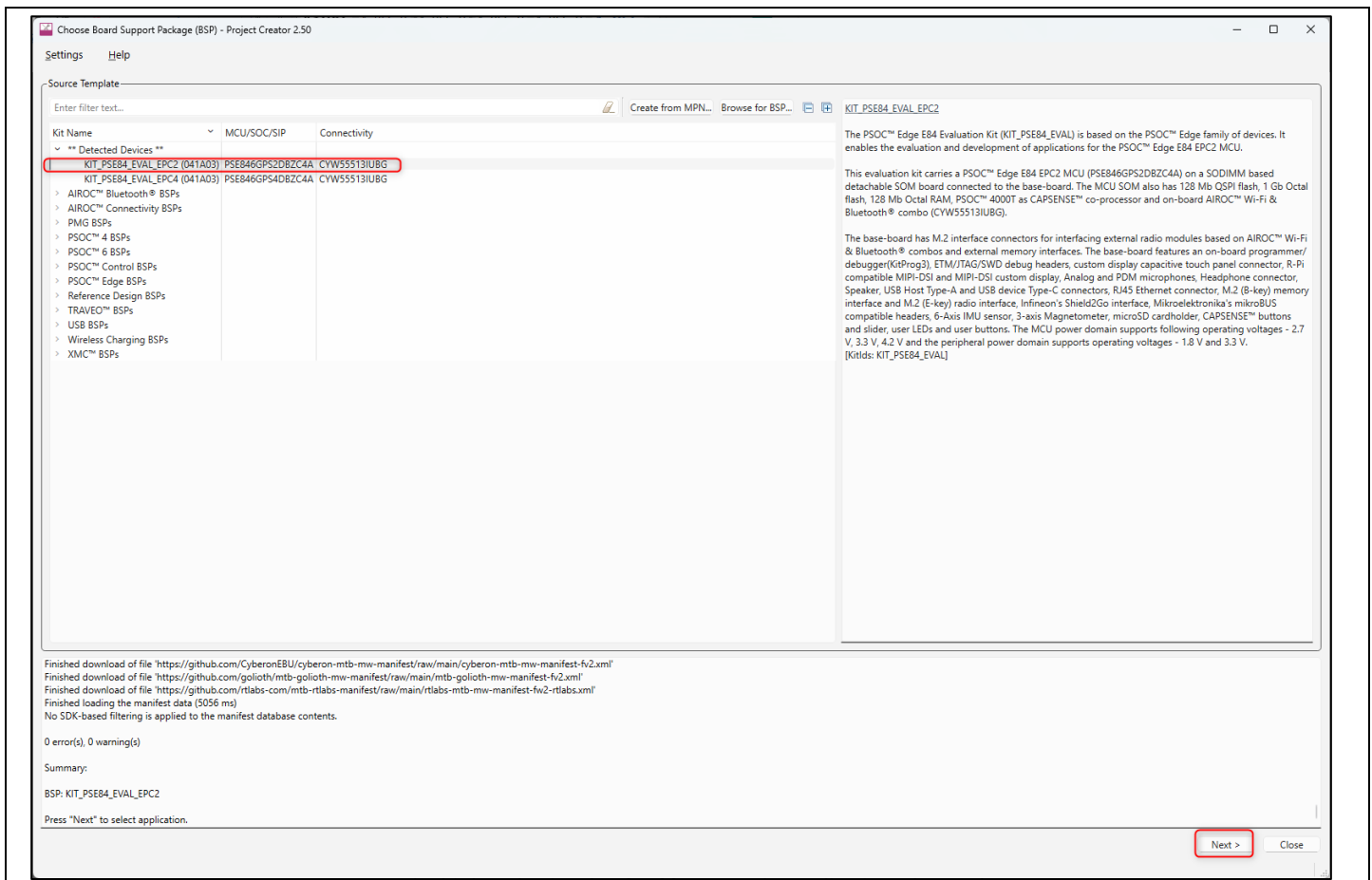


Figure 8 Choose BSP

- Clicking Next will take you to the next menu of the Project Creator which will ask you to select an Application. Select the **PSOC Edge Empty Application** under the Getting Started section. You can do this by selecting the **checkbox** near the application. Change the name to **"PSOC_Edge_Hello_World"** as we will be developing a hello world application and click on **Create**

Introduction to PSOC™ Edge E84

Getting Started with the Development Ecosystem

Code Example 1: Hello world + Blinky

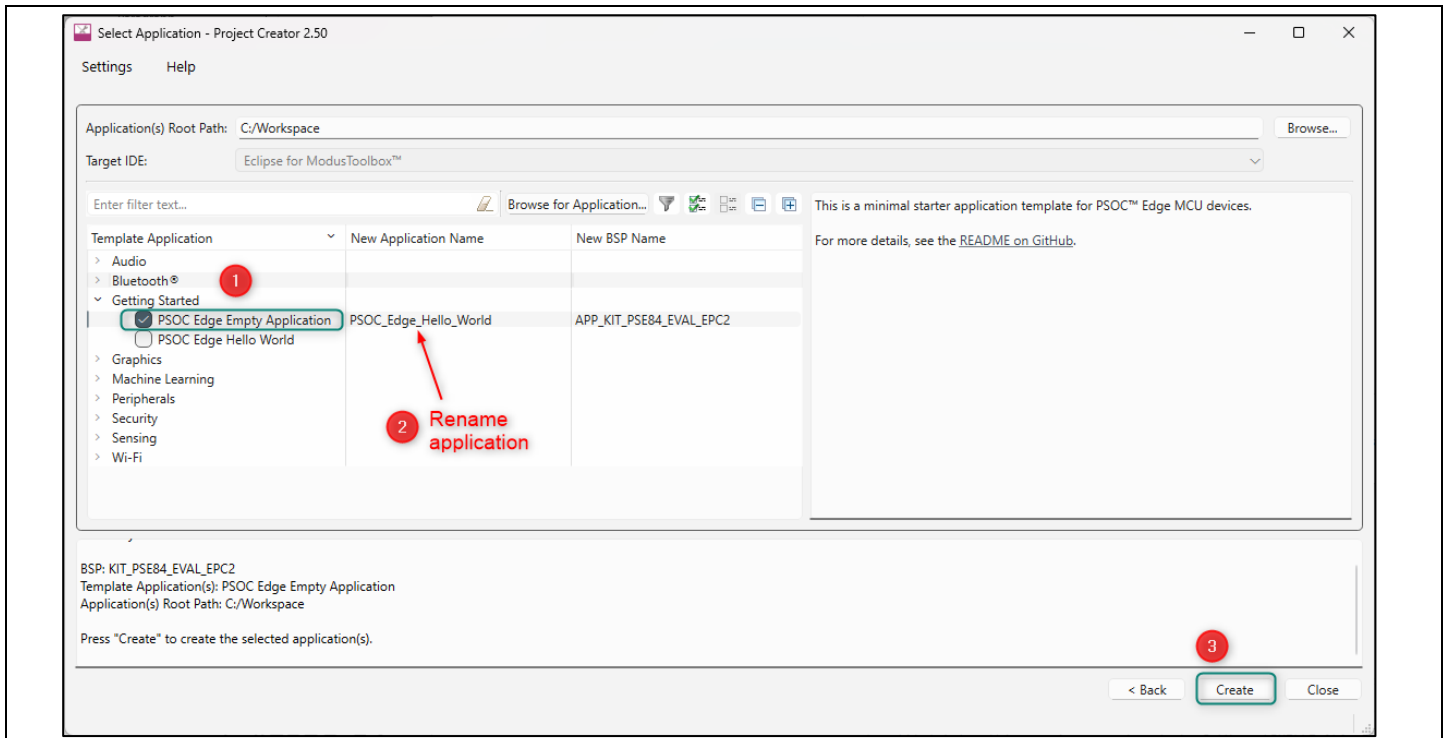


Figure 9 Select Template Application – Getting Started – PSOC Edge Empty Application

- When you click **Create**, a new project is created, and you will be able to see all the **files** in the Project Explorer window.

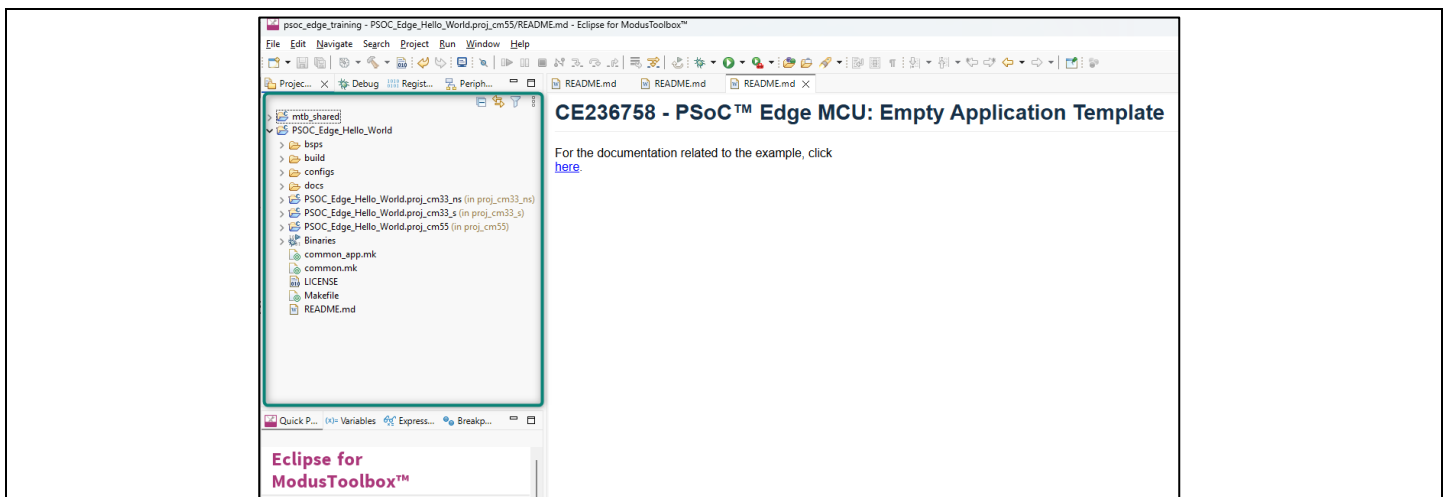


Figure 10 Project Explorer View

- Expanding the main project folder, you can see that there are multiple sub-folders within it.

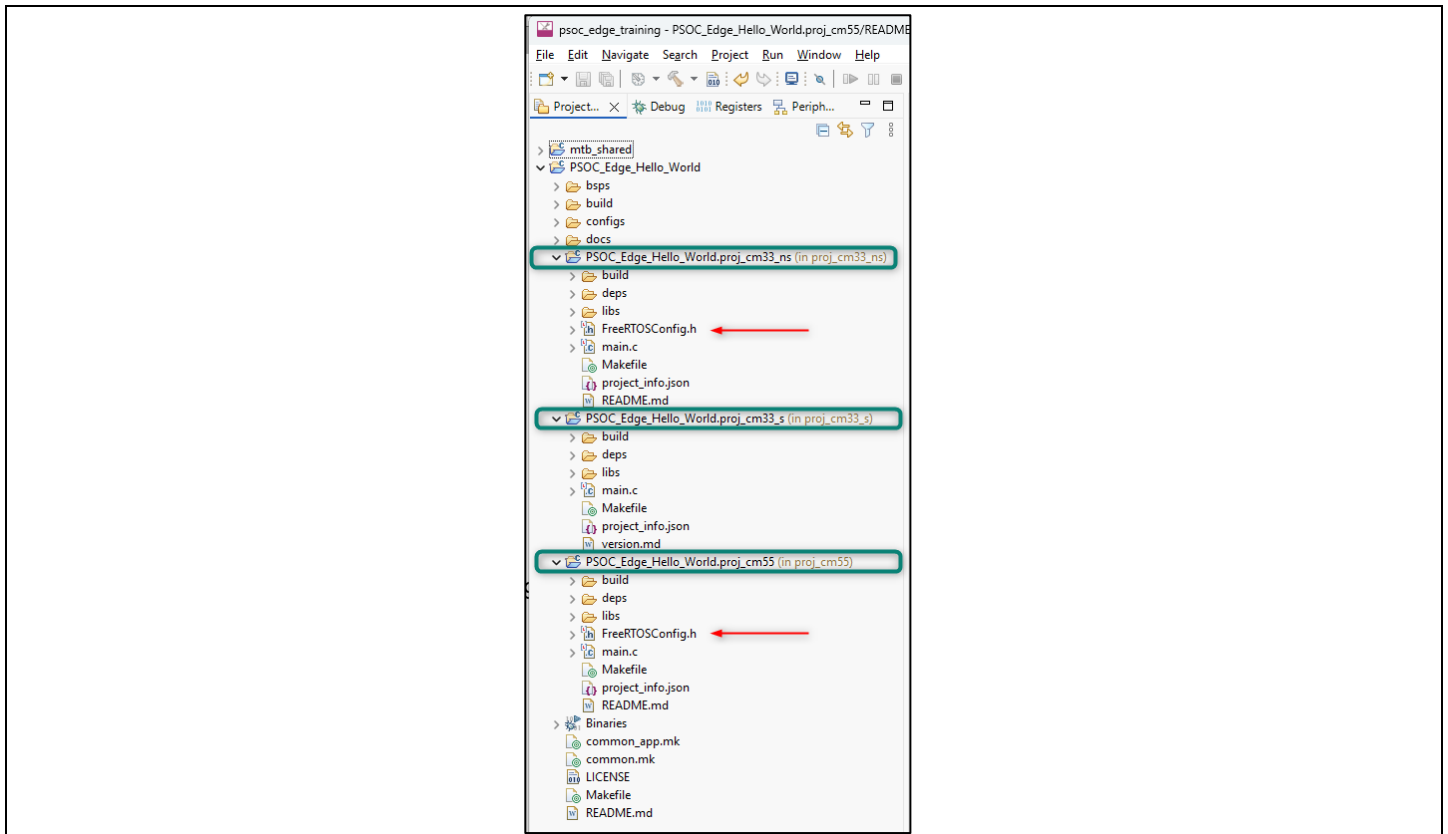


Figure 11 Project folders of a PSOC Edge Applications with included FreeRTOS libraries

Exploring the folders, you will see the FreeRTOS libraries have been imported into the project. But, since we are making a basic hello world blinky project, the FreeRTOS library is not needed. We will remove it using the **Library Manager** tool.

The **Library Manager** tool can be found in the **Quick Panel** tab directly below the **Project Explorer** in the **Tools** section. In addition, **retargetIO** will be needed for our project for using the UART functionalities. This will be added using the **Library Manager** as well.

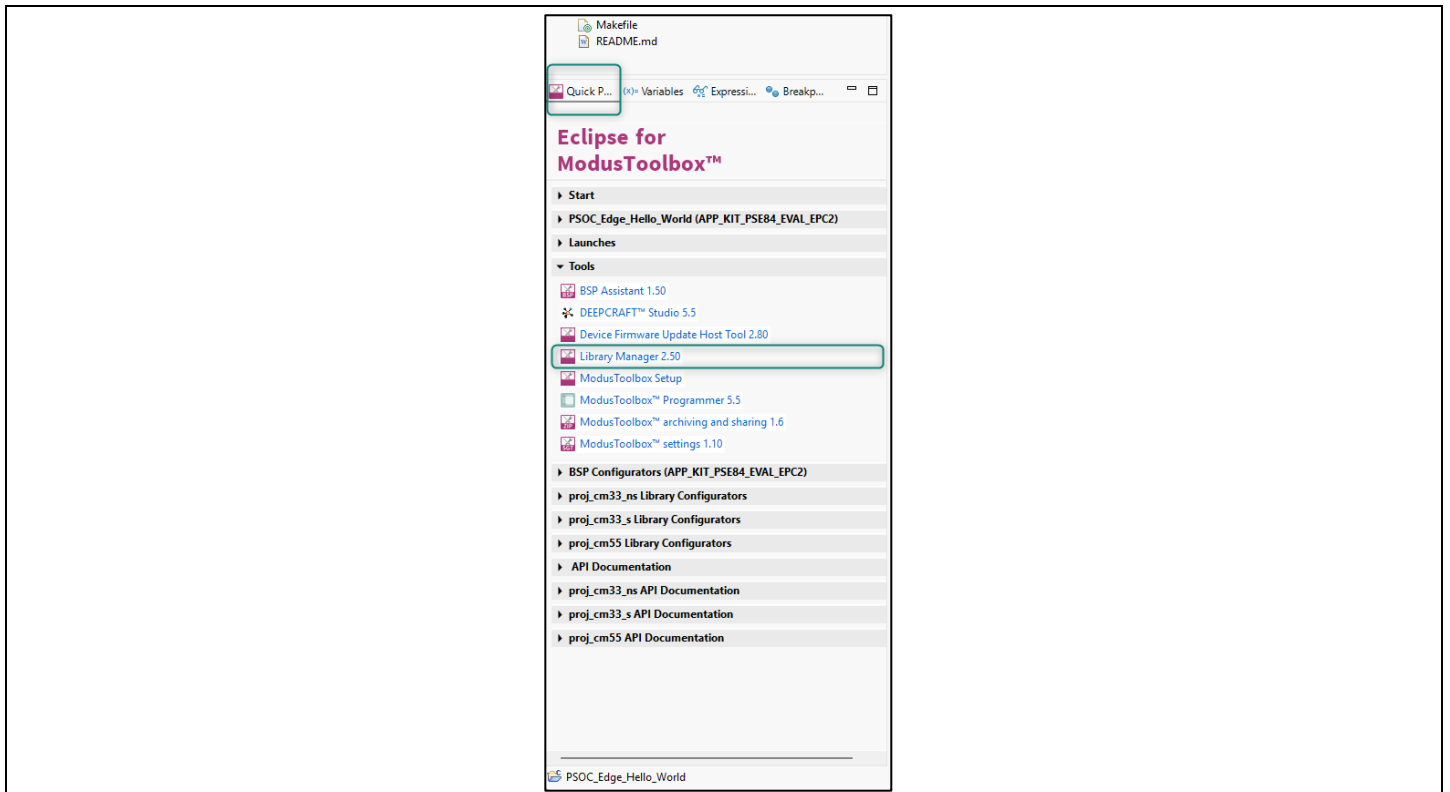


Figure 12 Library Manager

10. Select the **PSOC_Edge_Hello_World** project in the mtb project explorer and **Open the Library Manager** from **Quick Panel**. Find the **freertos** libraries under the **proj_cm33_ns** and **proj_cm55** project libraries. Click the 'X' mark to remove the **freertos** library in each project's libraries.

Introduction to PSOC™ Edge E84

Getting Started with the Development Ecosystem

Code Example 1: Hello world + Blinky

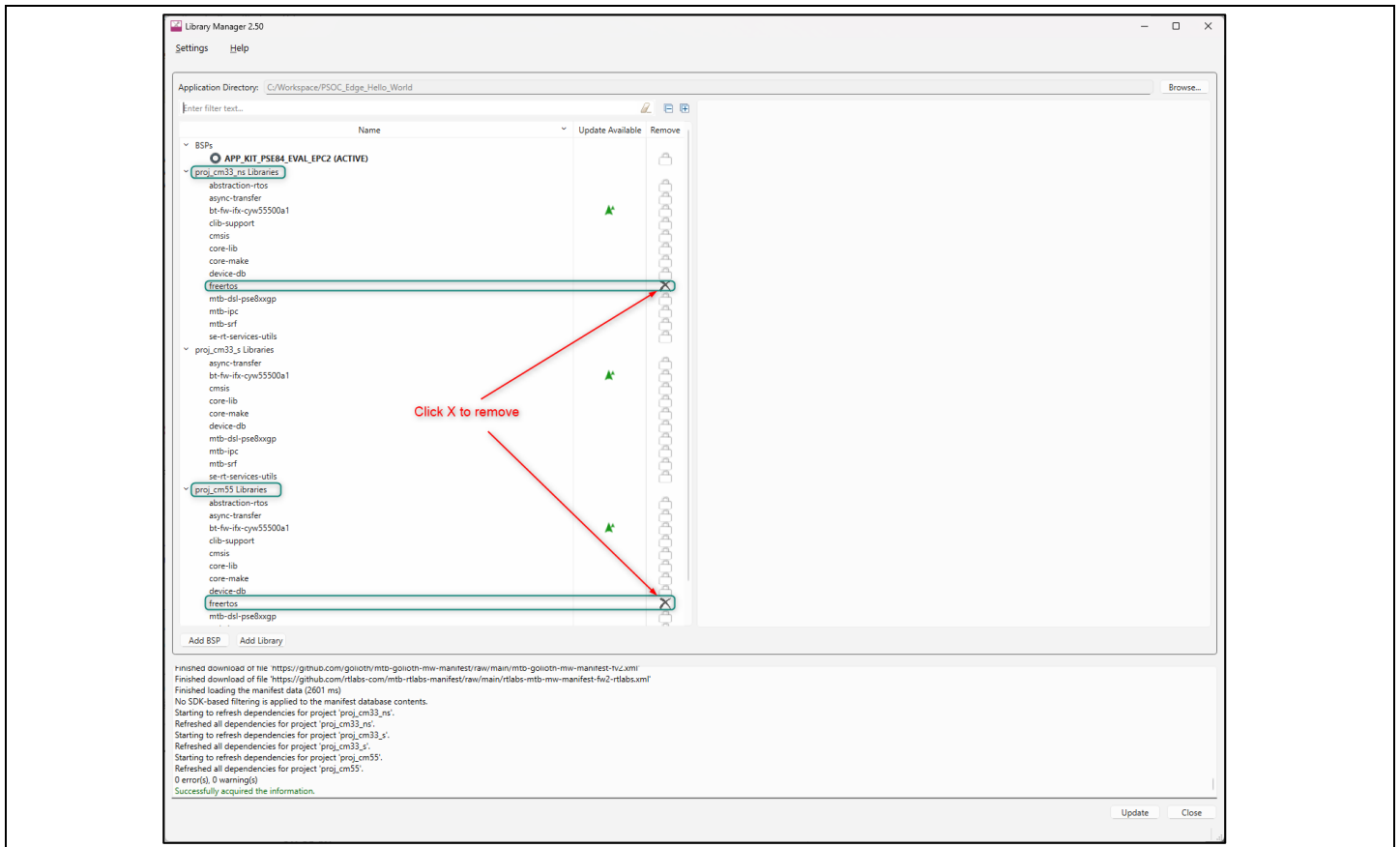


Figure 13 Removing freertos from cm33_ns and cm55

Now, click the **Add Library** option. A menu will open. Search for **retarget-io**. It can be found under the Peripheral section. Select the **retarget-io** option for **proj_cm33_ns** and click **OK**.

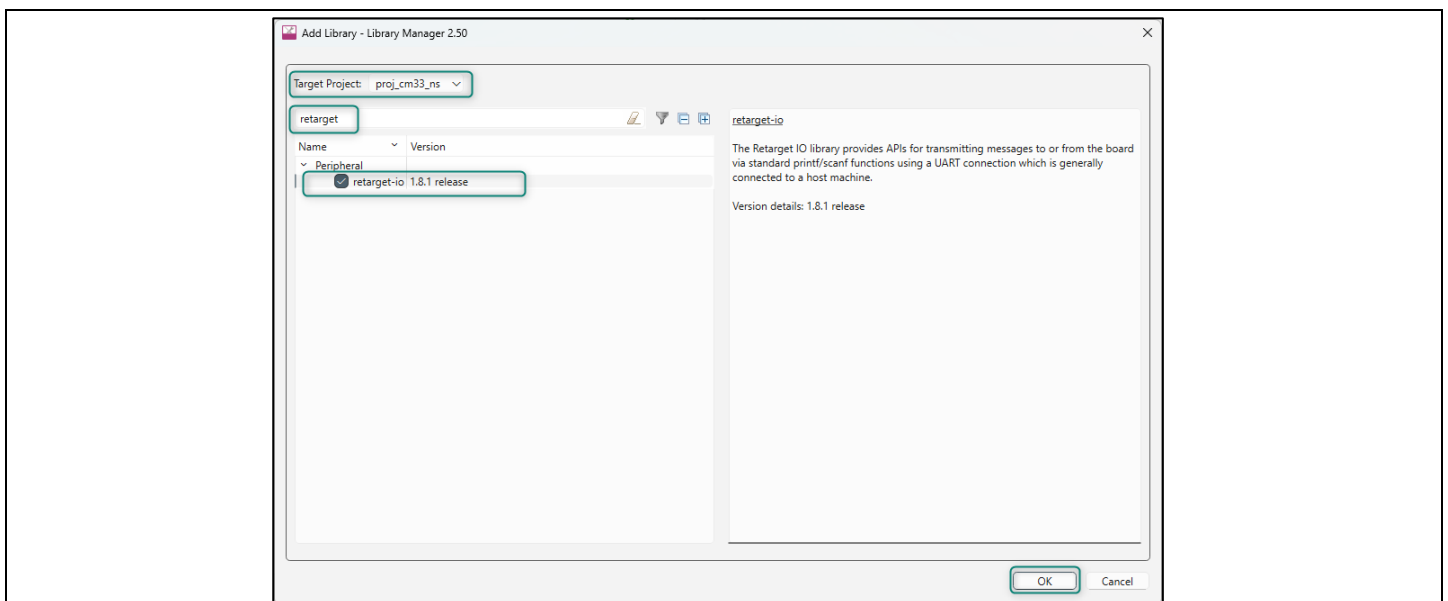


Figure 14 Adding retarget_io middleware

Now, click **Update** on the Library Manager.

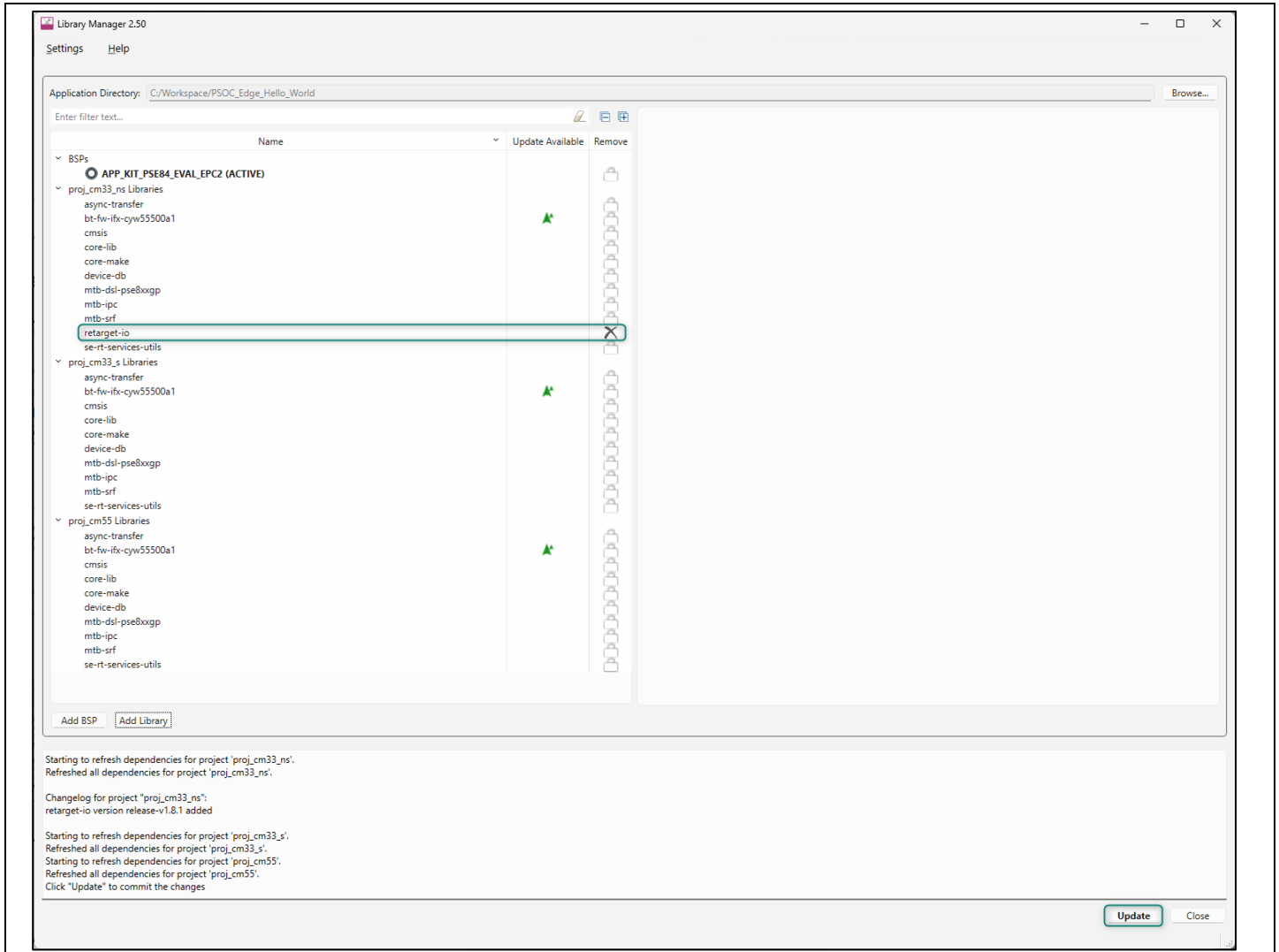


Figure 15 Library Manager – Removing freertos and adding retarget-io middleware

Library Manager will update all the changes made to the hello world project. **Close** the Library Manager after the updates have completed.

11. Now let's look how to configure or rename the pins using the Device Configurator.
Open the **Device Configurator** tool that can be found in the quick panel area under the **BSP Configurators** section.

The Device Configurator can be used to set up the system functions such as pins, interrupts, clocks, and DMA, as well as the basic peripherals, including UART, Timer, etc.

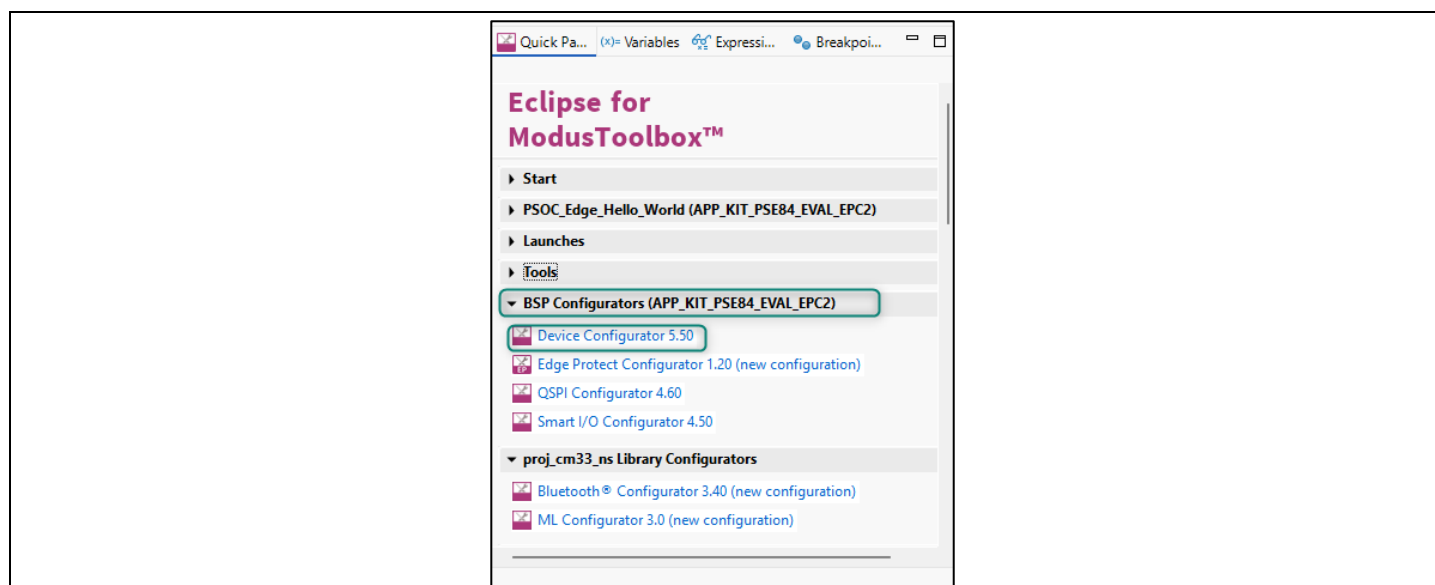


Figure 16 Selecting Device Configurator from Quick Panel – BSP Configurators

12. After opening the Device Configurator go to the Pins section.

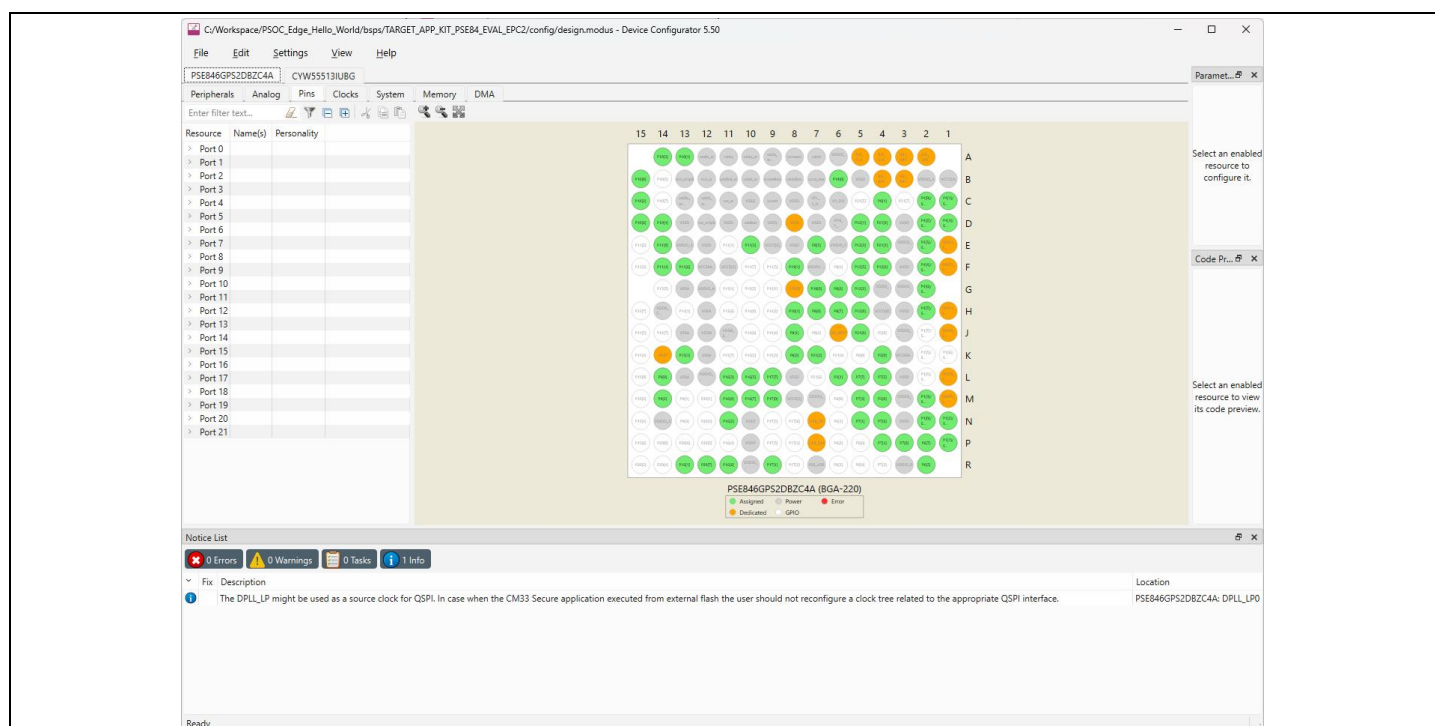


Figure 17 Device Configurator - Pins

13. In this particular Development Kit, the three User LEDs are connected to Port 16 Pins 5, 6 and 7 as per schematic

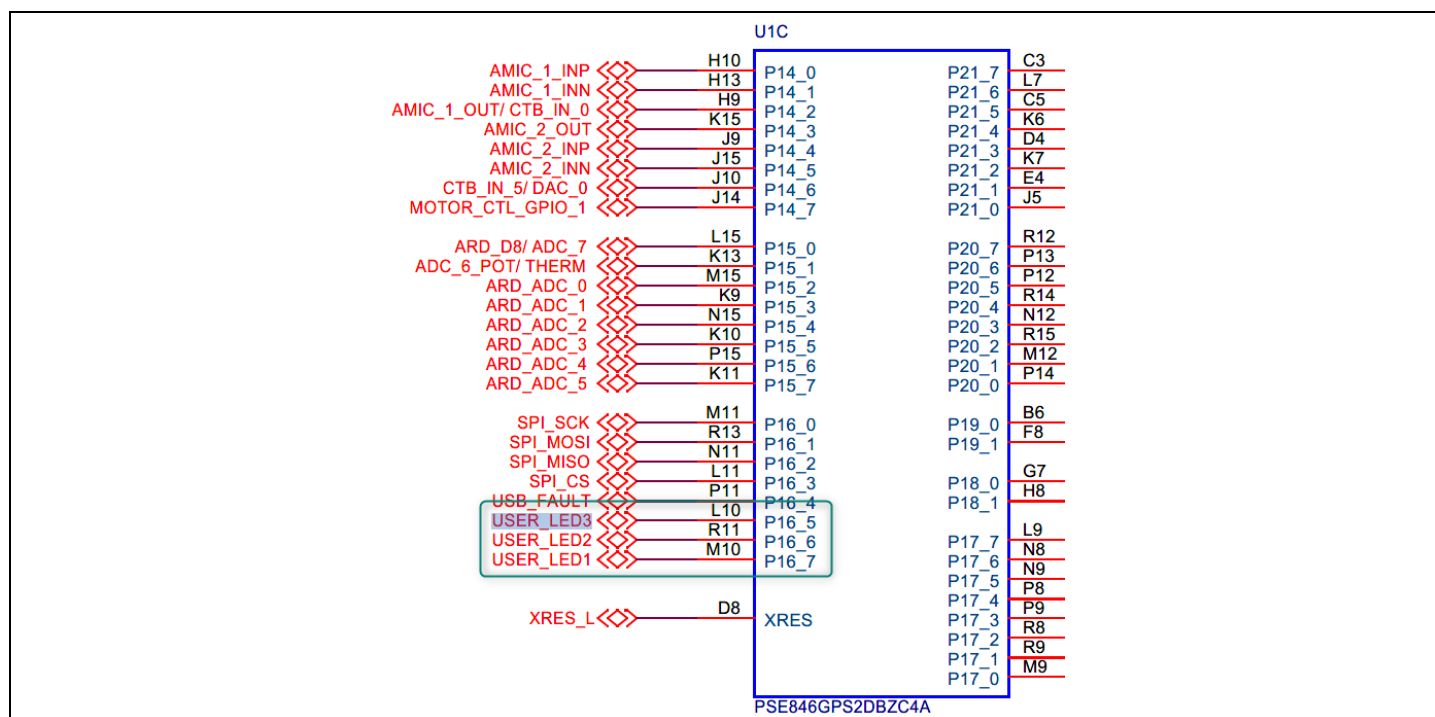


Figure 18 Schematic View of PSE84 USER_LEDx Port connections

Each pin can be uniquely identified by giving them a name of your choice. These names are known as aliases and can be used in the code to refer to the pins instead of using the explicit Pin numbers.

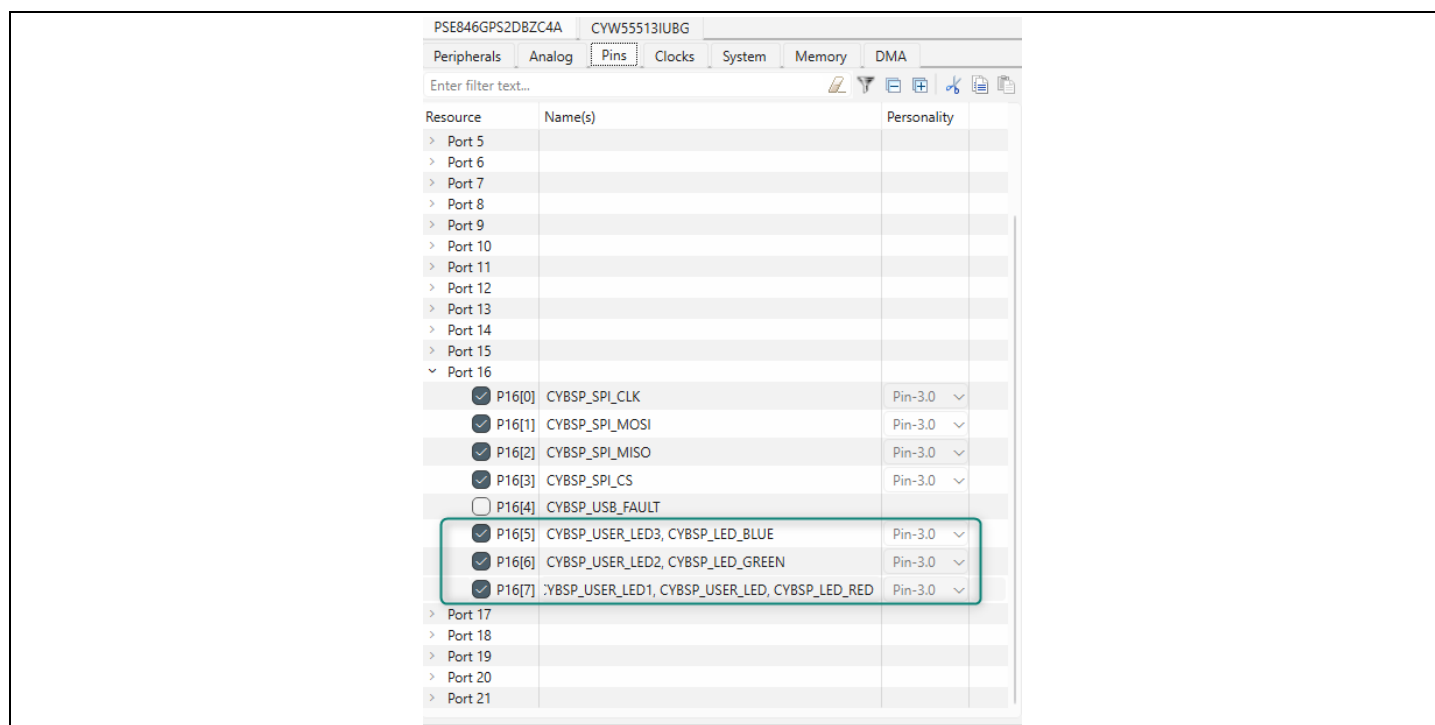


Figure 19 Device Configurator – Pins – Port16 Resource – CYBSP_USER_LEDx Aliases

14. For example, in Figure 20. Device Configurator - Pins – Creating Alias for USER_LED1 we have renamed Port 16 Pin 7 as TRAINING_LED. We will be using this LED in our application. Go to **File > Save** or alternatively hit **Ctrl+S** to save the change.

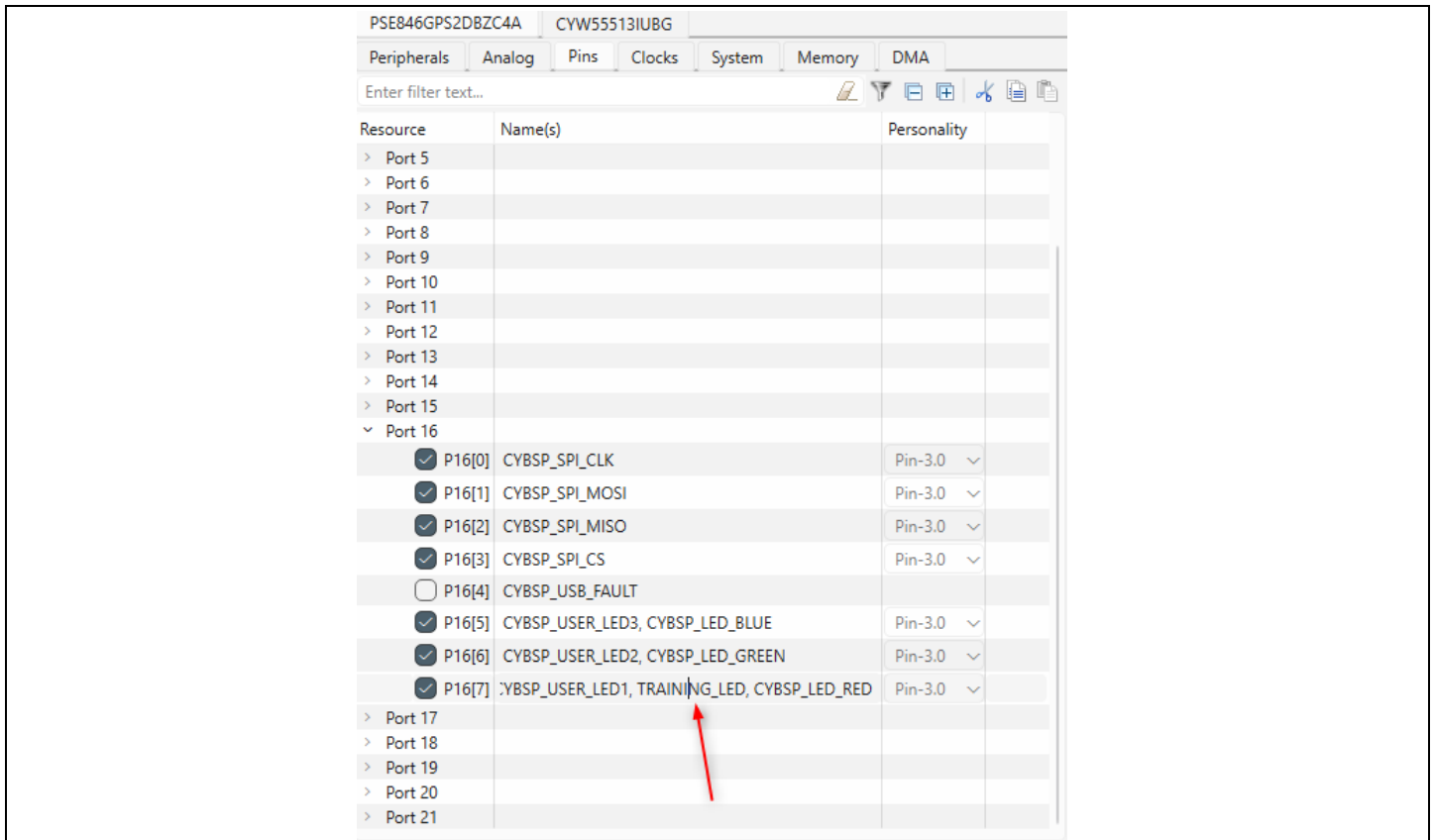


Figure 20. Device Configurator - Pins – Creating Alias for USER_LED1

Similarly let us look at the UART pins required for our PSOC_Edge_Hello_World code example. As per schematic Port 6 Pin 5 is used for UART_RX and Port 6 Pin 7 is used as UART_TX.

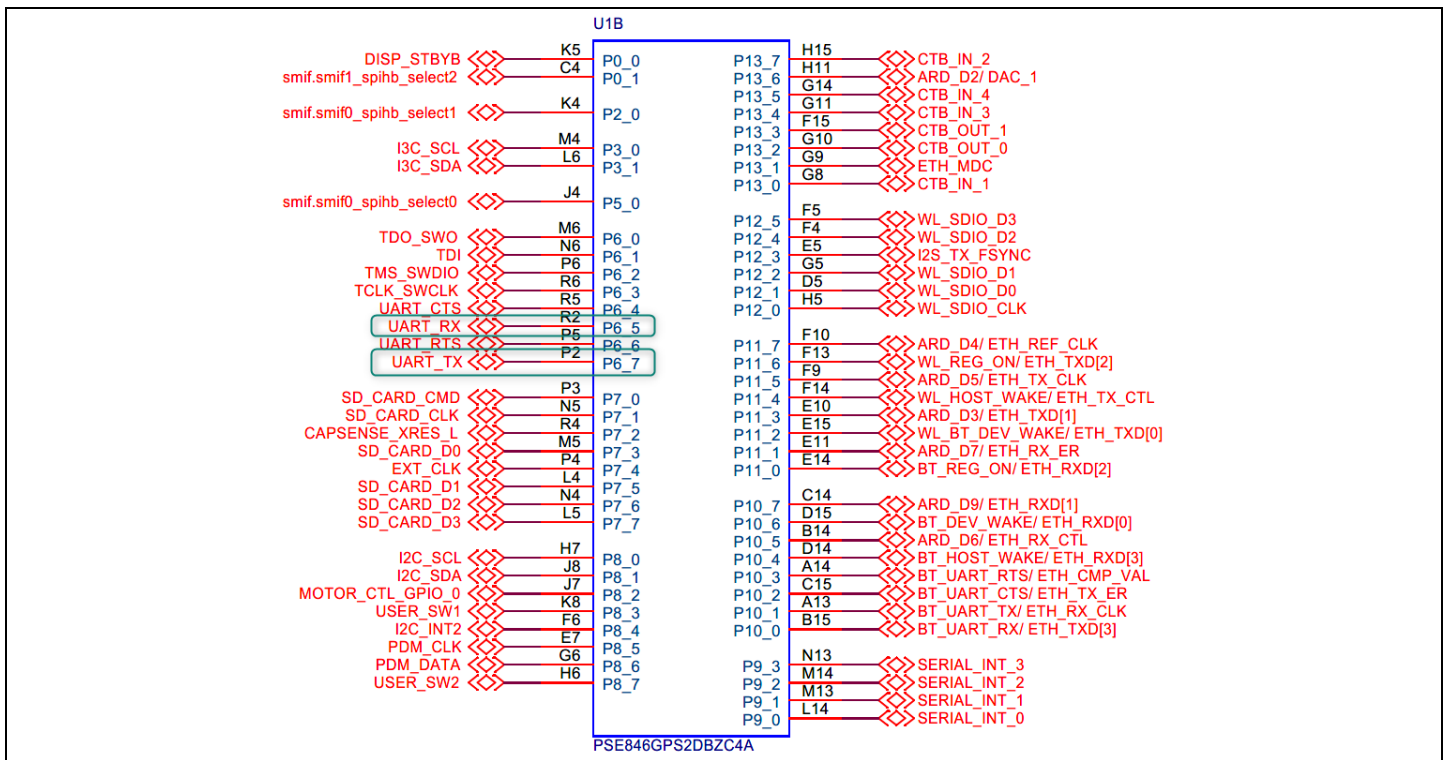


Figure 21 Pin assignments of UART_RX and UART_TX

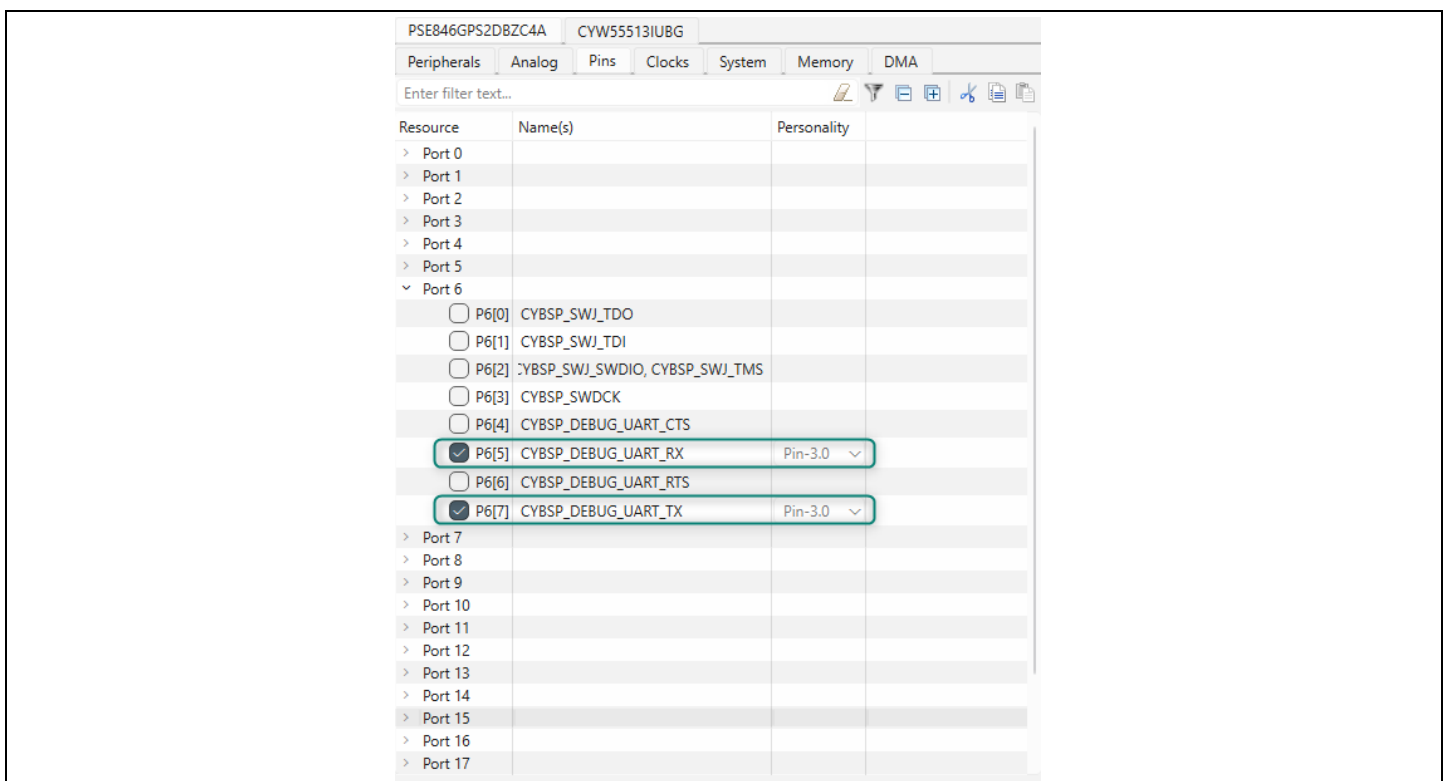


Figure 22 Device Configurator > Pins > BSP UART Pin assignments

Code Example 1: Hello world + Blinky

Remember these names “CYBSP_DEBUG_UART_TX” and “CYBSP_DEBUG_UART_RX”, as we will be using these during application development.

15. Select File > Save to save our changes, and then Close the window to quit the configurator.
16. The changes made in the Device Configurator can be see if you open the `cycfg_pins.h` file. The `cycfg_pins.h` file, along with all source files that get generated by the Device Configurator can be found **GenerateSource** folder under with the application’s bsp’s folder tree. Figure shows the update. can see the pin definitions and aliases. And the change we have made has been reflected in the file as shown below.

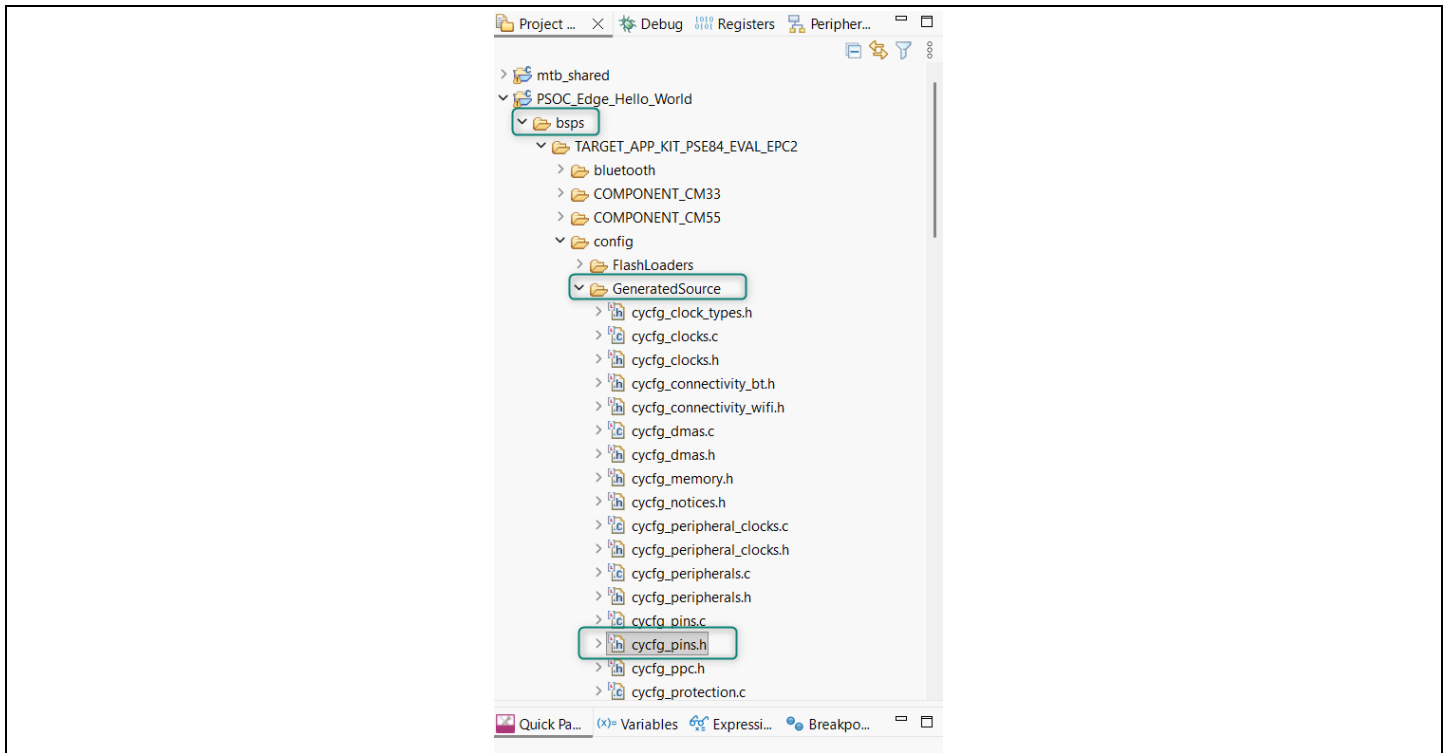


Figure 23 Device Configurator - GeneratedSource folder location with `cycfg_pins.h`

Code Example 1: Hello world + Blinky

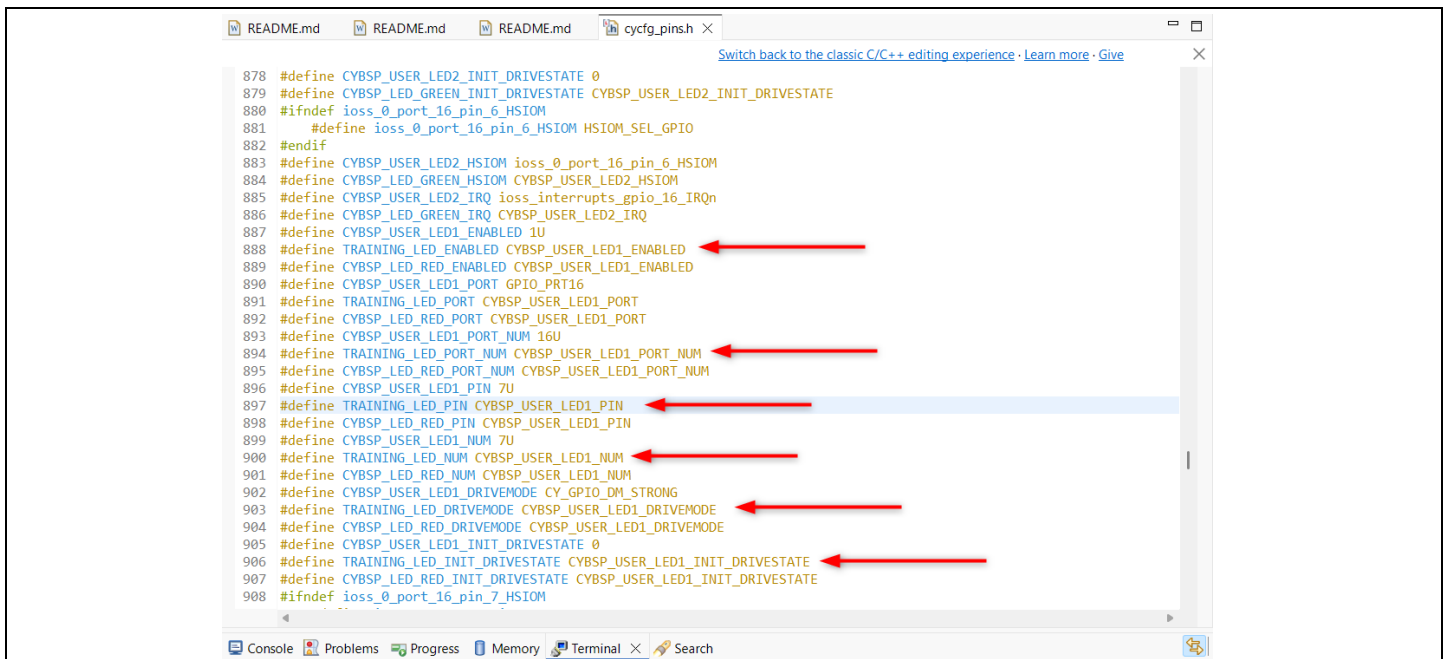


Figure 24 Training_LED Alias in cycfg_pins.h

17. In the project explorer you can see two folders named PSOC_Edge_Hello_World.proj_cm33_ns and PSOC_Edge_Hello_World.proj_cm55. Each of these folders will contain a main.c file which contains the application code. Replace these main.c files with main.c template files we have provided. (The template main.c files have required macros, definitions and basic initialization.

- Copy the contents from Lab_Source\templates\Hello_World+Blinky\main_cm33_ns.c and paste into PSOC_Edge_Hello_World.proj_cm33_ns main.c
- Copy the contents from Lab_Source\templates\Hello_World+Blinky\main_cm55.c and paste on to PSOC_Edge_Hello_World.proj_cm55 main.c

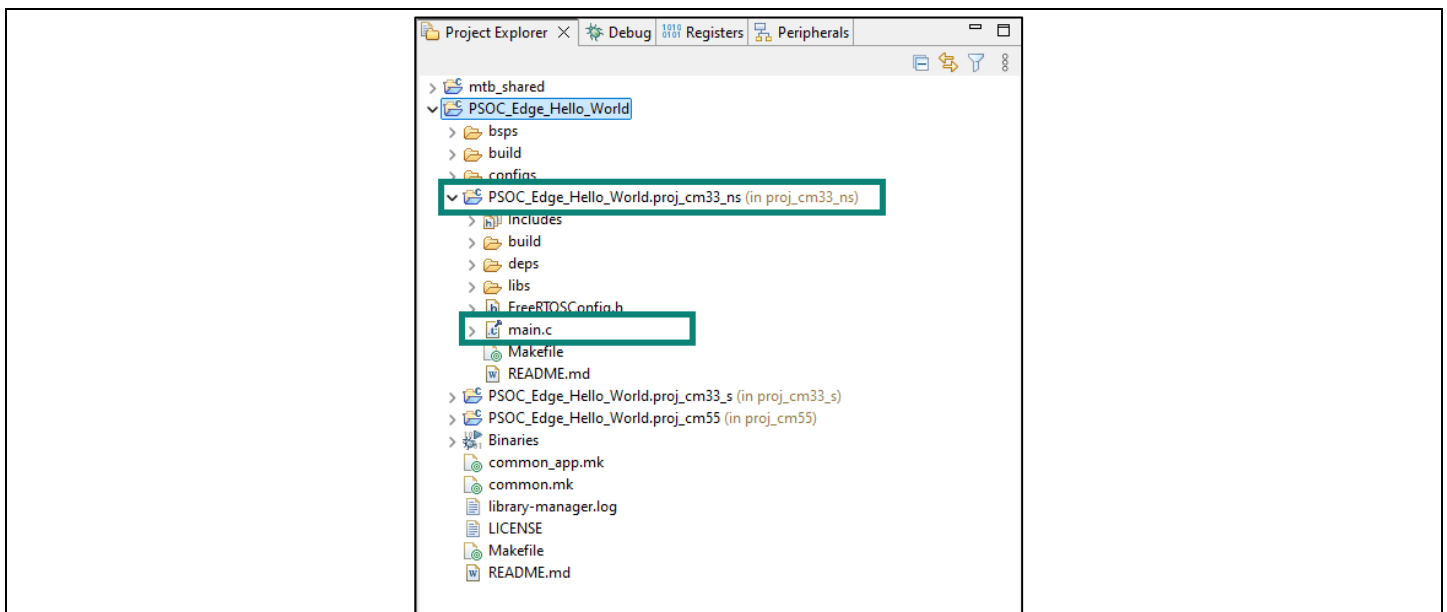


Figure 25 cm33_ns main.c file location

Introduction to PSOC™ Edge E84

Getting Started with the Development Ecosystem

Code Example 1: Hello world + Blinky

18. Remove “FREERTOS RTOS_AWARE” in the COMPONENTS from the proj_cm33_ns and proj_cm55 Makefile

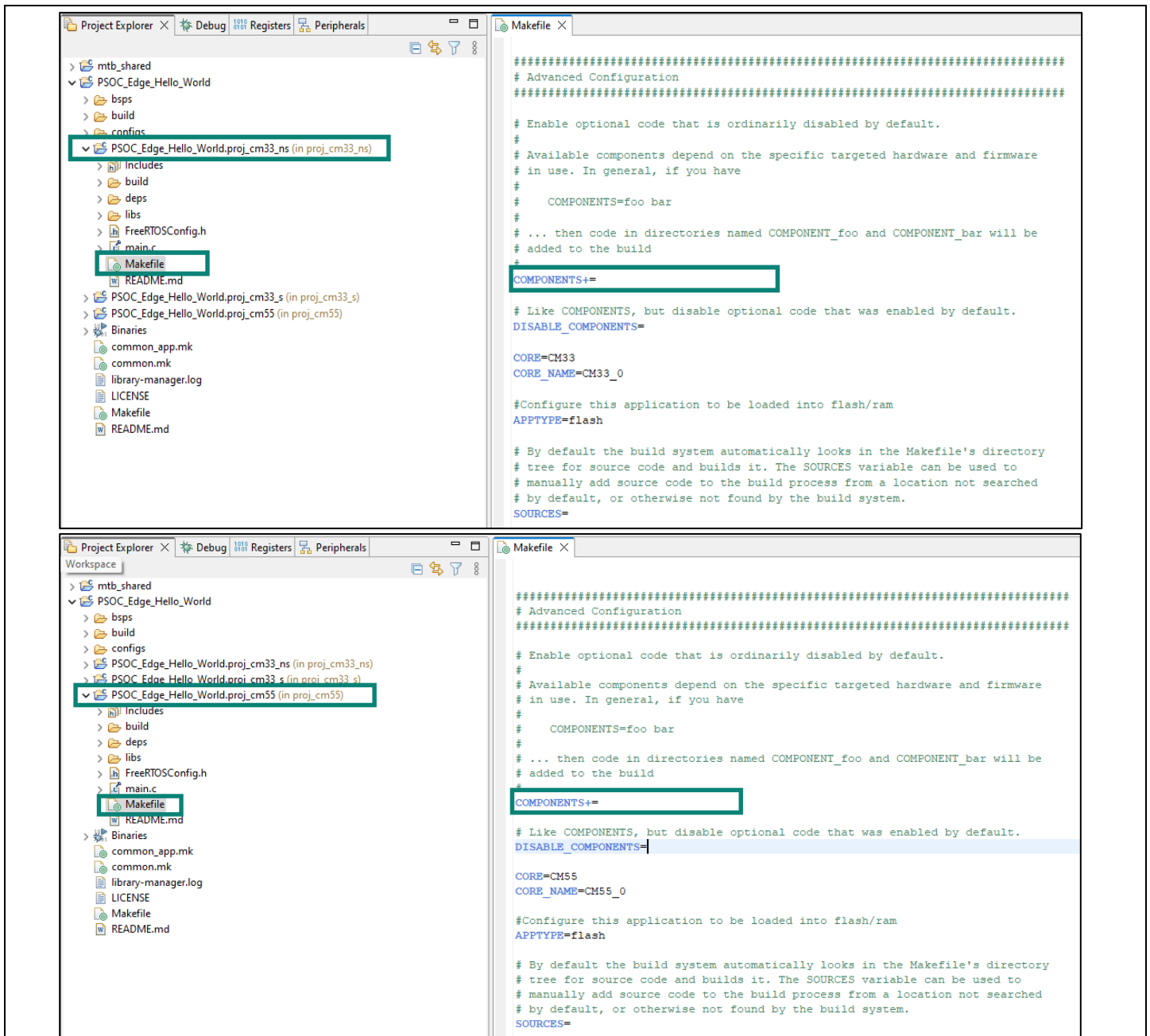


Figure 26 Makefile for cm33_ns and cm55 – Remove component FREERTOS AWARE

19. Build the projects and make sure nothing is broken at this point.
20. Now let's add the application code.
21. Open the main.c file under PSOC_Edge_Hello_World.proj_cm33_ns. We will be adding the necessary code to this file for CM33 Core

Code Example 1: Hello world + Blinky

Let us initialize the GPIO for Training LED. The Below code initializes the user LED pin. Copy and paste the below given code inside <code snippet> to the main.c file after `__enable_irq()`;

Code listing 1 Initialize the GPIO using the PDL API to drive the TRAINING_LED

```
/* Set GPIO for use with LED */
Cy_GPIO_SetDrivemode(TRAINING_LED_PORT, TRAINING_LED_PIN, CY_GPIO_DM_STRONG);
```

The reference for PDL APIs will be available in PDL documentation can be found [here](#), where you can search for particular function and get an overview of it. Here is an example for the `Cy_GPIO_SetDrivemode` we are using.



◆ Cy_GPIO_SetDrivemode()

```
__STATIC_INLINE void Cy_GPIO_SetDrivemode ( GPIO_PRT_Type * base,
                                             uint32_t pinNum,
                                             uint32_t value
                                             )
```

Configures the pin output buffer drive mode and input buffer enable.

The output buffer drive mode and input buffer enable are combined into a single parameter. The drive mode controls the behavior of the pin in general. Enabling the input buffer allows the digital pin state to be read but also contributes to extra current consumption.

Parameters

- base** Pointer to the pin's port register base address
- pinNum** Position of the pin bit-field within the port register
- value** Pin drive mode. Options are detailed in [Pin drive mode macros](#)

Note

This function modifies a port register in a read-modify-write operation. It is not thread safe as the resource is shared among multiple pins on a port.

Function Usage

```
/* Scenario: Enter deep-sleep with reduced leakage current on P0.0 */
/* Get the drive mode of P0.0 */
if (CY_GPIO_DM_STRONG == Cy_GPIO_GetDrivemode(P0_0_PORT, P0_0_NUM))
{
    /* Change the drive mode of P0.0 to analog (hi-z, input buffer off) */
    Cy_GPIO_SetDrivemode(P0_0_PORT, P0_0_NUM, CY_GPIO_DM_ANALOG);
}
```

Figure 27 PDL API Online Reference - `Cy_GPIO_SetDrivemode()`

22. Now let's initialize the UART. First, paste in the **global variables** for the UART.

Code listing 2 Setting up Global Variables for the UART

```
static mtb_hal_uart_t DEBUG_UART_hal_obj;
static cy_stc_scb_uart_context_t DEBUG_UART_context;
```

Second, place the UART <code snippet> below for use with `retarget-io` in the appropriate place in the main function.

Code listing 3 Initializing the UART for use with `retarget_io` middleware

```
/* Initialize UART for use with retarget_io */

Cy_SCB_UART_Init(CYBSP_DEBUG_UART_HW, &CYBSP_DEBUG_UART_config,
                 &DEBUG_UART_context);
Cy_SCB_UART_Enable(CYBSP_DEBUG_UART_HW);
```

```

result = mtb_hal_uart_setup(&DEBUG_UART_hal_obj,
                           &CYBSP_DEBUG_UART_hal_config,
                           &DEBUG_UART_context, NULL);

/* UART setup failed. Stop program execution. */
if (CY_RSLT_SUCCESS != result)
{
    handle_app_error();
}

/*init retarget_io with uart*/
cy_retarget_io_init(&DEBUG_UART_hal_obj);

```

23. Now to allow for writing to the UART, we need to make sure global interrupts are enabled.

Code listing 4 Enable global irq's to use UART

```
__enable_irq();
```

24. Since we have added code for the UART initialization, let's add some print/logs to get printed to the terminal. Copy and paste the below given code inside <code snippet> to the main.c file

Code listing 5 Write Hello World header to UART

```

/* Print hello world */
printf("\x1b[2J\x1b[H");

printf("***** "
"PSOC Edge MCU: Hello world using retarget-IO"
"***** \r\n\n");

printf("Hello World!!!\r\n\n");

printf("For more projects, "
"visit our code examples repositories:\r\n\n");

printf("https://github.com/Infineon/"
"Code-Examples-for-ModusToolbox™-Software\r\n\n");

```

25. We have added several code snippets, so take a moment to build the cm33_ns project to make sure there are no build errors. There will be a few warnings for some variables we haven't used yet, but there should be no build errors.

26. Next, let's focus on setting up to toggle the LED every 1 second. We need to setup a timer resource, so select the PSOC_Edge_Hello_World application in the project explorer and then select the Device Configurator in the Quick Panel > BSP Configurators section. Select the Peripherals tab, and open the Digital > Timer, Counter and PWM Group and select and used TWPCM 32-bit Counter

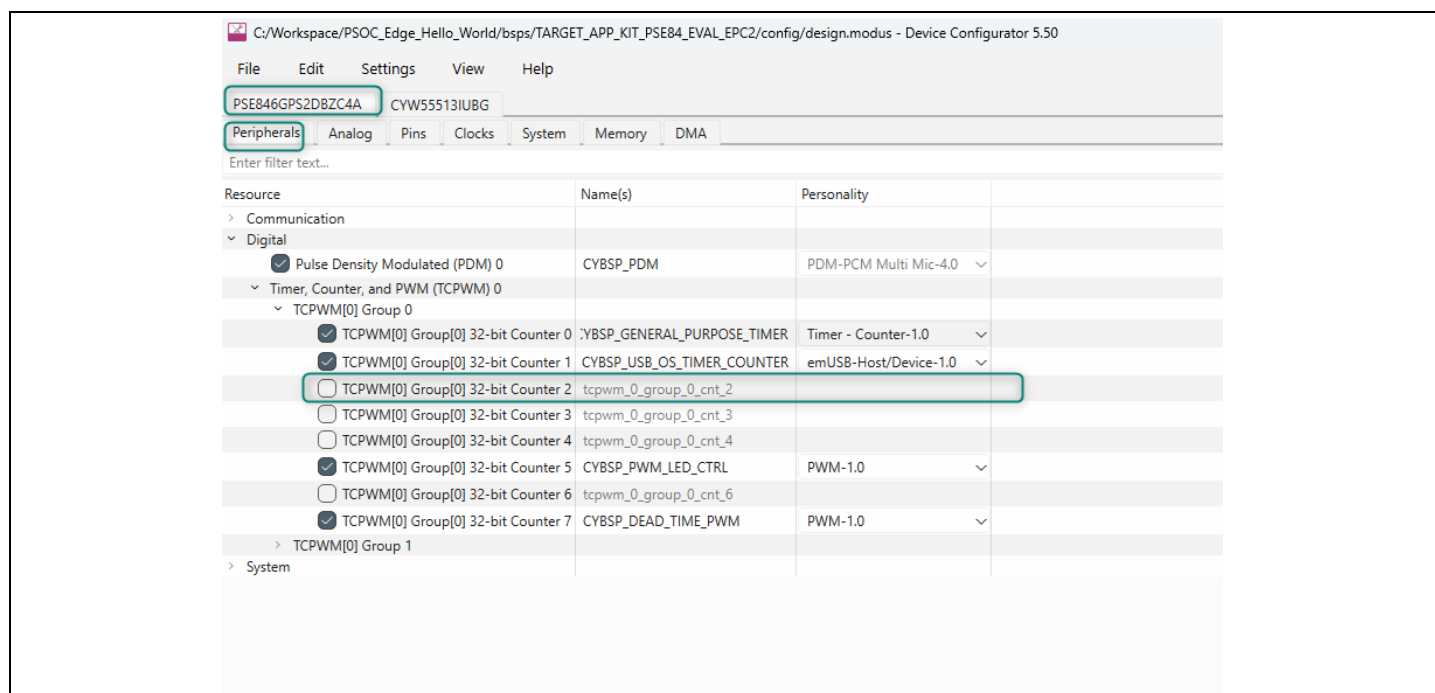


Figure 28 Device Configurator – Peripherals – Digital – Timer, Counter and PWM Group – Timer Selection

27. Chosose the Timer – Counter 1.0 personality and click OK

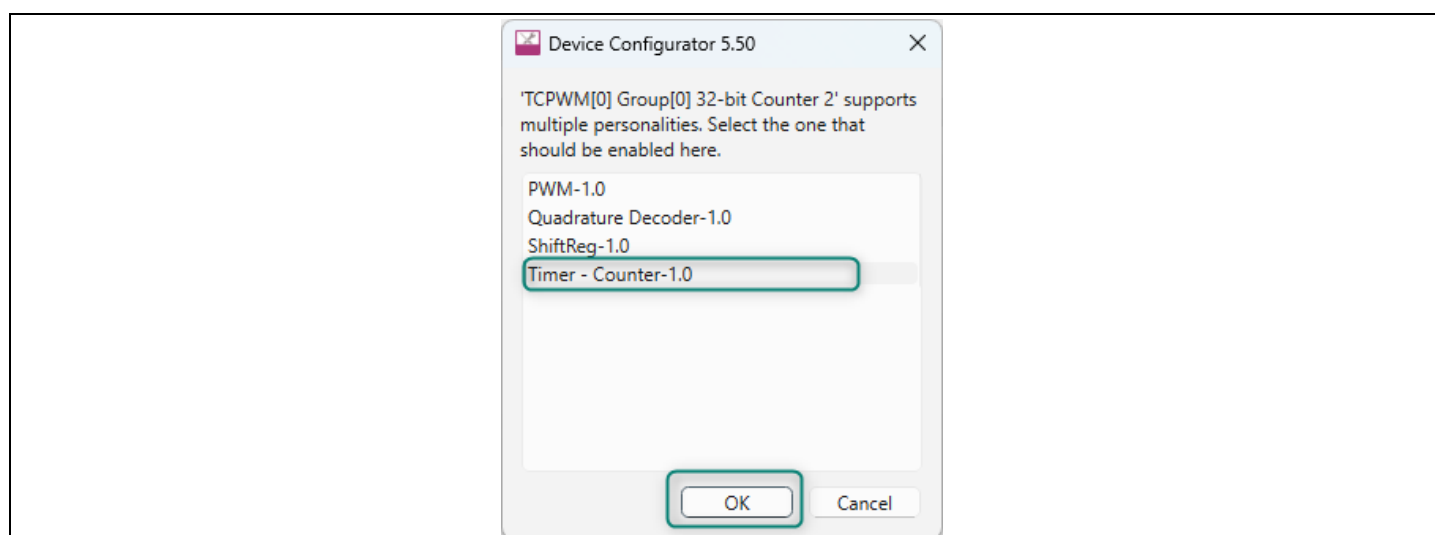


Figure 29 Timer – Counter 1.0 Personality

28. Rename the timer to TIMER

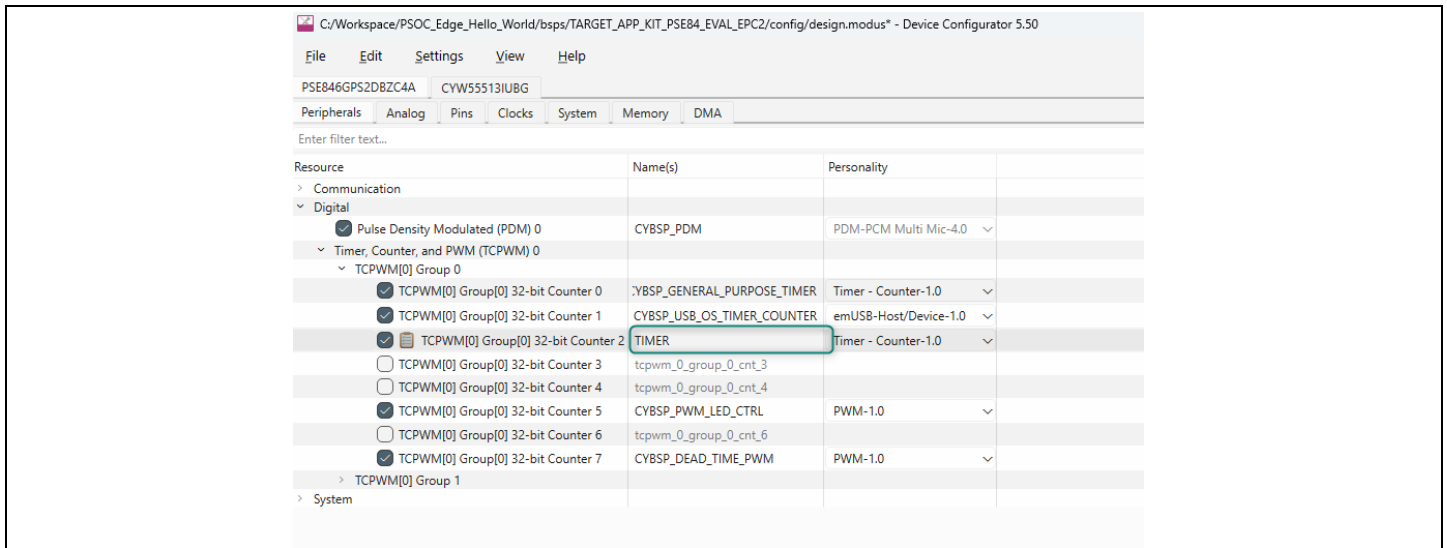


Figure 30 Naming the Timer

29. Set up the Parameters for the TIMER

- i. Run Mode to **Continuous**
- ii. Period to **32000**
- iii. Compare or Capture to **Compare**
- iv. Count Direction **Up**
- v. Compare value to **32000**
- vi. Check the box for Interrupt Source > Compare 0 & Capture 0

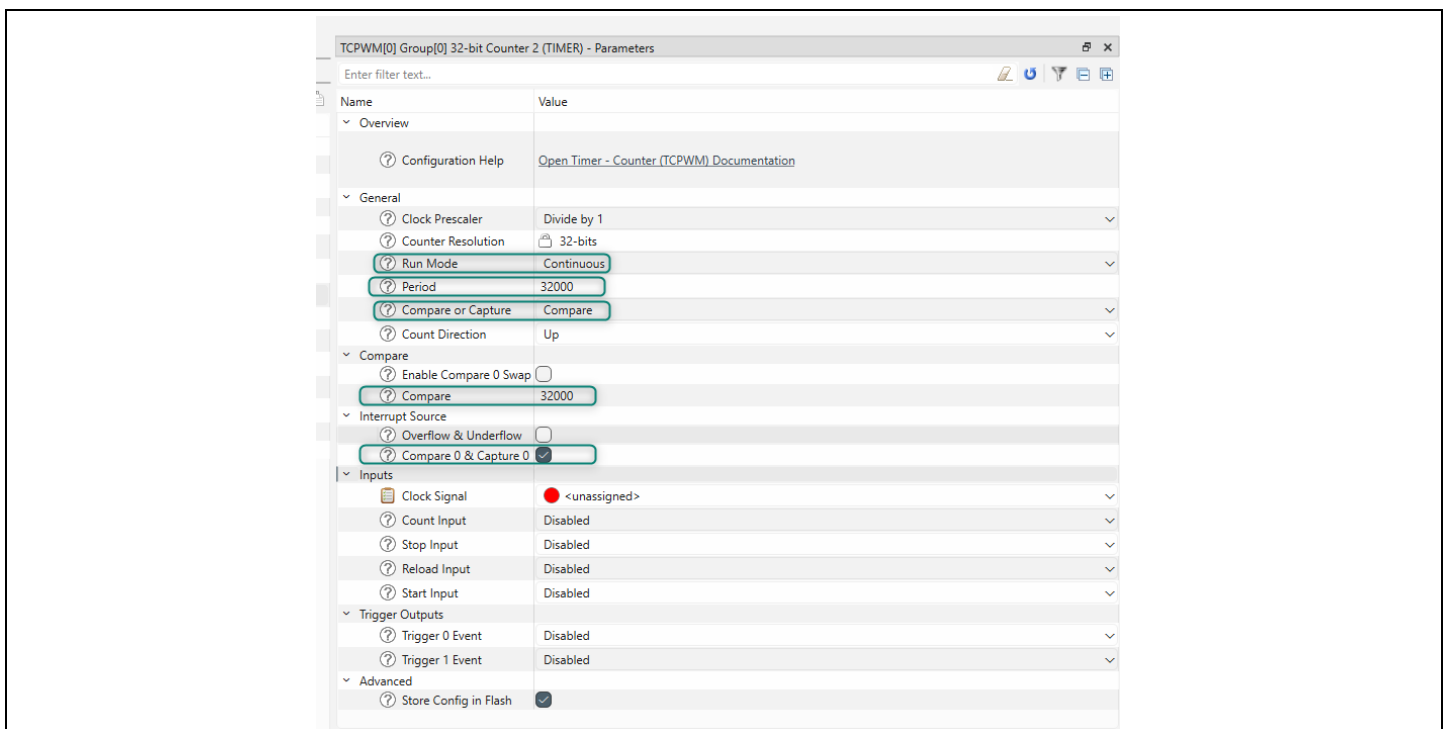


Figure 31 Timer Parameters

Introduction to PSOC™ Edge E84

Getting Started with the Development Ecosystem

Code Example 1: Hello world + Blinky

30. Select the Input > Clock Signal. We can use the use the CYBSP_GENERAL_PURPOSE_TIMER_CLK_DIV.

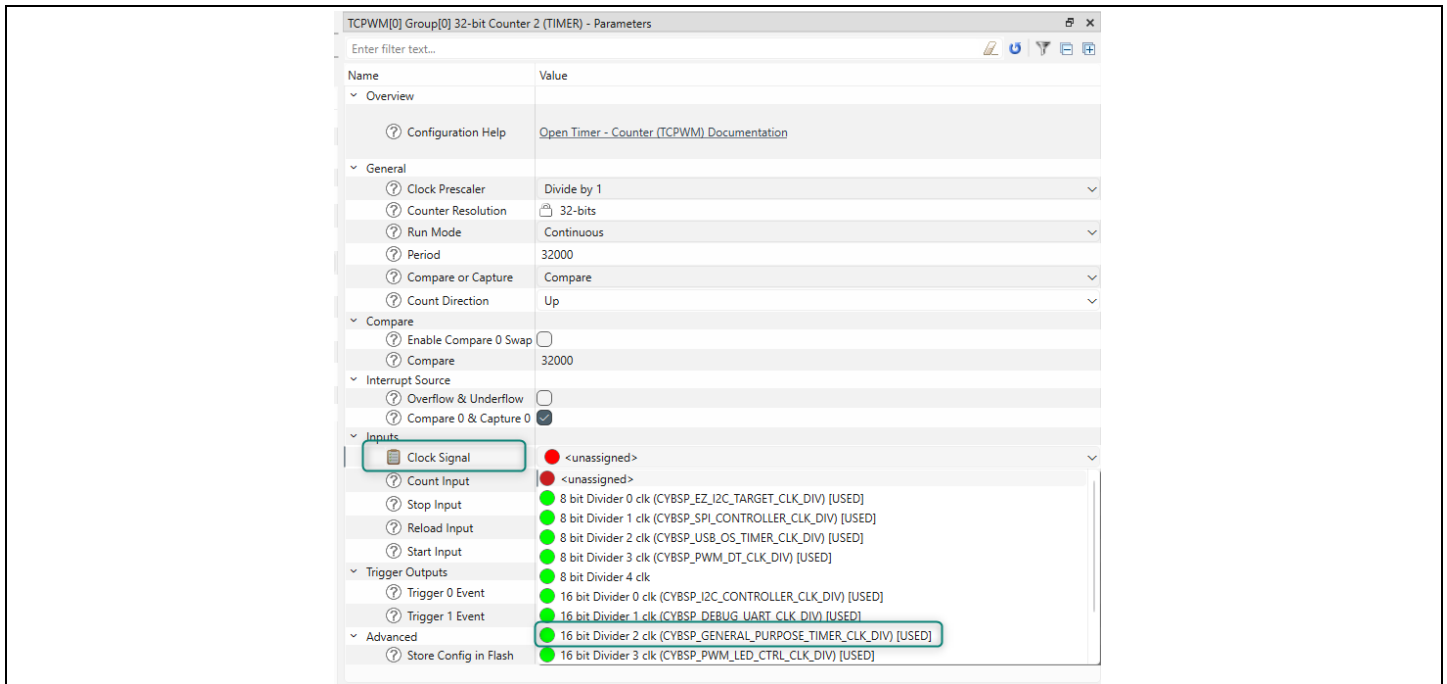


Figure 32 Timer Input Clock Signal selection

31. With the TIMER configured, choose **File > Save** or **Ctrl+S** and close the Device Configurator.

32. Let's place a call to `timer_init()` function which we will build later.

Code listing 6 Call to initialize the timer

```
/* Initialize timer to toggle the LED */
timer_init();
```

33. We will enable the second core as well using below code snippet. Copy and paste the below given code inside <code snippet> to the main.c file.

Code listing 7 Enable the CM55 CPU

```
/* Enable CM55 */
/* CM55_APP_BOOT_ADDR must be updated if CM55 memory layout is changed.*/
Cy_SysEnableCM55(MXCM55, CM55_APP_BOOT_ADDR, CM55_BOOT_WAIT_TIME_USEC);
```

34. Now Let's add the **Application code logic**

a. First, we will be **infinite** for loop Polling for the **key press** entry in the terminal

Code Example 1: Hello world + Blinky

- b. Once User Presses the key, we will verify if the key is “carriage return(\r)”.
- c. If It is carriage return, will check for “led_blink_active_flag”, if it set we will stop the timer and pause the LED Toggling
- d. Else the LED Toggling Resumes
- e. If the received key is not carriage return, we will check for “timer_interrupt_flag”, if it set we will Blink(toggle) the LED.
- f. these a to f steps are executed in an infinite loop

As explained above, the code blinks the LED using the interrupts received from the Timer. It also checks whether the Enter key is pressed by the user via the UART and pauses or resumes LED blinking based on it.

Code listing 8 Main forever for loop for the cm33

```

/* main forever loop*/

for(;;)
{
    /* Check to see if we have received anything on the uart*/
    if (Cy_SCB_UART_GetNumInRxFifo(CYBSP_DEBUG_UART_HW) > 0)
    {
        scanf("%c", &uart_read_value);

        if (uart_read_value == '\r')
        {
            led_blink_active_flag = !led_blink_active_flag;
            if (led_blink_active_flag)
            {
                Cy_TCPWM_TriggerStart_Single(TIMER_HW, TIMER_NUM);
                printf("Enter key detected: Blinking Start\r\n");
            }
            else
            {
                Cy_TCPWM_TriggerStopOrKill_Single(TIMER_HW, TIMER_NUM);
                printf("Enter key detected: Blinking Stops\r\n");
            }
        }
    }

    /* Check if timer elapsed (interrupt fired) and toggle the LED */
    if (timer_interrupt_flag)
    {

```

Code Example 1: Hello world + Blinky

```

        /* Clear the timer irq flag */
        timer_interrupt_flag = false;

        /* Toggle LED state */
        Cy_GPIO_Inv(TRAINING_LED_PORT, TRAINING_LED_PIN);
    }
}

```

35. The `timer_init()` function prototype was added to `main.c`, but we haven't added the function yet. First, let's set up the needed global variable.

Code listing 9 Global Variables for the timer irq configuration

```

cy_stc_sysint_t timer_irq_cfg =
{
    .intrSrc = TIMER_IRQ,
    .intrPriority = TIMER_INT_PRIORITY
};

```

Second add the `timer_init()` function code below the `timer_init` summary.

Code listing 10 Function definition for the `timer_init()`

```

/*****
* Function Name: timer_init
*****/
Summary:
* This function creates and configures a Timer object. The timer ticks
* continuously and produces a periodic interrupt on every terminal count
* event. The period is defined by the 'period' and 'compare_value' of the
* timer configuration structure 'led_blink_timer_cfg'. Without any changes,
* this application is designed to produce an interrupt every 1 second.
*
* Parameters:
*   none
*
* Return :
*   void
*
*****/
void timer_init(void)
{
    /* Initialize TCPWM block with required timer configuration */
    if (CY_TCPWM_SUCCESS != Cy_TCPWM_Counter_Init(TIMER_HW,
                                                    TIMER_NUM,
                                                    &TIMER_config))
    {
        CY_ASSERT(0);
    }
}

```

Code Example 1: Hello world + Blinky

```

/* Set the interrupt line for COUNTER_HW */
if (CY_SYSINT_SUCCESS != Cy_SysInt_Init(&timer_irq_cfg, &timer_isr))
{
    CY_ASSERT(0);
}

/* setup the timer interrupt */
NVIC_EnableIRQ(TIMER_IRQ);

/* Enable the initialized counter */
Cy_TCPWM_Counter_Enable(TIMER_HW,
                        TIMER_NUM);

/* Start the counter */
Cy_TCPWM_TriggerStart_Single(TIMER_HW,
                            TIMER_NUM);
}

```

36. Now add the interrupt handler function for the timer. Copy and paste the below given code inside <code snippet> to the main.c file below timer_init summary.

Code listing 11 Function definition for the timer irq handler

```

/*****
* Function Name: timer_isr
*****/
* Summary:
* This is the interrupt handler function for the timer interrupt.
*
* Parameters:
*   callback_arg   Arguments passed to the interrupt callback
*   event Timer/counter interrupt triggers
*
* Return:
*   void
*****/
static void timer_isr(void)
{
    uint32_t interrupts = Cy_TCPWM_GetInterruptStatusMasked(TIMER_HW,
                                                            TIMER_NUM);

    Cy_TCPWM_ClearInterrupt(TIMER_HW, TIMER_NUM, interrupts);

    /* Check for Timer Count event */
    if (BIT_MASK_CHECK != (CY_TCPWM_INT_ON_CC0 & interrupts))
    {
        timer_interrupt_flag = true;
    }
}

```

37. Save the file. We are done with adding the application code for PSOC_Edge_Hello_World.proj_cm33_ns

Code Example 1: Hello world + Blinky

38. Let's add the application code for PSOC_Edge_Hello_World.proj_cm55. Open **main.c** under PSOC_Edge_Hello_World.proj_cm55 and put the CM55 core to system deep sleep, as we are not performing any task in this core. Copy and paste the below given code inside **<code snippet>** to the PSOC_Edge_Hello_World.proj_cm55 main.c after **__enable_irq()**;

Code listing 12 Main forever for loop for the cm55

```
/* Put the CPU to deepsleep */
for (;;)
{
    Cy_SysPm_CpuEnterDeepSleep(CY_SYSPM_WAIT_FOR_INTERRUPT);
}
```

39. **Save** the file. We are done with adding the application code to **both** cores
40. Let's build the Application by making sure we select the PSOC_Edge_Hello_World folder in the project explorer and then by clicking on the **Build Application** option in the Quick Panel.

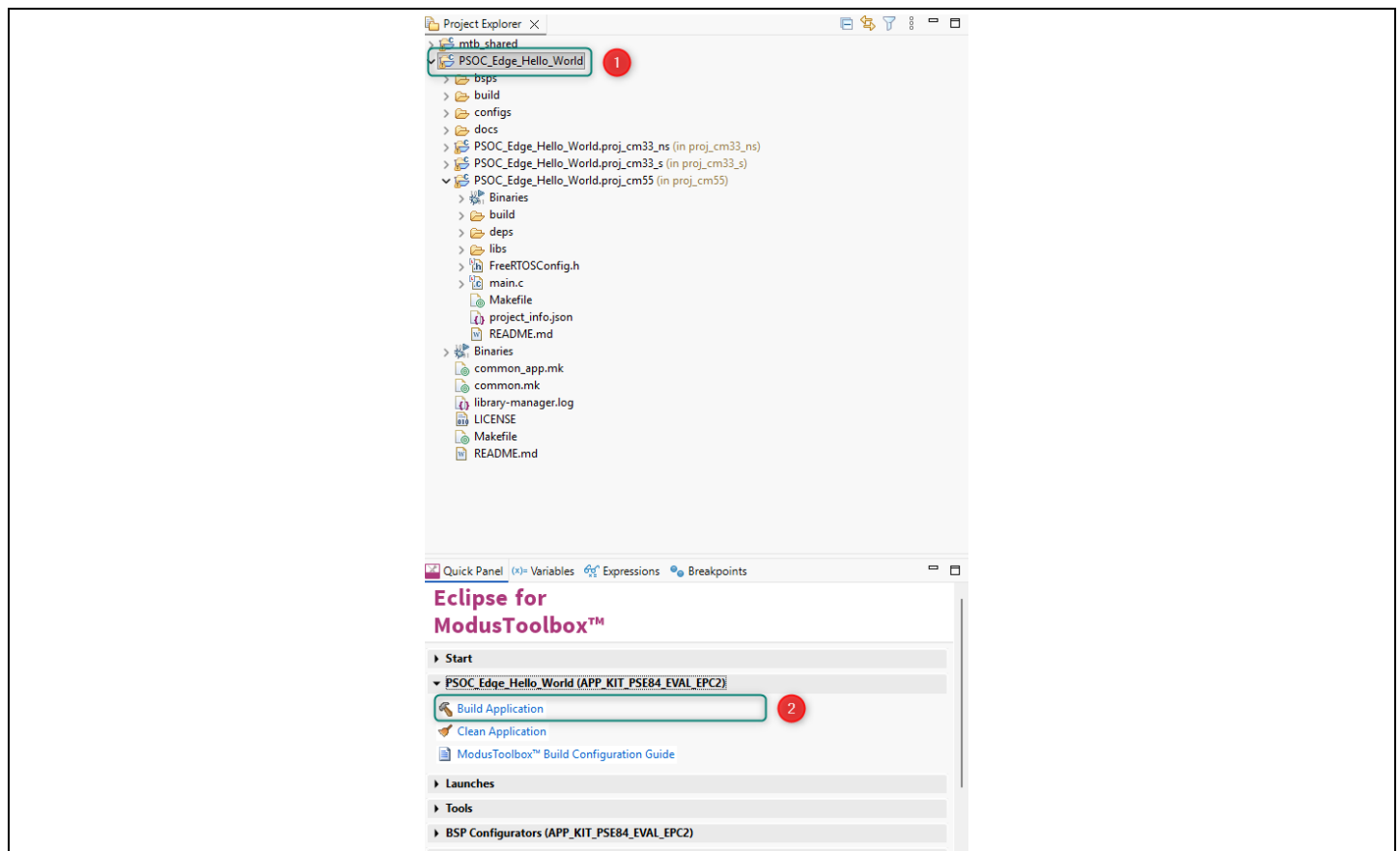


Figure 33 PSOC_Edge_Hello_World Build Application

41. After the Build process is successfully completed. We can **program** the board by clicking the Launch Program Application tool in the Quick Panel > Launches > PSOC_Hello_World.

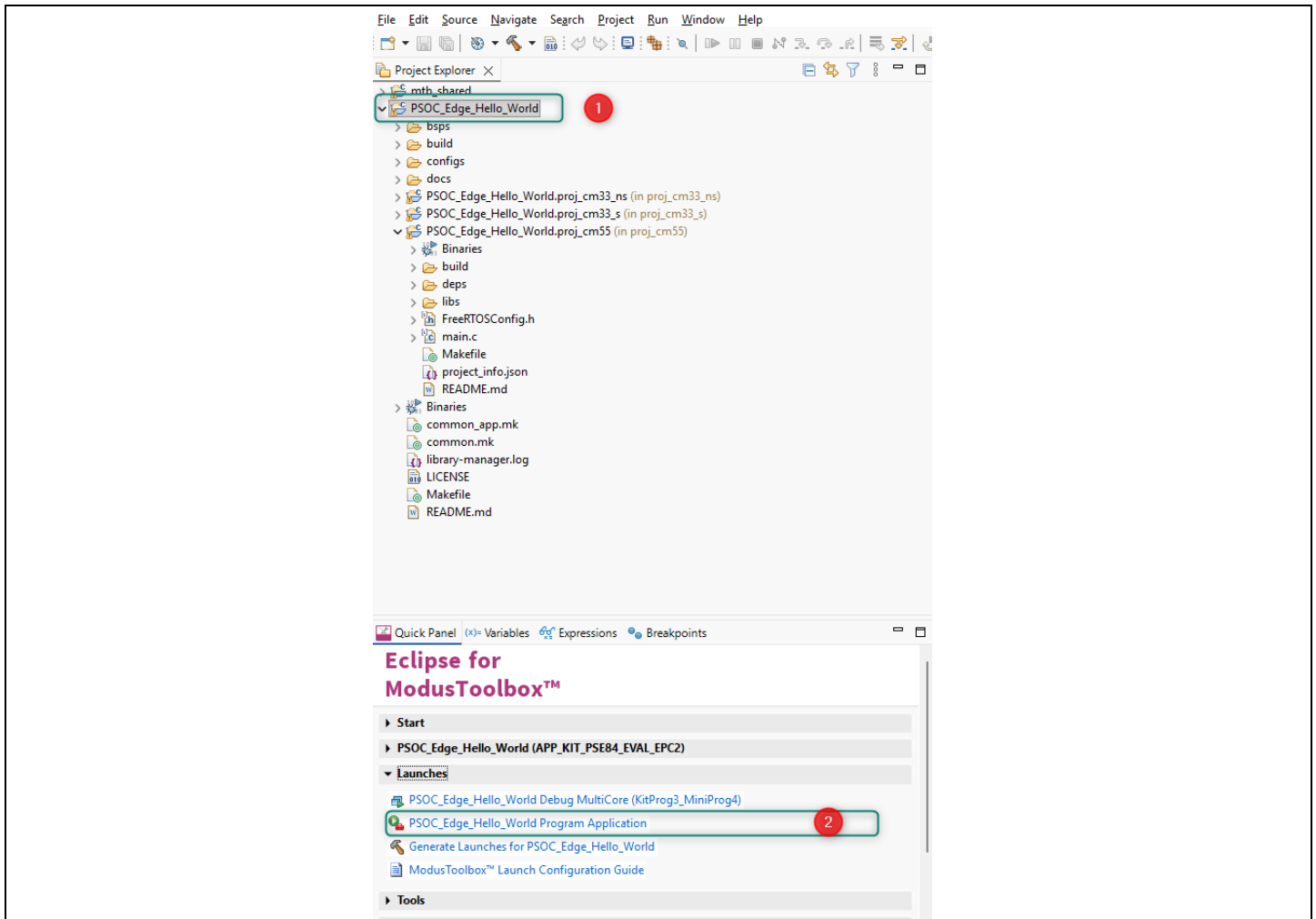


Figure 34 PSOC_Edge_Hello_World Launch Program Application

42. After programming is successful, Open a Serial Terminal inside the Eclipse IDE and configure with a baud rate 115200 and 8-N-1 then restart the EVK. You will see the message as shown in below image and the kit RED LED blinks at approximately 1 Hz. Press Enter Key to pause and resume the LED Blinking respectively.



Figure 35 Eclipse IDE Terminal Selection

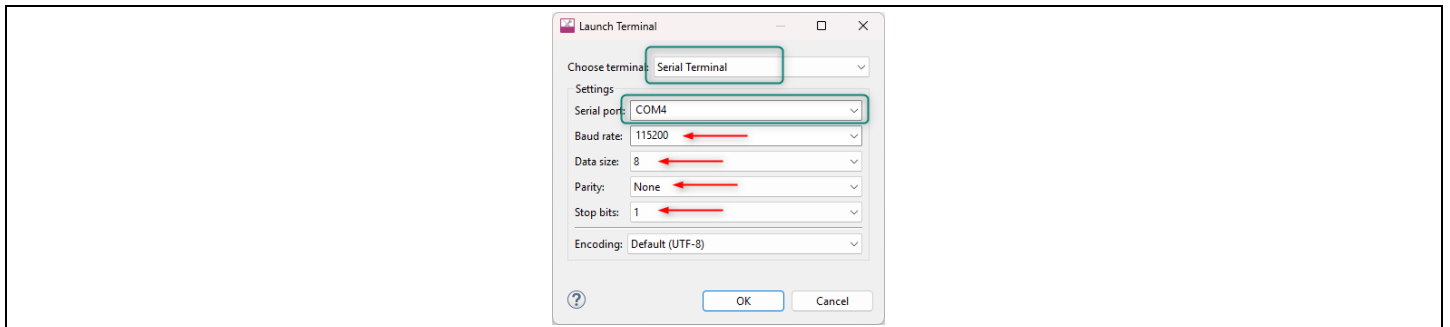


Figure 36 Eclipse IDE Terminal Serial Configuration

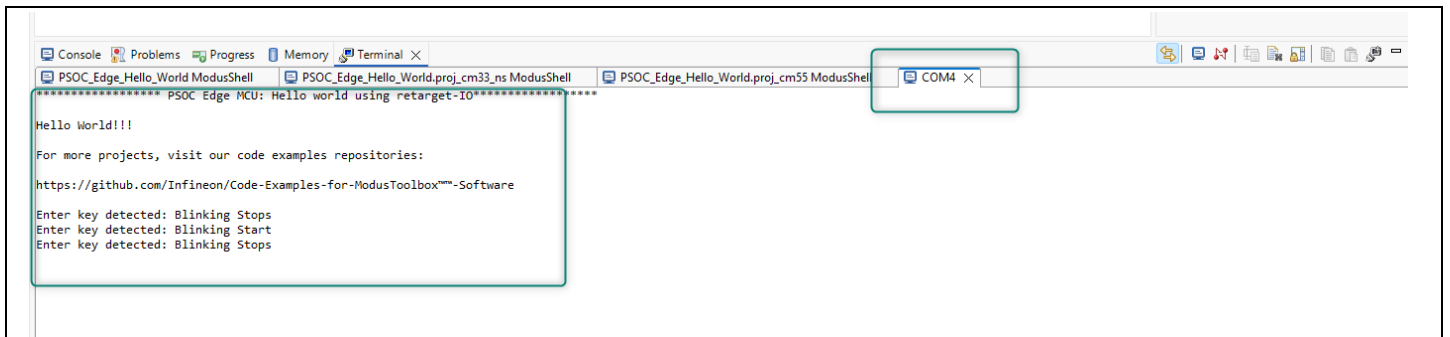


Figure 37 Eclipse IDE Terminal Serial Communication from the EVK

4.6 Conclusion

We successfully created a PSOC™ Edge E84 application using ModusToolbox™, utilizing the Device Configurator for GPIO, UART and Timer, the Library Manager for adding the retarget-io middleware, and application code for print logs, key capture on a terminal, and a timer interrupt to blink the LED every 1 second.

5 Code Example 2: IPC Semaphore using PDL

5.1 Objective

This lab exercise demonstrates how to use the inter-processor communication (IPC) PDL driver to implement a simple semaphore scheme with PSOC™ Edge. A semaphore is used to synchronize start up and manage access to a common resource, a UART, between the CPUs.

5.2 Description

In this code example, at device reset, the secure boot process starts from the ROM boot with the secure enclave (SE) as the root of trust (RoT). From the secured enclave, the boot flow is passed on to the system CPU subsystem where the secure CM33 application starts. After all necessary secure configurations, the flow is passed on to the non-secure CM33 application. Resource initialization for this example is performed by this CM33 non-secure project. It configures the system clocks, pins, clock to peripheral connections, and other platform resources. It then enables the CM55 core using the `Cy_SysEnableCM55()` function. After this, the cores execute their respective firmware.

The same semaphore is also used as a method to synchronize the access to a shared resource by the two CPUs. In this case, a UART is the resource. Both CPUs print a message to the computer terminal through UART every time the user button is pressed. Before printing a message, both CPUs attempt to lock the semaphore. If it succeeds, it prints the message and then unlocks the semaphore afterwards. If the semaphore is already locked, it keeps trying to lock it until it succeeds.

The firmware has an option to disable the semaphore in the code. The semaphore is no longer locked by the CPUs by simply changing `#define ENABLE_SEMA` to `0U` in the `ipc_def.h` file; additionally, the kit LED turns ON. Without the semaphore to synchronize access to the UART, messages from both CPUs might overlap. See the flowchart to see visually how the code will operate.

5.3 IPC Semaphore Code Example Flowchart

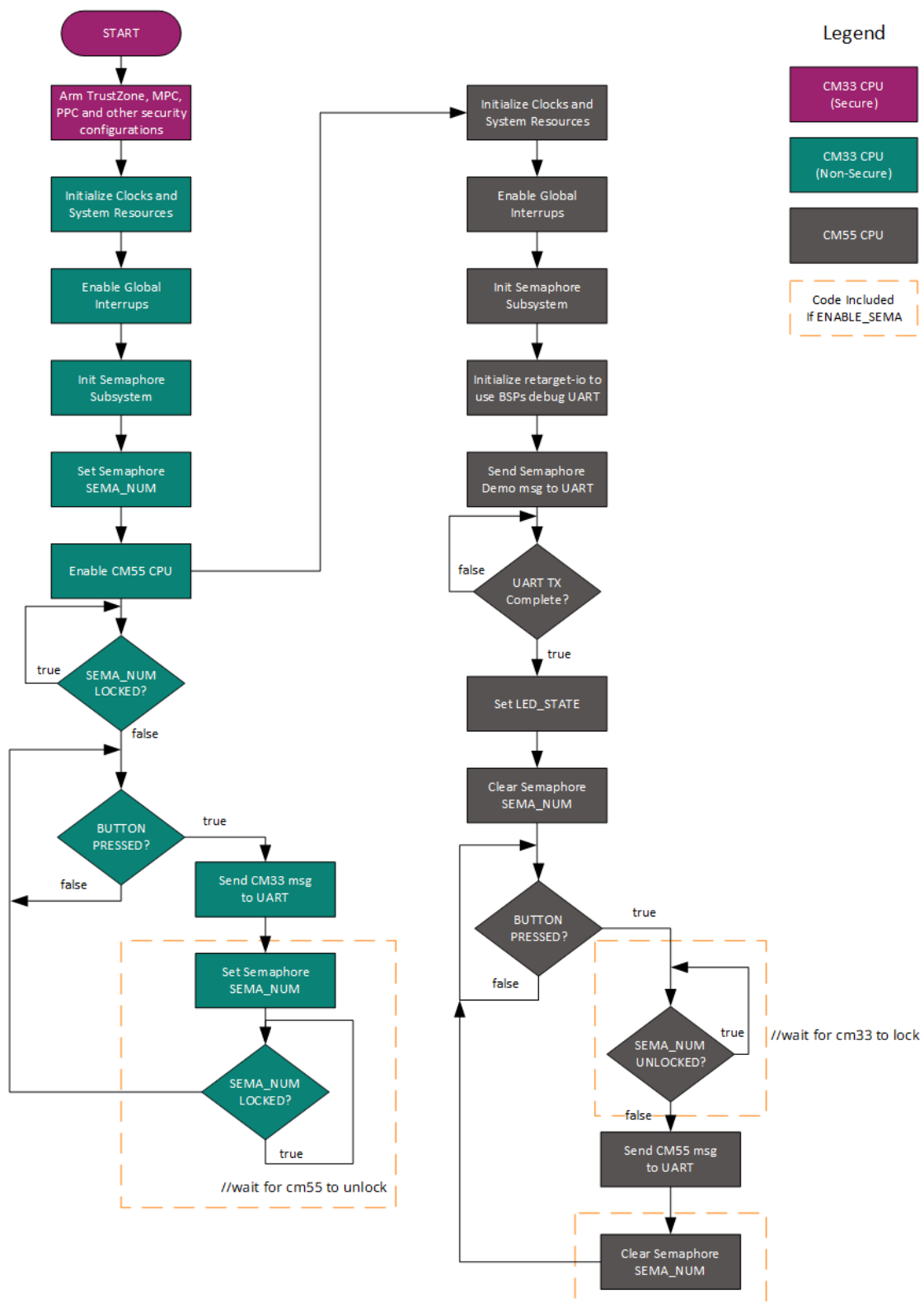


Figure 38 IPC Semaphore Code Example Flow

5.4 Hardware Setup

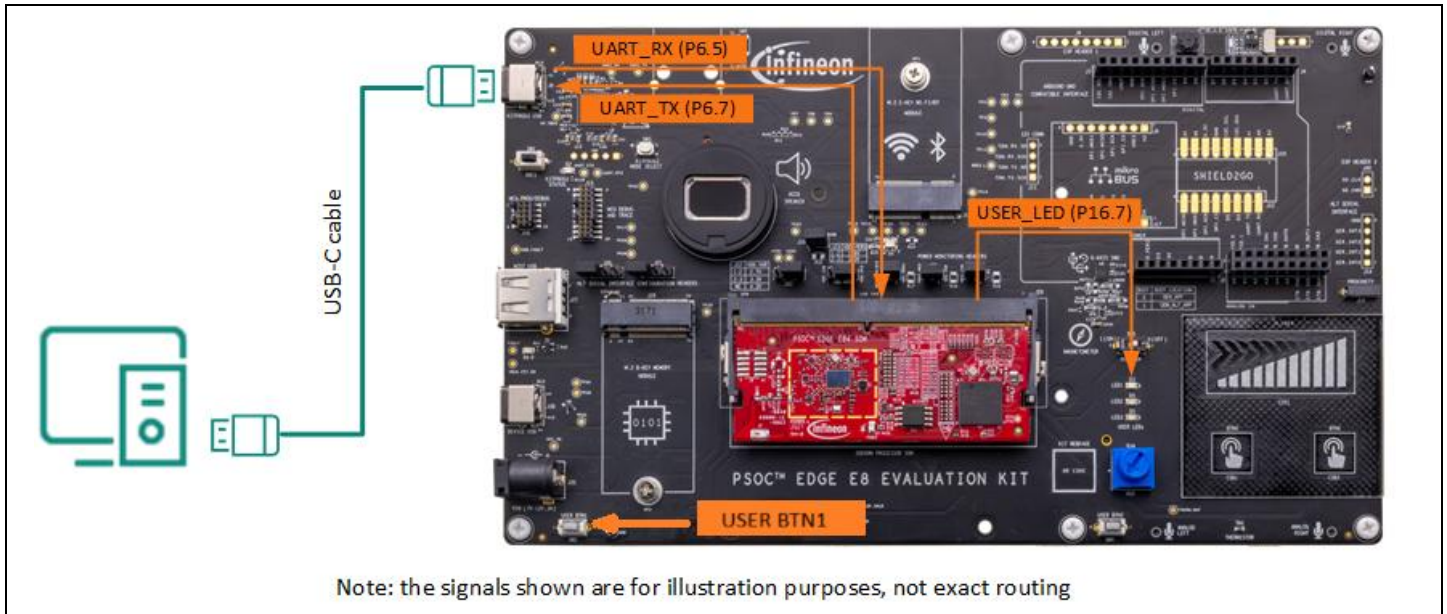


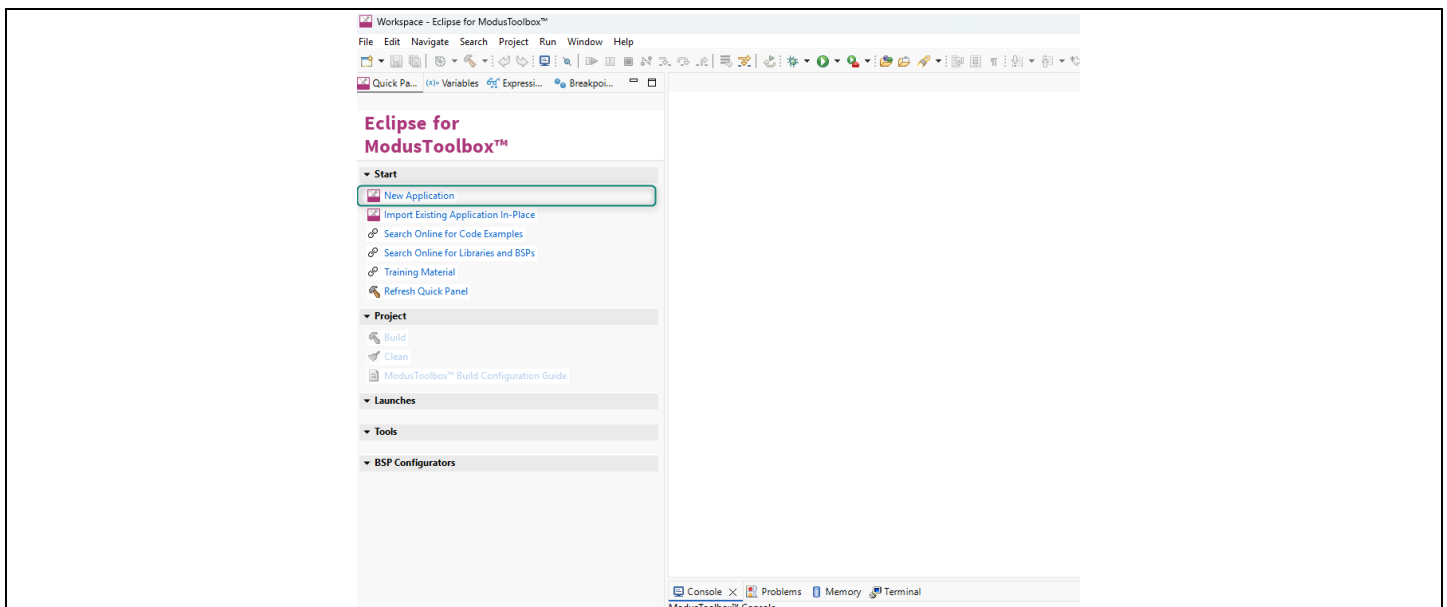
Figure 39 Hardware setup with USB cable for programming and serial UART

5.5 Project Creation

1. Since we are already familiar with the project creation process, we will start by creating a New Application.

We will start with a PSOC™ Edge Empty Application and turn it into an PSOC™ Edge IPC Semaphore using PDL application.

2. Click **New Application** from the Quick Panel.



Introduction to PSOC™ Edge E84

Getting Started with the Development Ecosystem

Code Example 2: IPC Semaphore using PDL



Figure 40 Start a New Application

3. Clicking New Application will open the Project Creator Tool and prompt you to choose a BSP (Board Support Package).
4. Once Project creator launches, Select the **KIT_PSOCE84_EVK_EPC2** BSP under the PSOC™ Edge or Detected Devices BSPs. Then click **Next**.

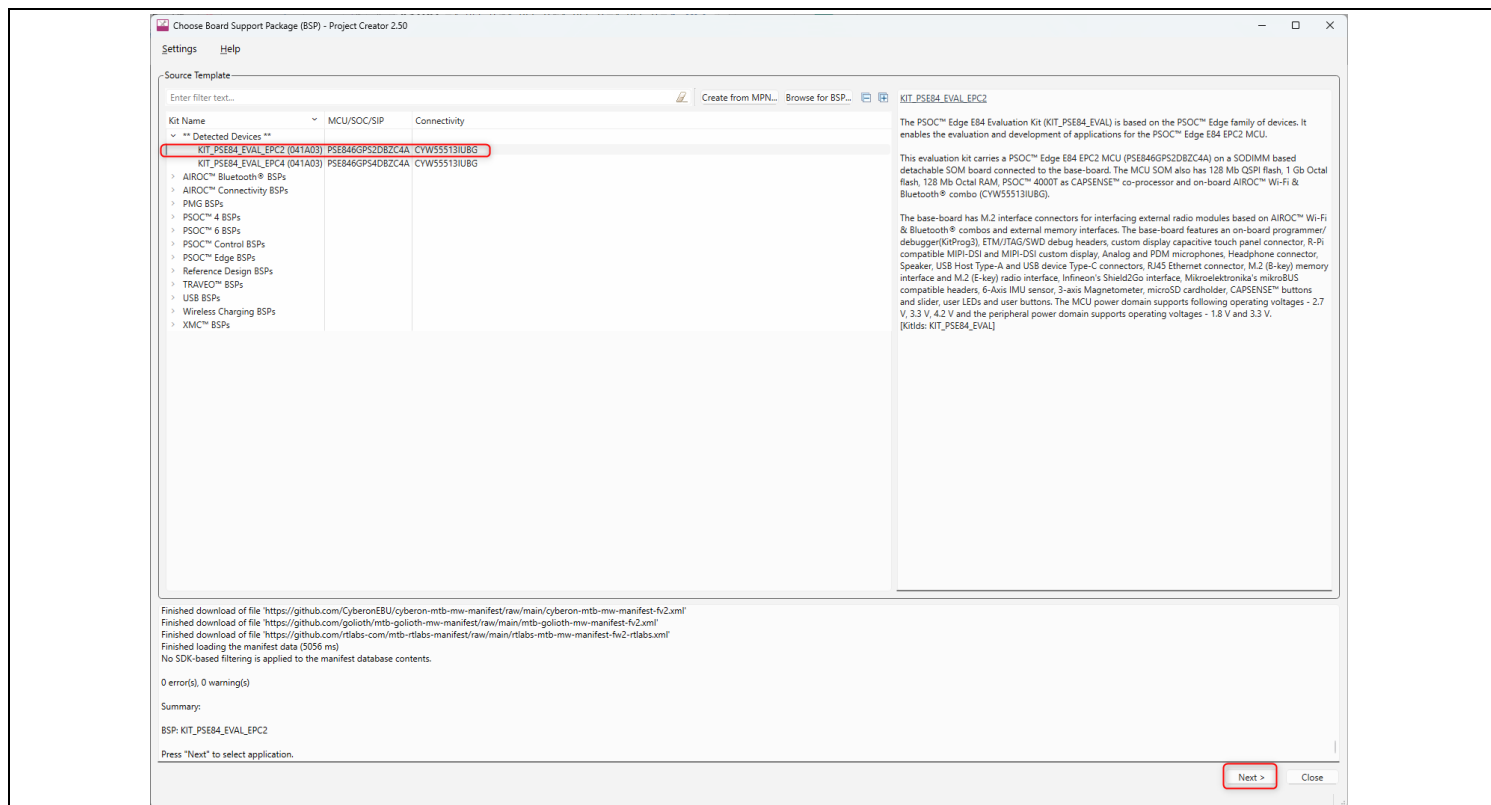


Figure 41 Select BSP

5. Clicking Next will take you to the next menu of the Project Creator which will ask you to select an application. Select the **PSOC Edge Hello World** under the Getting Started section. You can do this by selecting the checkbox near the application. Change the name to **“PSOC_Edge_IPC_Semaphore”** as we will be developing the Hello World application into a dual-core IPC application.
6. Click on **Create**.

Introduction to PSOC™ Edge E84

Getting Started with the Development Ecosystem

Code Example 2: IPC Semaphore using PDL

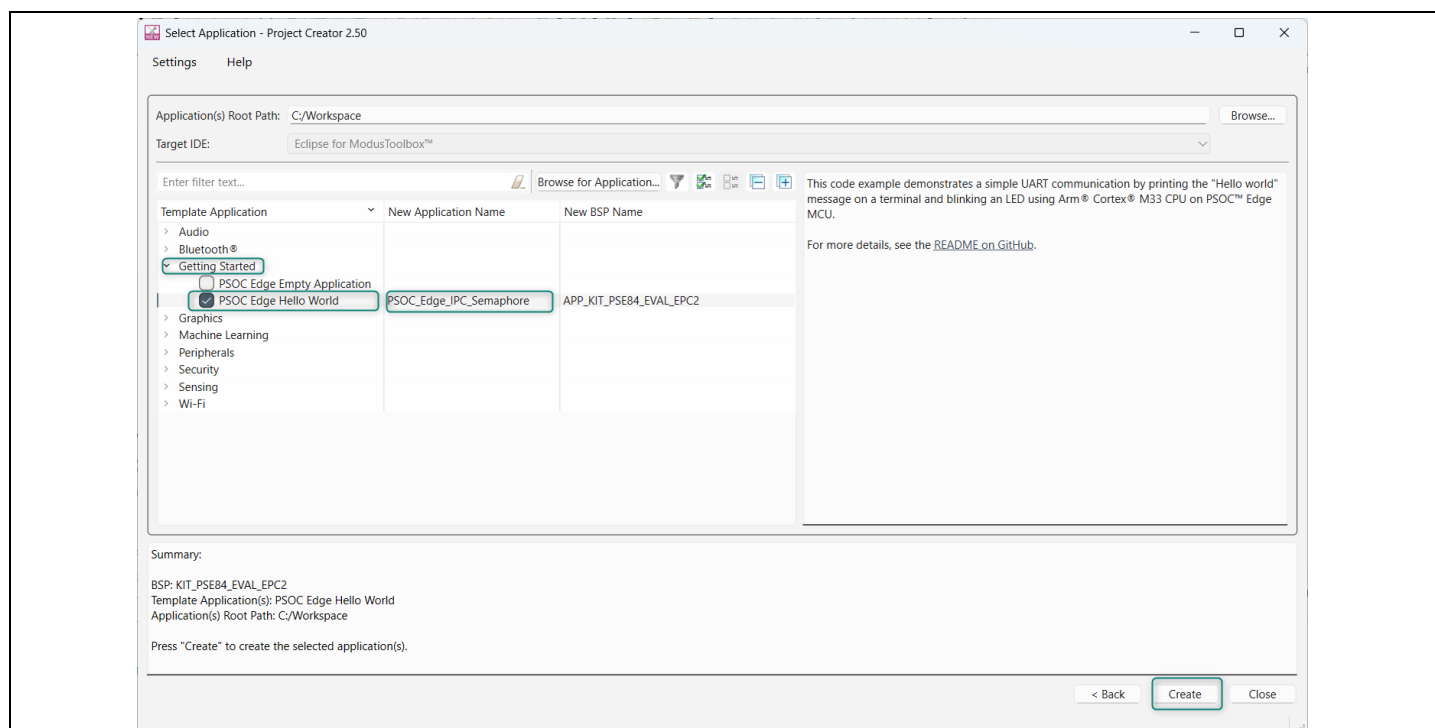


Figure 42 Select Template Application – Rename Application - Create

7. If you expand the main project folder, you can see that there are multiple sub-folders within it.

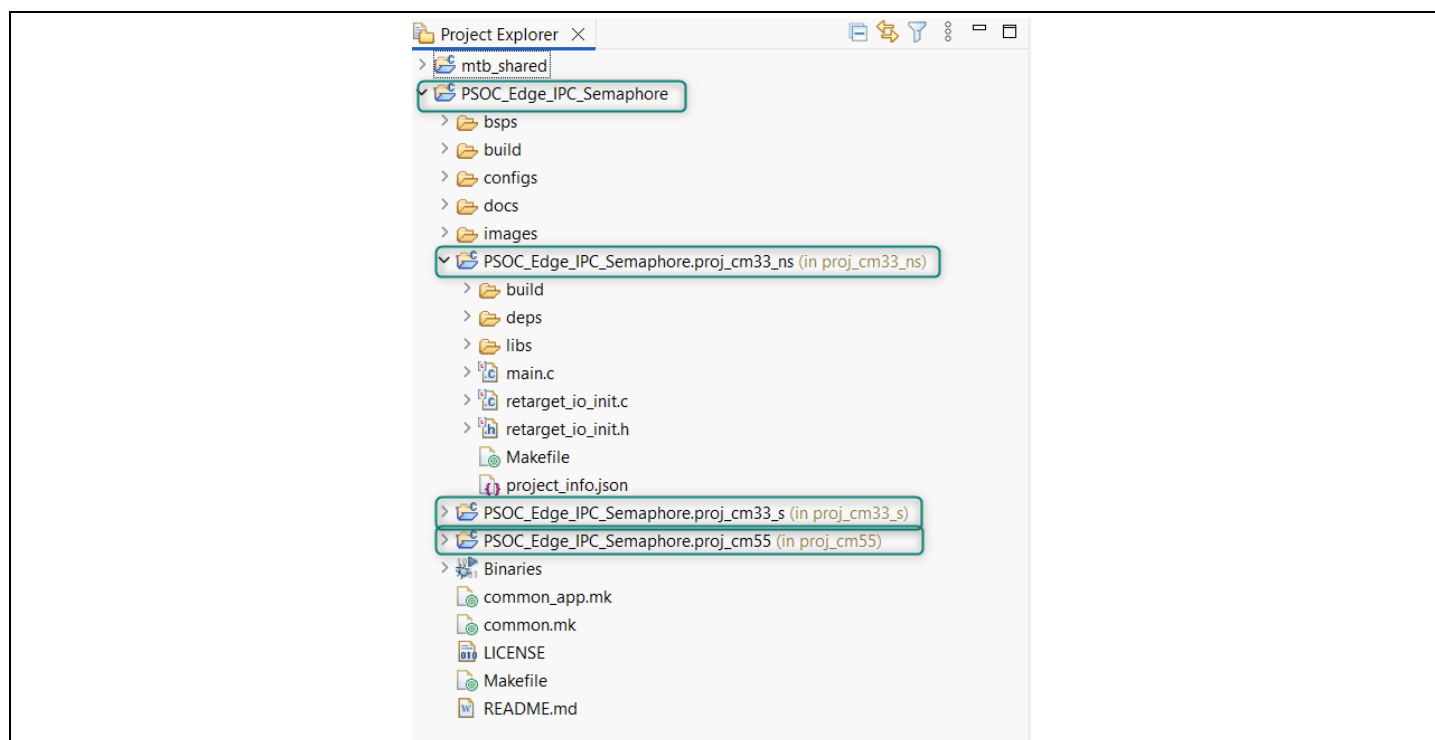


Figure 43 Project Explorer View

The Hello_World project uses the retargetIO in proj_cm33_ns. We will remove it the cm33_ns project using the **Library Manager** tool. The Library Manager can be found in the Quick Panel > Tools menu. Also, **retarget-io** is needed for our project for using the UART functionalities in proj_cm55. This too can be added using the Library Manager.

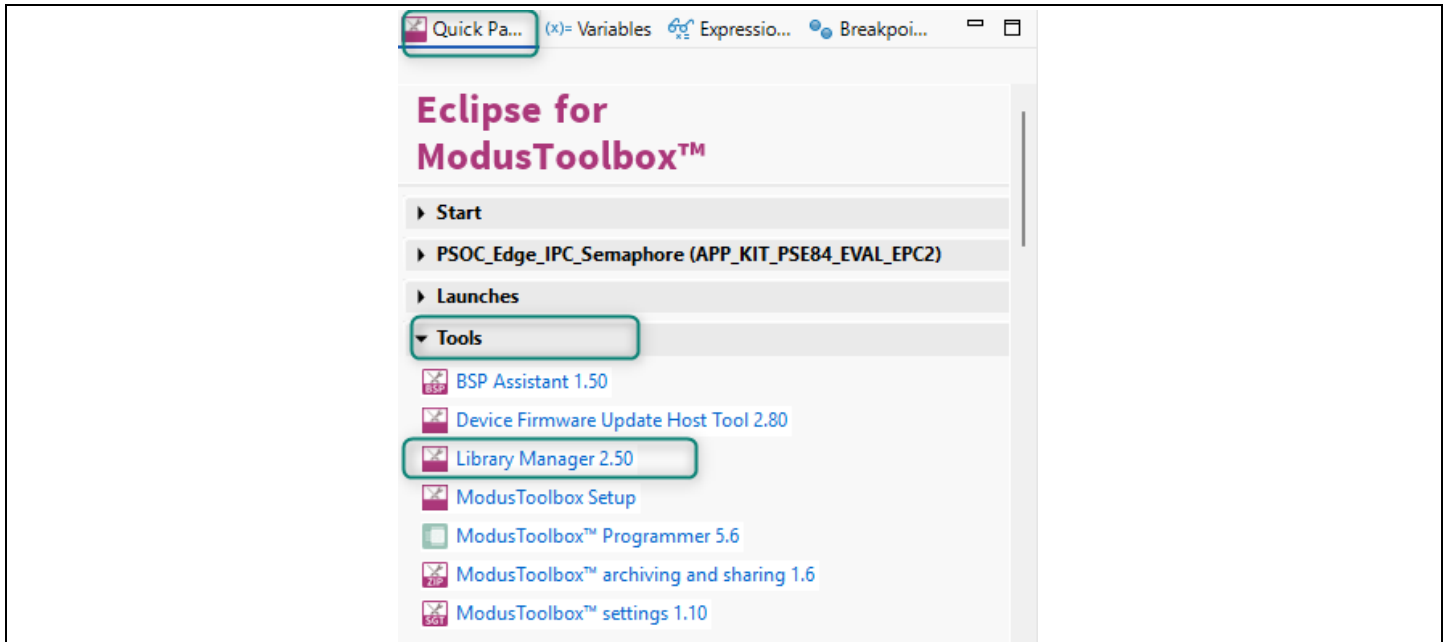


Figure 44 Opening the Library Manager from the Quick Panel

8. Select the PSOC_Edge_IPC_Semaphore Project in the mtb project explorer and open the **Library Manager** from quick panel. You will be able to see the **retarget-io** within the proj_cm33_ns libraries group. Click the 'X' icon to remove the library.

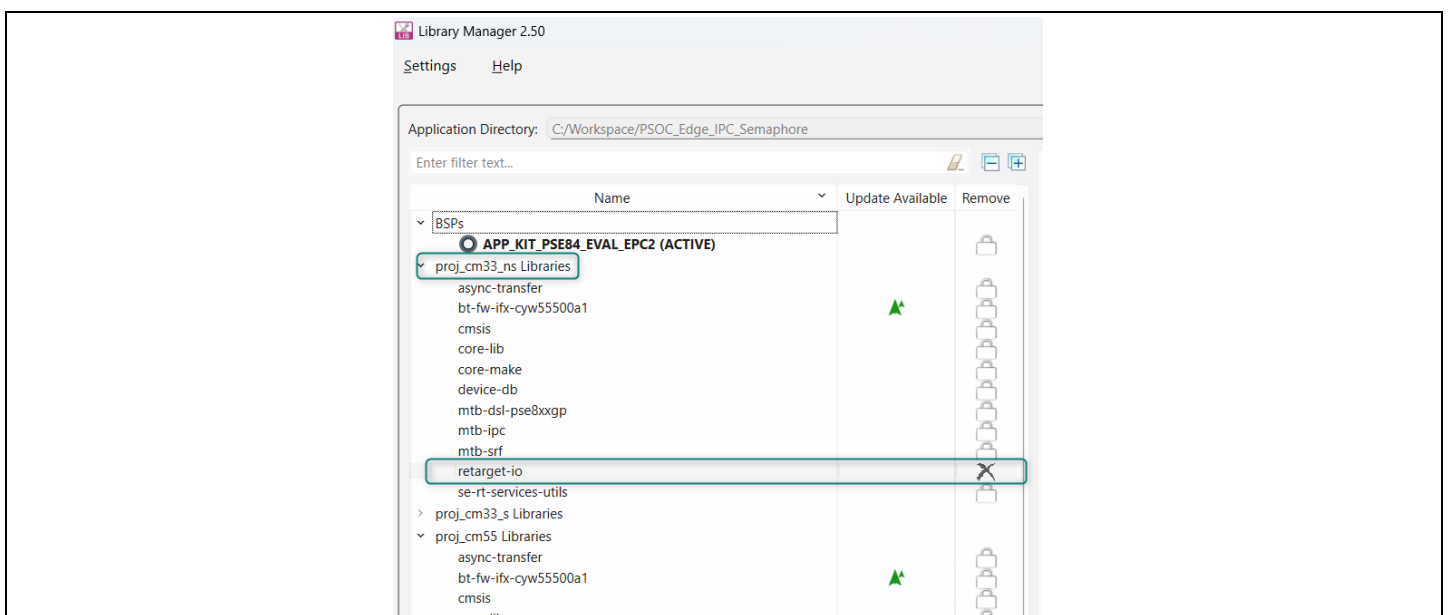


Figure 45 Library Manager – Remove retarget-io from cm33_ns project

Now, click the **Add Library** option. A menu will open. Change the **Target Project** to **proj_cm55** and Search for **retarget-io**. It will be found under the **Peripheral** section. Select the **retarget-io** option and click **OK**.

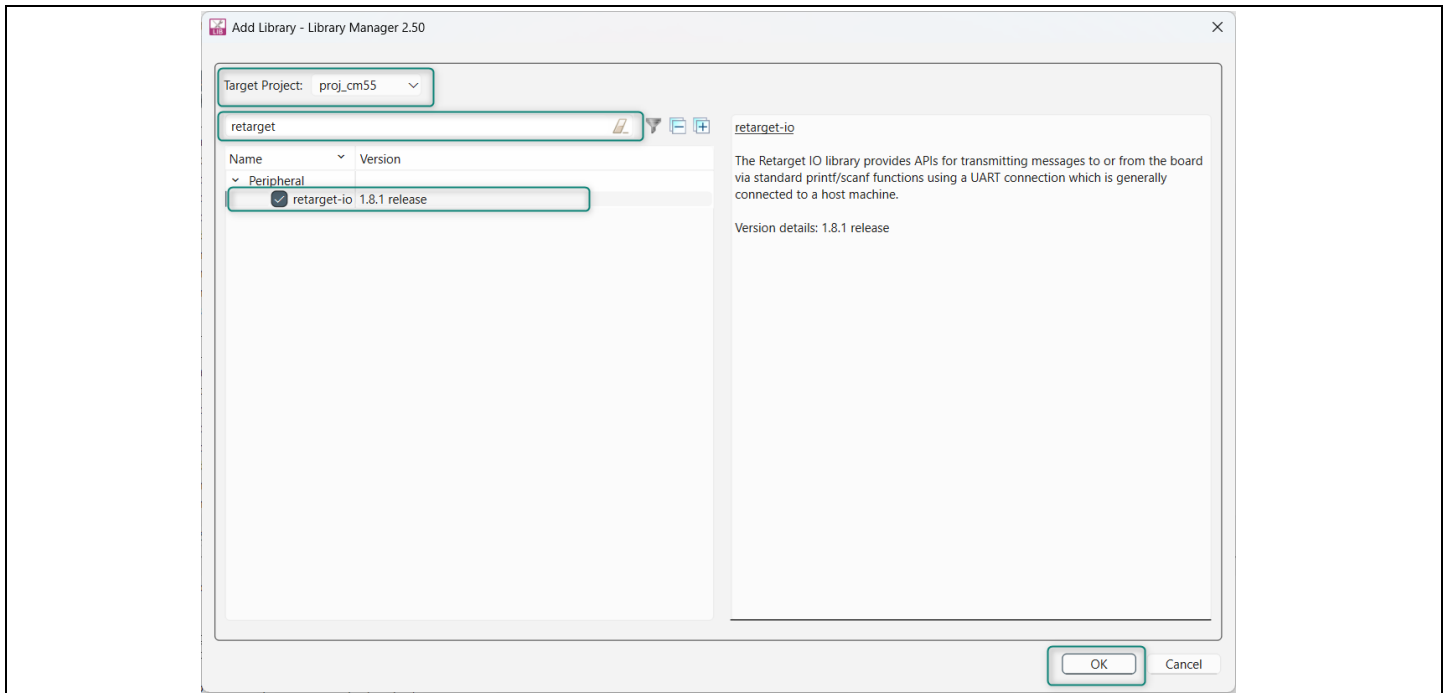


Figure 46 Library Manager – Adding retarget-io middleware to the cm55 project

Now, click **Update** on the Library Manager.

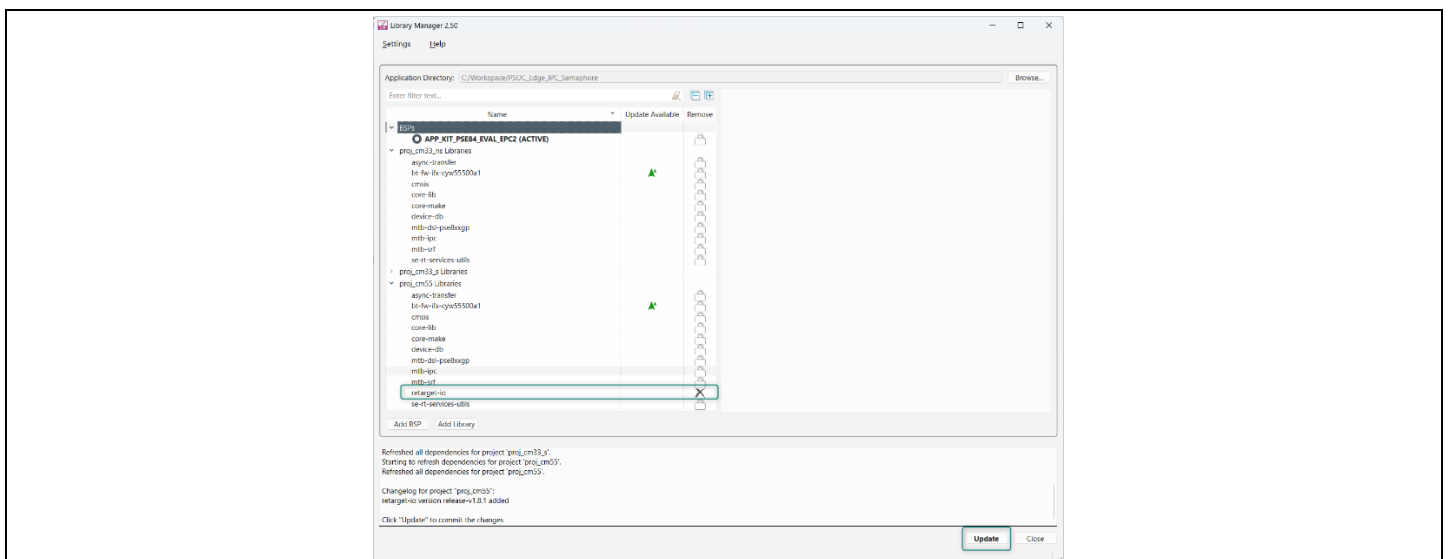


Figure 47 Library Manager – Update changes to retarget-io

9. Library Manager will update all the changes made to the PSOC_Edge_IPC_Semaphore application.
10. Since we removed the `retarget_io` middleware from the `cm33_ns` project and add it to the `cm55` project, we also need to move the files `retarget_io_init.c` and `retarget_io_init.h` from the `PSOC_Edge_IPC_Semaphore.proj_cm33_ns` folder to the `PSOC_Edge_IPC_Semaphore.proj_cm55` folder.

Back in the Project Explorer of the Eclipse IDE, open the `PSOC_Edge_IPC_Semaphore.proj_cm33_ns` folder and right click on the `retarget_io_init.c` file and select Move...

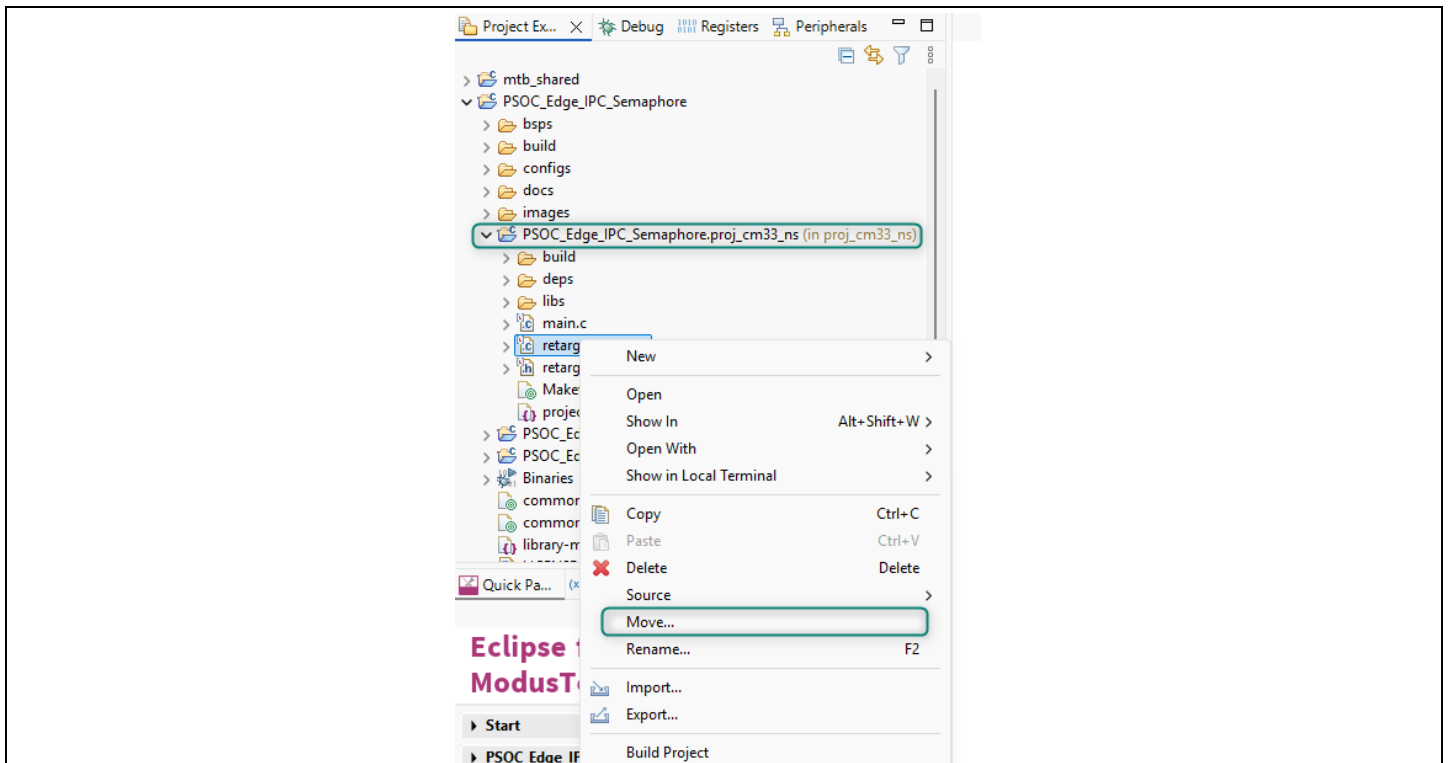


Figure 48 Moving `retarget_io_init.c` and `.h` from `cm33_ns` project to `cm55` project

Then select the `cm455` project folder as the destination folder

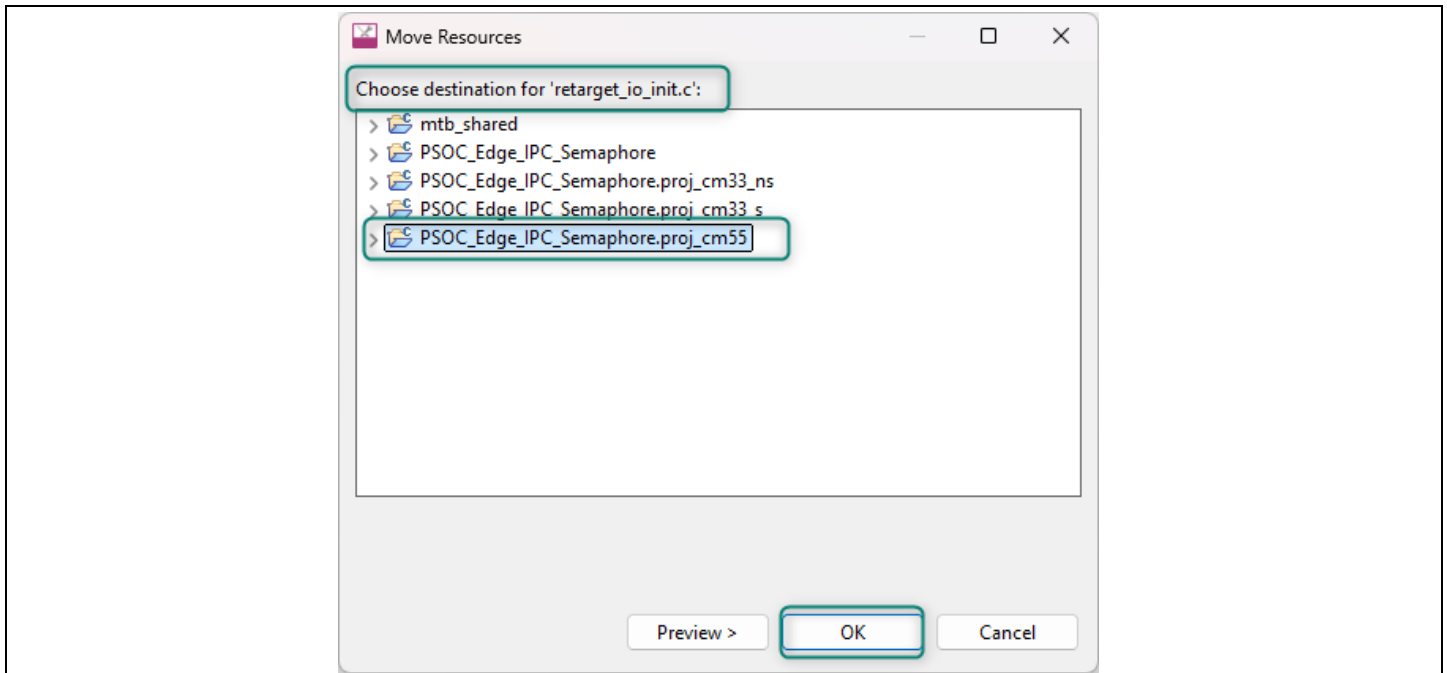


Figure 49 Select the PSOC_Edge_IPC_Semaphore.proj_cm55 folder as the destination for retarget_io_init.c

Do the same procedure for the retarget_io_init.h file, moving it to the PSOC_Edge_IPC_Semaphore.proj_cm55 folder. In the Project Explorer, verify that both files are now located in the PSOC_Edge_IPC_Semaphore.proj_cm55 folder as shown in the figure below

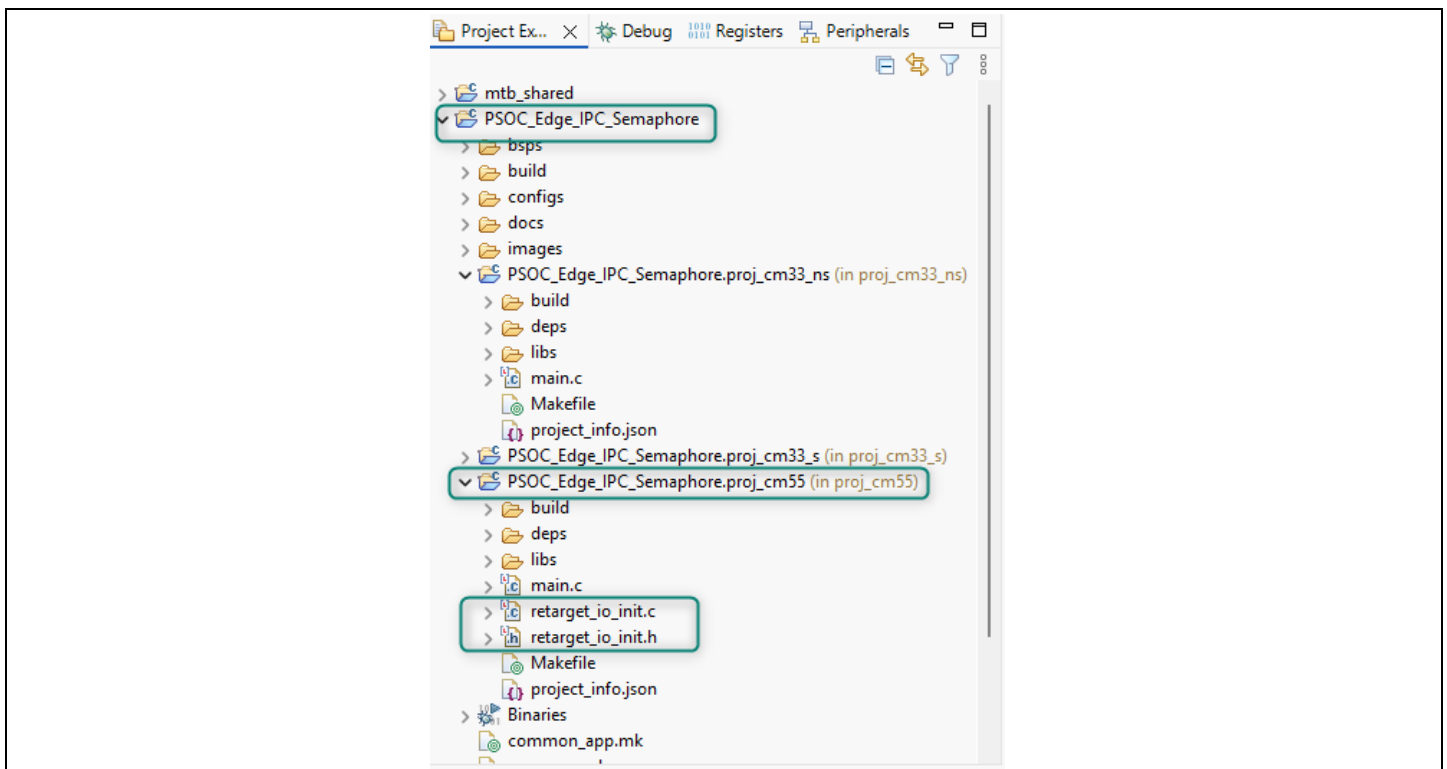


Figure 50 Verification of moving retarget_io_init.c and .h to the cm55 project folder

Code Example 2: IPC Semaphore using PDL

11. Now we need to replace the starter code for the main.c files in folders named PSOC_Edge_IPC_Semaphore.proj_cm33_ns and PSOC_Edge_IPC_Semaphore.proj_cm55. Each of these folders contain a **main.c** file which contains the application starter code. **Replace** the code in these main.c files with the code from the main.c template files we have provided. (The template main.c files have required Macros, definitions and basic initialization.)

Copy the *contents* from Lab_Source\templates\ PSOC_Edge_IPC_Semaphore \main_cm33_ns.c and paste into PSOC_Edge_IPC_Semaphore.proj_cm33_ns main.c replacing the default code.

Copy the *contents* from Lab_Source\templates\ PSOC_Edge_IPC_Semaphore \main_cm55.c and paste into PSOC_Edge_IPC_Semaphore.proj_cm55. main.c replacing the code.

12. After replacing the code in both main.c file, make sure to **Save** them both, and the **Build** the Application and make sure nothing as broken at this point. Remember to select the PSOC_Edge_ICP_Semaphore application folder before using the Quick Panel > PSOC_Edge_IPC_Semaphore (APP_KIT_PSE84_EVAL_EPC2) > Build Application command.
13. Now let's add the application code.
14. In Windows Explorer (or the equivalent folder explorer on your OS), copy the **shared** folder from Lab_Source\templates\PSOC_Edge_IPC_Semaphore\ , navigate to the workspace folder on your local drive, locate the application folder for PSOC_Edge_IPC_Semaphore and paste the shared folder there.

This shared folder contains the “ipc_def.h” file, which contains definitions of constants and structures for setting up user semaphores and function prototypes for configuring the IPC subsystem.

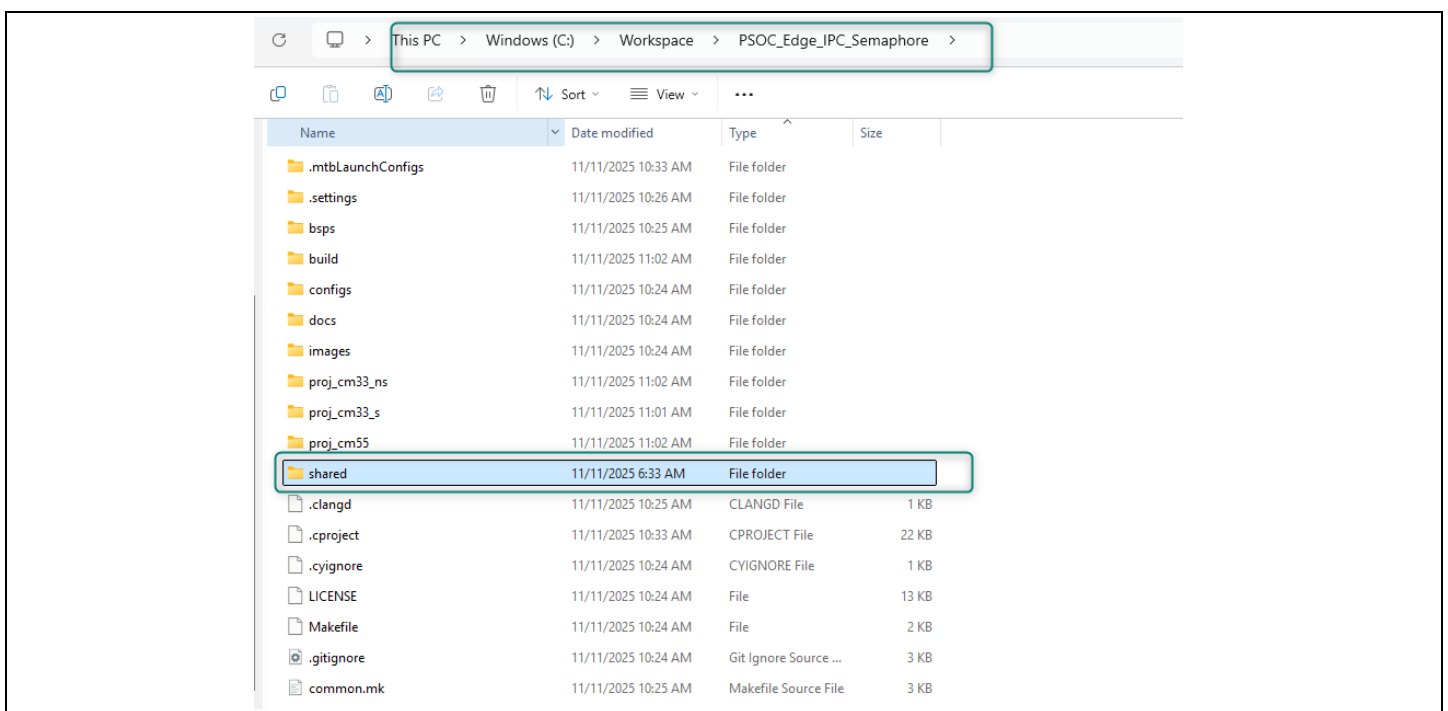


Figure 51 Placing a copy of the shared folder into the PSOC_Edge_IPC_Semaphore application folder

15. In order for the build system to be able to utilize the shared folder we need to modify the **Makefile** to include the “ipc_def.h” file folder location. In the Project Explorer, open the **Makefile** for project PSOC_Edge_IPC_Semaphore.proj_cm33_ns

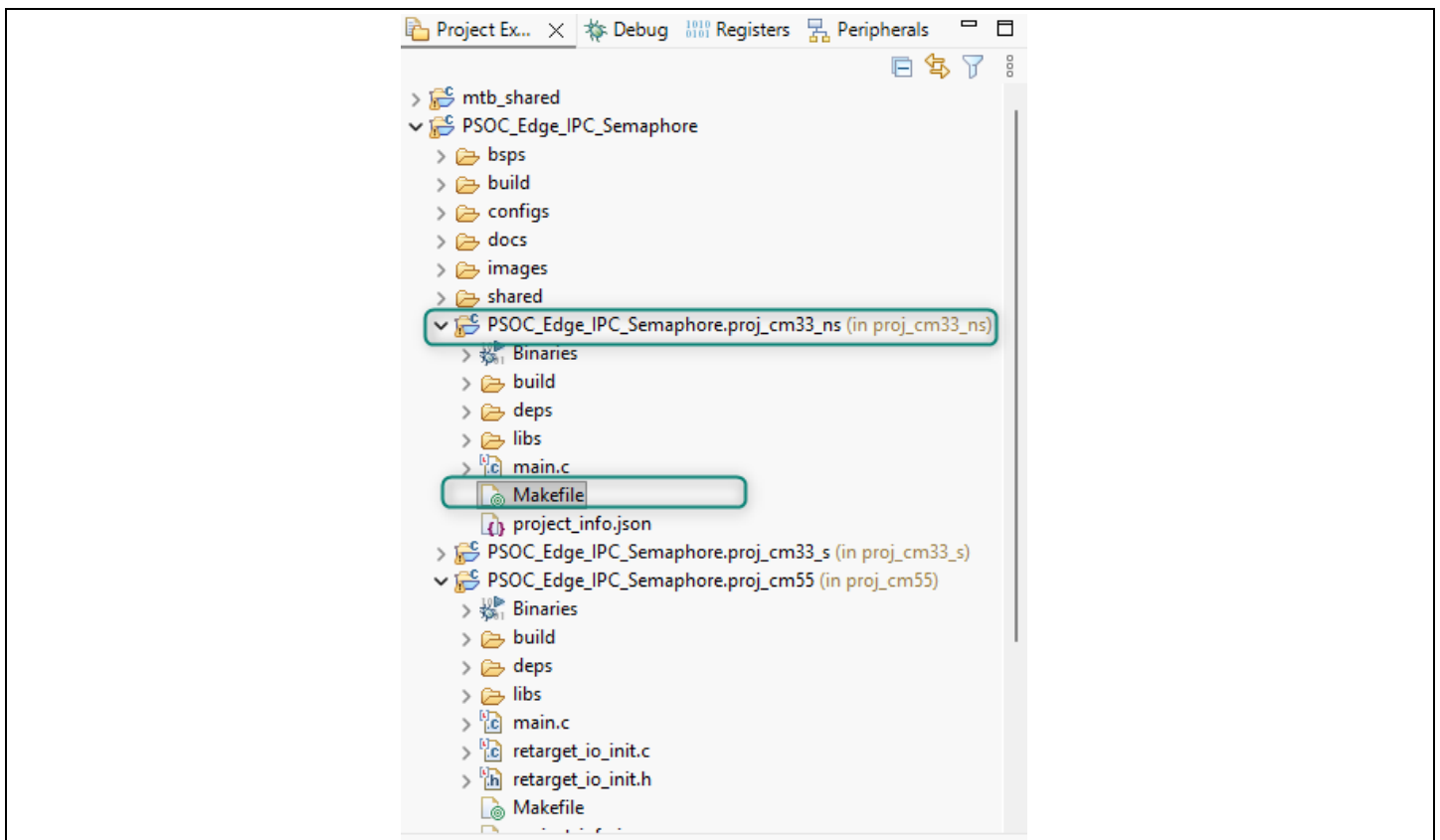


Figure 52 Opening the cm33_ns project Makefile

16. Inside the **Makefile**, navigate to the Advance Configuration section and change the **INCLUDES=** directive to **INCLUDES=../shared/include** and **Save** and **Close** the file



```

43 # Advanced Configuration
44 #####
45
46 # Enable optional code that is ordinarily disabled by default.
47 #
48 # Available components depend on the specific targeted hardware and firmware
49 # in use. In general, if you have
50 #
51 #   COMPONENTS=foo bar
52 #
53 # ... then code in directories named COMPONENT_foo and COMPONENT_bar will be
54 # added to the build
55 #
56 COMPONENTS+=
57
58 # Like COMPONENTS, but disable optional code that was enabled by default.
59 DISABLE_COMPONENTS=
60
61 CORE=CM33
62 CORE_NAME=CM33_0
63
64 # By default the build system automatically looks in the Makefile's directory
65 # tree for source code and builds it. The SOURCES variable can be used to
66 # manually add source code to the build process from a location not searched
67 # by default, or otherwise not found by the build system.
68 SOURCES+=
69
70 # Like SOURCES, but for include directories. Value should be paths to
71 # directories (without a leading -I).
72 INCLUDES+=../shared/include
73
74 # Add additional defines to the build process (without a leading -D).
75 DEFINES+=CY_RETARGET_IO_CONVERT_LF_TO_CRLF
76
77 # Select softfp or hardfp floating point. Default is softfp.
78 VFP_SELECT+=
79
80 # Additional / custom C compiler flags.
81 #

```

Figure 53 Modifying the INCLUDES directive of the cm33_ns project's Makefile

****Make sure to make the same change to the INCLUDES directive in the Makefile for the cm55 project as both projects will be utilizing the ipc_def.h header file.****

17. Open the **main.c** file under **PSOC_Edge_IPC_Semaphore.proj_cm33_ns**. We will be adding the necessary code to this file for **CM33** Core.

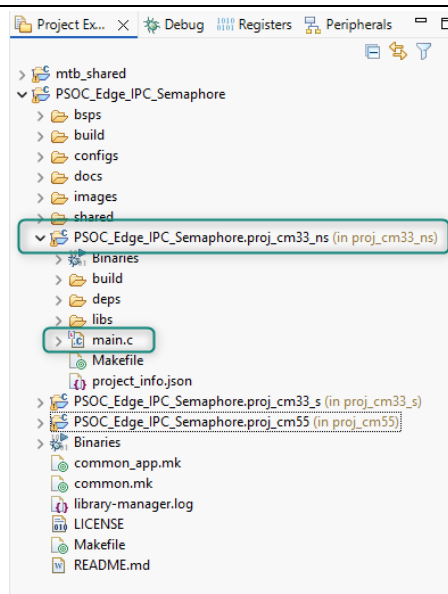


Figure 54 Opening the cm33_ns project main.c file from the Project Explorer

Code Example 2: IPC Semaphore using PDL

18. Add IPC communication header file to the **main.c**

Copy and paste the code provided within the Code listing below to the **main.c** file after other #includes near the beginning of the file.

Code listing 13 Include ipc_def.h in cm33_ns main.c

```
#include "ipc_def.h"
```

19. Add the **ipc_status** variable in the main function

Copy and paste the code provided within the Code listing below to the main function after the associated comment /* Initialize IPC subsystem */

Code listing 14 Local variable to main function for ipc_status

```
cy_en_ipcsema_status_t ipc_status;
```

20. Add the code to initialize the IPC semaphore subsystem

Copy and paste the code provided below within the Code listing within to the main function after the associated comment /* variable to hold ipc_status*/

Code listing 15 Code snippet for initializing IPC Semaphore Subsystem

```
ipc_status = Cy_IPC_Sema_Init(IPC_CHANNEL_NUM, CY_IPC_SEMA_COUNT, sema_data);

if (CY_IPC_SEMA_SUCCESS != ipc_status)
{
    /* Disable all interrupts. */
    __disable_irq();

    CY_ASSERT(0);
    /* Infinite loop */
    while(true);
}
```

21. Have the cm33_ns code lock the semaphore in preparation of starting up the cm55

Copy and paste the code provided below within the Code listing within to the main function after the associated comment /* lock IPC semaphore SEMA_NUM*/

Code listing 16 Code snippet for locking the IPC Semaphore SEMA_NUM

```
Cy_IPC_Sema_Set(SEMA_NUM, false);
```

22. The cm33_ns application has enabled the cm55 CPU, so now we need to wait until the cm55 initializes and releases/unlocks the semaphore.

Copy and paste the code provided below within the Code listing within to the main function after the associated comment `/* Wait for cm55 to finish startup and release the semaphore*/`

Code listing 17 Code snippet for wait for cm55 to complete initialization

```
while(Cy_IPC_Sema_Status(SEMA_NUM) == CY_IPC_SEMA_STATUS_LOCKED) {}
```

23. With the cm33 setup and the cm55 having completed its initialization we enter the main forever for loop. In the loop we will

- Check if the user has pressed the USER_BTN1
- If the we detect a button press
 - Send the cm33 message to the UART
 - Set/lock the semaphore SEMA_NUM
 - Wait for the cm55 to clear/unlocked the semaphore SEMA_NUM

Copy and paste the code provided below within the Code listing within the infinite for loop at the end of the main function

Code listing 18 Code snippet for cm33_ns main forever for loop

```
/* If there is a user button press */
if (Cy_GPIO_Read(CYBSP_USER_BTN_PORT, CYBSP_USER_BTN_PIN) == 0)
{
    /* Send cm33 message to the UART */
    Cy_SCB_UART_PutString(CYBSP_DEBUG_UART_HW, "Message sent from CM33 \r\n");

    #if ENABLE_SEMA
        /* Lock the semaphore */
        Cy_IPC_Sema_Set(SEMA_NUM, false);

        /* Wait for CM55 to release the semaphore */
        while(Cy_IPC_Sema_Status(SEMA_NUM) == CY_IPC_SEMA_STATUS_LOCKED) {}
    #endif
}
```

24. This is a good time to make sure the application still builds without errors. Remember to select the **PSOC_Edge_IPC_Semaphore** application folder before using the Quick Panel > PSOC_Edge_IPC_Semaphore (APP_KIT_PSE84_EVAL_EPC2) > **Build Application** command.
25. Now that the application is building without errors, we can move on to the cm55 project. In the Project Explorer, open the **PSOC_Edge_IPC_Semaphore.proj_cm55** folder and double click the **main.c** file in this folder to open main.c

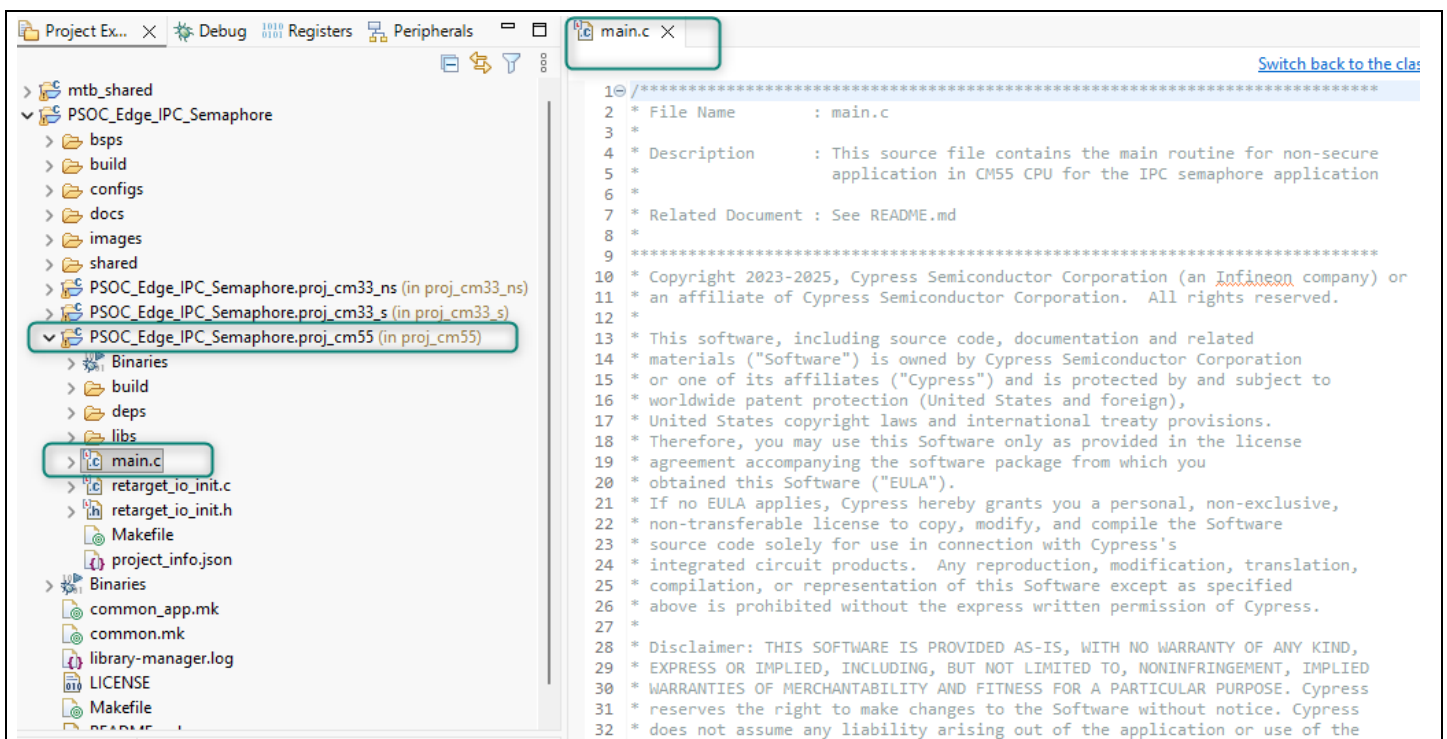


Figure 55 Open main.c in the PSOC_Edge_IPC_Semaphore.proj_cm55 project folder

26. Like we did for the cm33_ns project, add the IPC communication header file to the **main.c**

Copy and paste the code provided within the Code listing below to the **main.c** file after other #includes near the beginning of the file.

Code listing 19 Include ipc_def.h in cm55 main.c

```
#include "ipc_def.h"
```

27. Now, let's add the Macro for the LED_STATE.

Copy and paste the code provided within the Code listing below to the Macros section of the file after the associated comment /* Define the state of the USER_LED based on whether a semaphore is used*/

Code listing 20 Local variable to main function for ipc_status

```
#if ENABLE_SEMA
#define LED_STATE    CYBSP_LED_STATE_OFF
#else
#define LED_STATE    CYBSP_LED_STATE_ON
#endif
```

28. Add the `ipc_status` variable in the main function

Copy and paste the code provided within the Code listing below to the main function after the associated comment /* Initialize IPC subsystem */

Code listing 21 Local variable to main function for ipc_status

```
cy_en_ipcsema_status_t ipc_status;
```

29. Next, we need to add code for the cm55 to initialize the IPC subsystem. Note that the cm33 already setup the default semaphore count for the channel and a shared memory buffer for the IPC subsystem, so the cm55 will initialize with 0 count and a NULL pointer. Below is an excerpt from the [IPC PDL Semaphore layer initialization](#) documentation.

◆ Cy_IPC_Sema_Init()

```
cy_en_ipcsema_status_t Cy_IPC_Sema_Init ( uint32_t  ipcChannel,
                                           uint32_t  count,
                                           uint32_t  memPtr[]
                                           )
```

This function initializes the semaphores subsystem.

The user must create an array of unsigned 32-bit words to hold the semaphore bits. The number of semaphores will be the size of the array * 32. The total semaphores count will always be a multiple of 32.

Either this function or `Cy_IPC_Sema_InitExt` must be called from each core that will use IPC semaphores.

Note

In a multi-CPU system this init function should be called with all initialized parameters on one CPU only to provide a pointer to SRAM that can be shared between all the CPUs in the system that will use semaphores. On other CPUs user must specify the IPC semaphores channel and pass 0 / NULL to count and memPtr parameters correspondingly.

If semaphores will be used from within a secure processing environment, the secure processing environment must call either this function or `Cy_IPC_Sema_InitExt` with non-0 / NULL arguments, and it must do so before launching any other processing environment.

On cores that support DCache, user needs to make sure that the memPtr pointer passed to the Cy_IPC_Sema_Init function points to 32 byte aligned array of words that contain the semaphore data. User can use CY_ALIGN(32) macro for 32 byte alignment.

If semaphores will be used from within a secure processing environment, the secure processing environment must **Cy_IPC_Sema_InitExt** with non-0 / NULL arguments. This must be done even if this function or **Cy_IPC_Sema_InitExt** was previously called from another (non-secure) processing environment.

Parameters

ipcChannel The IPC channel number used for semaphores

count	The maximum number of semaphores to be supported (multiple of 32).
--------------	--

memPtr This points to the array of (count/32) words that contain the semaphore data. This must be located in memory which is accessible to all processing environments which will interact with semaphores

Returns

Status of the operation

Return values

CY_IPC_SEMA_SUCCESS Successfully initialized

CY_IPC_SEMA_BAD_PARAM	Memory pointer is NULL and count is not zero, or count not multiple of 32
------------------------------	---

CY_IPC_SEMA_ERROR_LOCKED Could not acquire semaphores IPC channel

Figure 56 DSL > PDL > IPC > IPC semaphores layer > Functions > Cy_IPC_Sema_Init() Documentation

Copy and paste the code provided below within the Code listing within to the main function after the associated comment `/* Initialize IPC subsystem */`

Code listing 22 Code snippet for cm55 initialization of the IPC Semaphore Subsystem

```
ipc_status = Cy_IPC_Sema_Init(IPC_CHANNEL_NUM, 0UL, NULL);
if(ipc_status != CY_IPC_SEMA_SUCCESS)
{
    handle_app_error();
}
```

30. Let's have the cm55 send out the IPC Semaphore Example header and then wait for the UART to complete transmitting the message.

Code Example 2: IPC Semaphore using PDL

Copy and paste the code provided within the Code listing below to the main function after the associated comment /* Send Header message out to UART*/

Code listing 23 Header message for the IPC Semaphore example to UART from cm55

```
/* \x1b[2J\x1b[;H - ANSI ESC sequence for clear screen */
printf("\x1b[2J\x1b[;H");

printf("***** PSOC Edge MCU: IPC Semaphore
Example ***** \r\n\n");

printf("<Press the kit's user button to print messages>\r\n\n");

/* Wait for UART traffic to stop */
while(!(Cy_SCB_UART_IsTxComplete(CYBSP_DEBUG_UART_HW))) {}
```

31. With the initial header message sent to UART, let's set the USER_LED according to whether the build includes the code within the #if ENABLE_SEMA or not. Note that when the LED is illuminated the main loop is running *without* semaphore control.

Copy and paste the code provided within the Code listing below to the main function after the associated comment /* Turn on LED when Semaphores are not being used*/

Code listing 24 Set GPIO to LED_STATUS indicating the use of semaphones or not

```
Cy_GPIO_Write(CYBSP_USER_LED1_PORT, CYBSP_USER_LED1_PIN, LED_STATE);
```

32. Now that the cm55 setup is complete, let's have the cm55 release/unlock the semaphore SEMA_NUM

Copy and paste the code provided within the Code listing below to the main function after the associated comment /* unlock IPC semaphore SEMA_NUM*/

Code listing 25 Release/unlock the semaphore SEMA_NUM

```
Cy_IPC_Sema_Clear(SEMA_NUM, false);
```

33. With the cm55 setup and having released/unlocked the semaphore letting the cm33_ns code know that the cm55 has completed its start up tasks, we enter the main forever for loop. In the loop we will

Code Example 2: IPC Semaphore using PDL

- Check if the user has pressed the USER_BTN1
- If we detect a button press
 - Wait for the cm33 to set/lock the semaphore SEMA_NUM
 - Send the cm55 message to the UART
 - Clear/unlock the semaphore SEMA_NUM

Copy and paste the code provided below within the Code listing within the infinite for loop at the end of the main function after the associated comment /* If there is a user button press */

Code listing 26 Code snippet for cm55 main forever for loop

```
if (Cy_GPIO_Read(CYBSP_USER_BTN_PORT, CYBSP_USER_BTN_PIN) == 0)
{
    #if ENABLE_SEMA
        /* Wait for CM33 to lock the semaphore */
        while(Cy_IPC_Sema_Status(SEMA_NUM) == CY_IPC_SEMA_STATUS_UNLOCKED) {}
    #endif
    /* Send cm55 message to the UART */
    printf("Message sent from CM55 \r\n");

    #if ENABLE_SEMA
        /* Release semaphore */
        Cy_IPC_Sema_Clear(SEMA_NUM, false);
    #endif
}
```

34. **Save** the file. We are done with adding the application code for both the cores.

35. Let's **build** by making sure the PSOC_Edge_IPC_Semaphore application folder is selected and then clicking on the **Build Application** option in the **Quick Panel**.

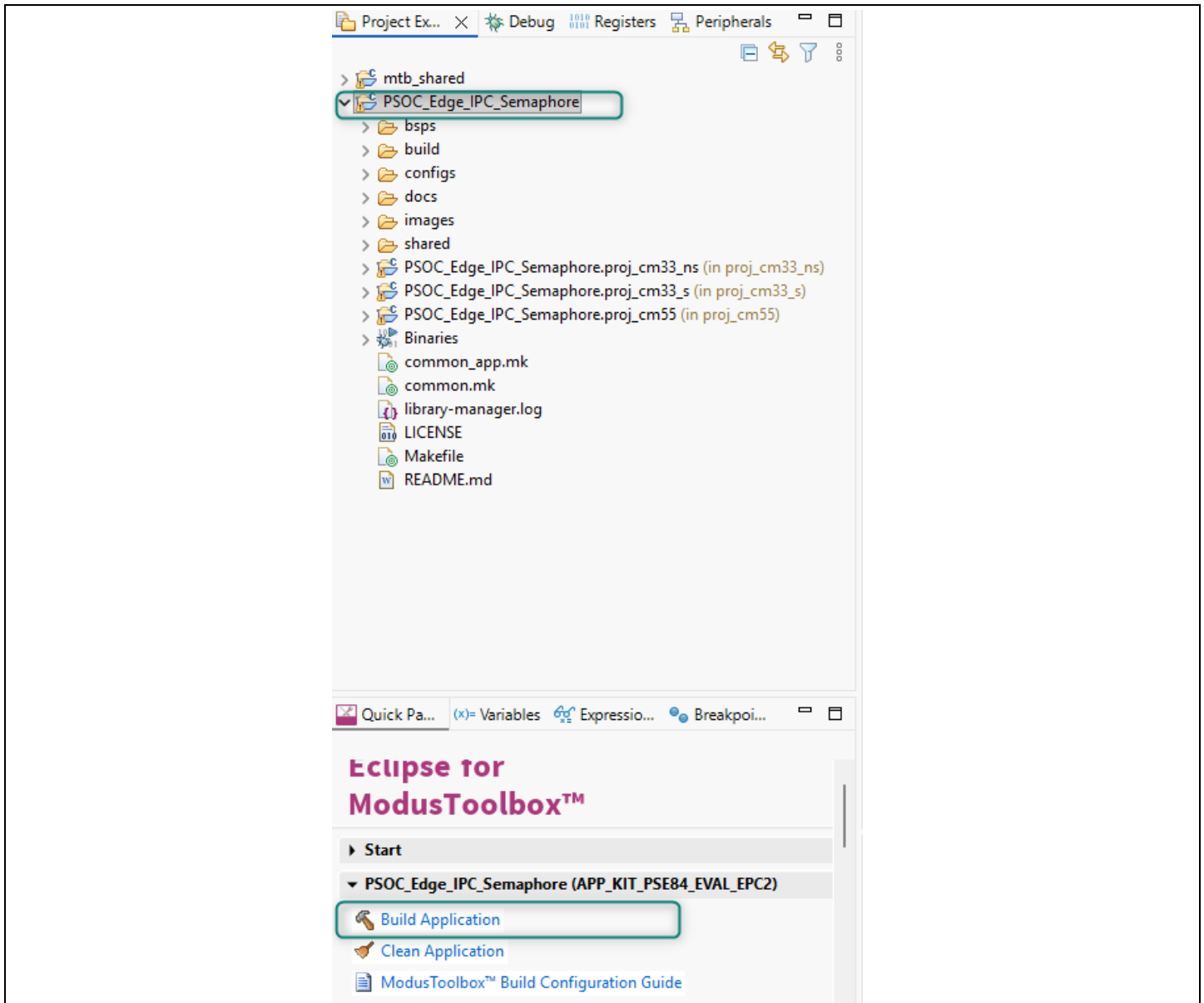


Figure 57 Building the PSOC_Edge_IPC_Semaphore application

36. After the Build process is successfully completed. We can **program** the board by clicking the using the Quick Panel > Launches > PSOC_Edge_IPC_Semaphore Program Application button.

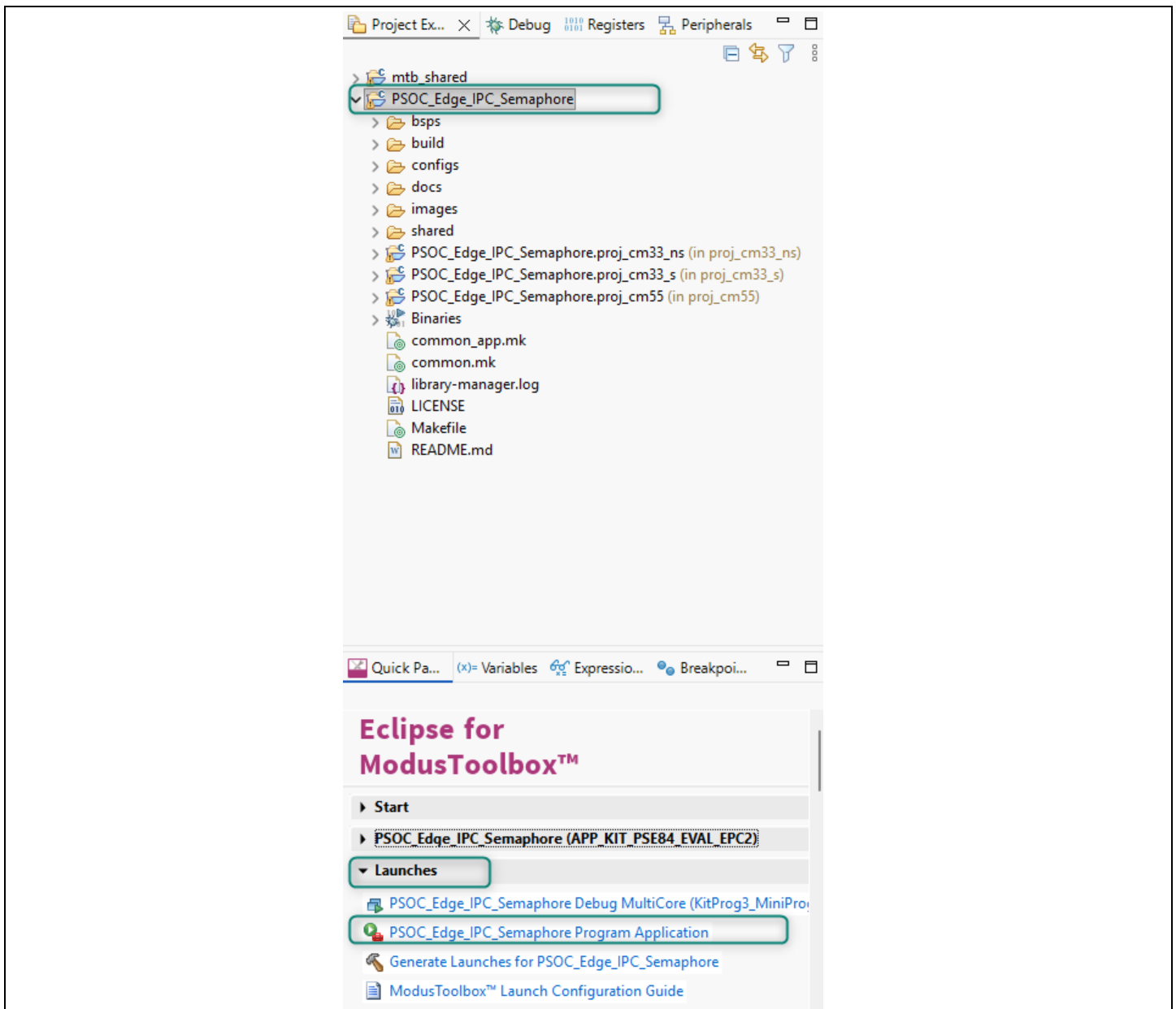


Figure 58 Programming the PSOC_Edge_IPC_Semaphore application to the target EVK

43. After programming is successful, Open a Serial Terminal inside the Eclipse IDE and configure with a baud rate 115200 and 8-N-1 then restart the EVK. You will see the message as shown in below.

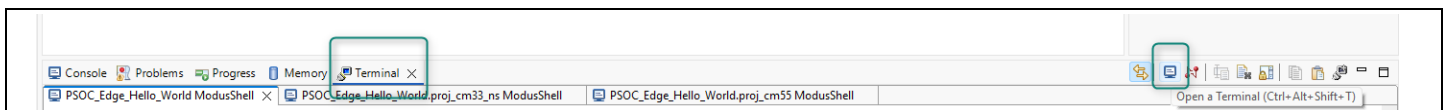


Figure 59 Eclipse IDE Terminal Selection

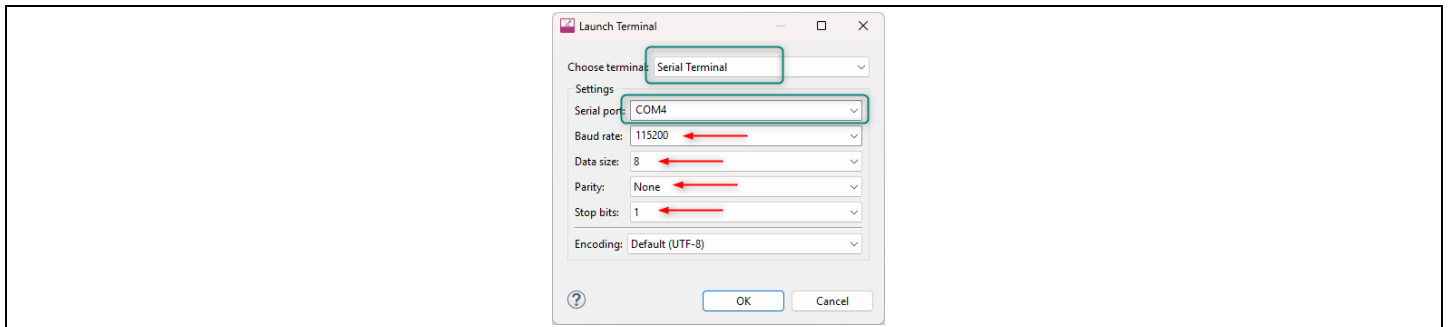


Figure 60 Eclipse IDE Terminal Serial Configuration

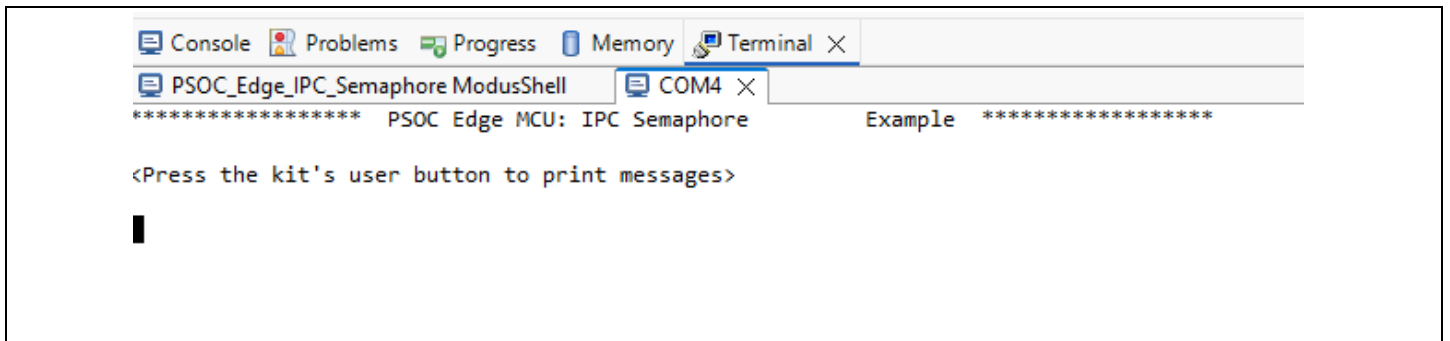


Figure 61 Eclipse IDE Terminal Serial Communication from the EVK showing UART IPC Header

44. Confirm the USER_LED is not illuminated, indicating the application was built utilizing a semaphore to control access to the UART. Press the USER_BTN1, and observe the messages in the serial terminal window. Since a semaphore is being used, no mixing of messages is observed and the messages alternate cleanly between the cm33_ns and cm55 code bases.

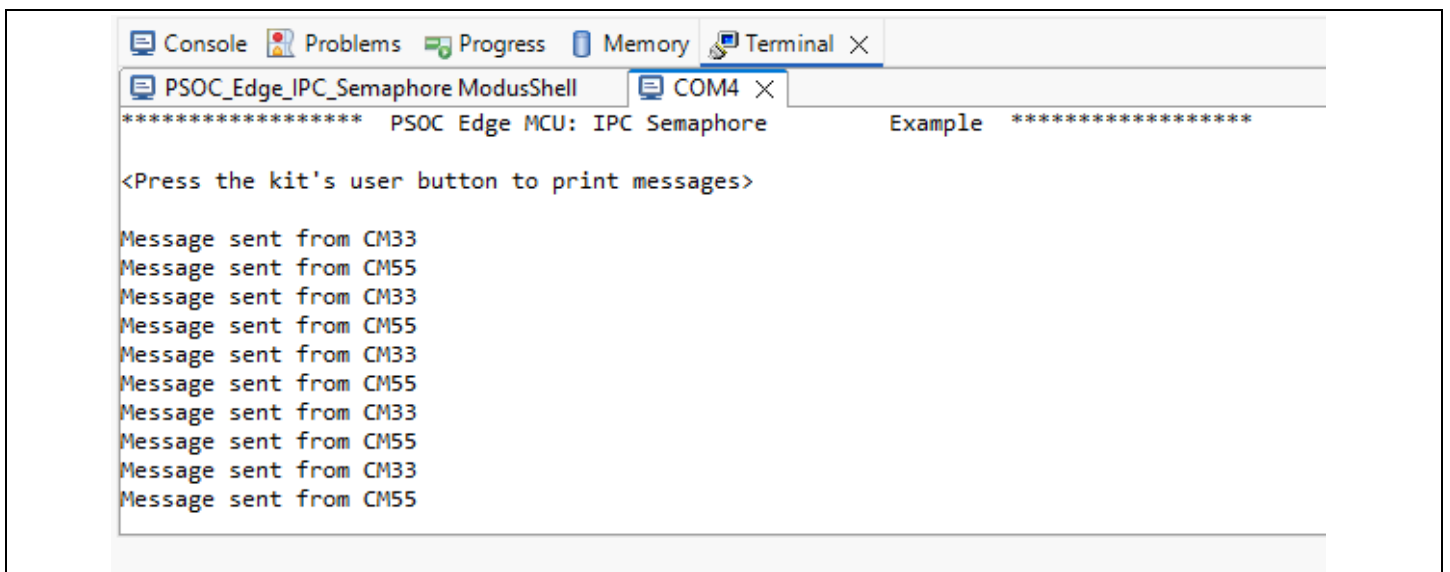


Figure 62 Clean UART messages from cm33 and cm55 when using a semaphore

Code Example 2: IPC Semaphore using PDL

45. Now, let's turn off the use of semaphores to manage access to the shared UART, build the application and observe the results on the serial terminal.

Using the Project Explorer navigate to and open the ipc_def.h file

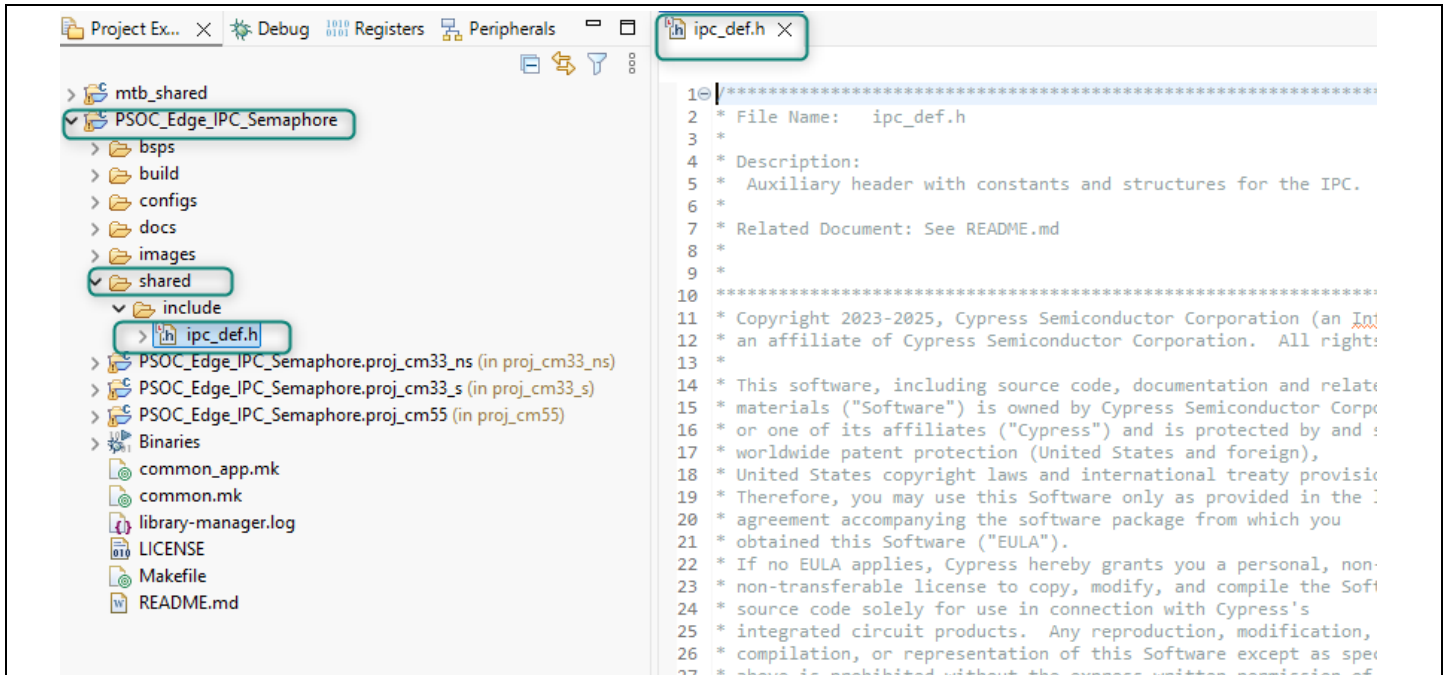


Figure 63 Open the shared > include > ipc_def.h header file

Scroll down and find the ENABLE_SEMA macro and change its value from 1U to 0U

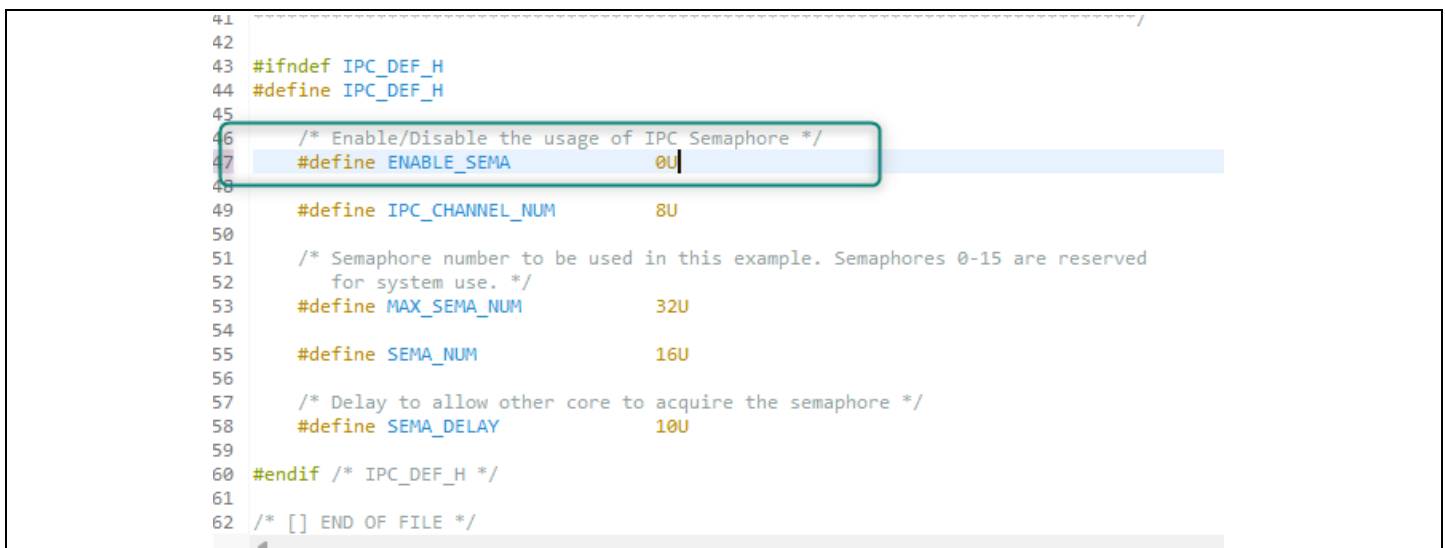


Figure 64 Update ENABLE-SEMA macro value to 0

Save and Close the file. Select the PSOC_Edge_IPC_Semaphore application folder, then build and launch the application

46. Confirm the USER_LED is now illuminated, indicating the application was built *without* utilizing a semaphore to control access to the UART. **Press** the USER_BTN1, and observe the messages in the serial terminal window. Since a semaphore is no longer being used, we now observe mixing of the messages as they get transmitted by both the cm33_ns and cm55 code bases.

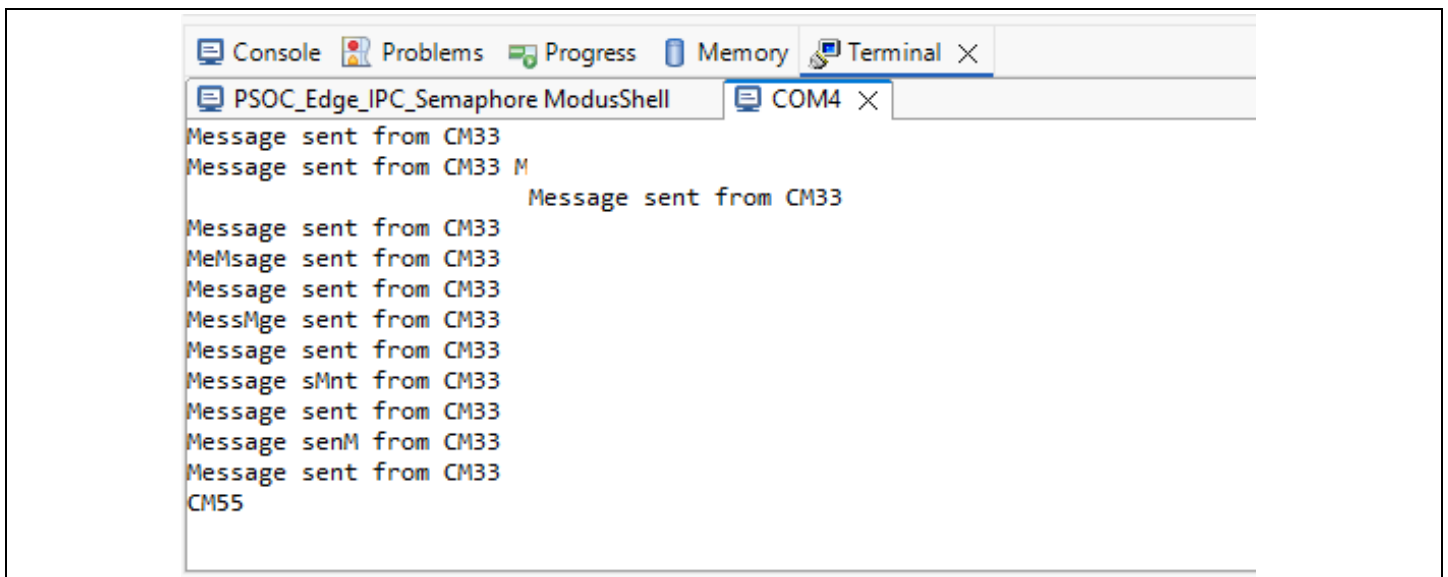


Figure 65 Mixed UART messages from cm33 and cm55 when **not** using a semaphore

5.6 Conclusion

We successfully created a PSOC™ Edge E84 IPC PDL Semaphore application using ModusToolbox™, figured out how to use IPC PDL Libraries and access the same resource, in this example the debug UART, from two cores using the IPC PDL Semaphore layer.

Edition 2025-11-11
Published by
Infineon Technologies AG
81726 Munich, Germany

© 2025 Infineon Technologies AG.
All Rights Reserved.

Do you have a question about this document?

Email: erratum@infineon.com

Document reference
AppNote Number

IMPORTANT NOTICE

The information contained in this application note is given as a hint for the implementation of the product only and shall in no event be regarded as a description or warranty of a certain functionality, condition or quality of the product. Before implementation of the product, the recipient of this application note must verify any function and other technical information given herein in the real application. Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind (including without limitation warranties of non-infringement of intellectual property rights of any third party) with respect to any and all information given in this application note.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer's technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

For further information on the product, technology delivery terms and conditions and prices please contact your nearest Infineon Technologies office (www.infineon.com).

WARNINGS

Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies' products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.